

**Evaluierung hardwarebasierter IPsec
Verschlüsselungssysteme**
**Performance von Hardware-Offloading mit der Mellanox
ConnectX-6 Dx SmartNIC**

Bachelorarbeit

von

Marc Fabian Lindner



Universität Potsdam

Institut für Informatik und Computational Science
Professur Betriebssysteme und Verteilte Systeme

Aufgabenstellung und Betreuung:

Prof. Dr. Bettina Schnor

Dr. Bernhard Heep

Potsdam, den 21. August 2022

Lindner, Marc Fabian

marlindner@uni-potsdam.de

Evaluierung hardwarebasierter IPsec Verschlüsselungssysteme

Bachelorarbeit, Institut für Informatik und Computational Science

Universität Potsdam, August 2022

Ich bedanke mich bei meiner Familie, die mich immer auf meinem Weg unterstützt haben. Außerdem bedanke ich mich bei Prof. Schnor, die mich während der Arbeit betreut hat und mir immer hilfreiche Vorschläge gegeben hat. Ich möchte mich außerdem bei Rohde & Schwarz Cybersecurity bedanken, bei denen ich diese Arbeit geschrieben habe und ohne deren Unterstützung diese Arbeit nicht möglich gewesen wäre. Besonderen Dank möchte ich dabei an Bernhard Heep und Martin Wilske richten, die mich während der Arbeit betreut haben und immer Zeit hatten, mir Feedback zu geben und aktuelle Probleme zu besprechen. Mein Dank gilt auch allen anderen Mitarbeitern für ihre freundliche und stets hilfsbereite Art. Zuletzt möchte ich mich bei allen meinen Freunden bedanken, ohne die die letzten paar Monate deutlich langweiliger gewesen wären.

Selbständigkeitserklärung

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbständig angefertigt, nicht anderweitig zu Prüfungszwecken vorgelegt und keine anderen als die angegebenen Hilfsmittel verwendet habe. Sämtliche wissentlich verwendeten Textausschnitte, Zitate oder Inhalte anderer Verfasser wurden ausdrücklich als solche gekennzeichnet.

Potsdam, den 21. August 2022

Marc Fabian Lindner

Zusammenfassung

Die vorliegende Arbeit befasst sich mit der Evaluierung von Hardware-Offloading von IPsec-Verschlüsselungen auf SmartNICs, also der Beschleunigung von IPsec durch Auslagerung der Verschlüsselung auf spezielle Hardware. Als SmartNIC wird in dieser Arbeit die Mellanox ConnectX-6 Dx Netzwerkkarte verwendet. Besonderer Fokus liegt dabei auf dem Vergleich des Durchsatzes bei Nutzung von Hardware-Offloading und Erweiterungen des CPU-Instruktionssatzes um spezielle Instruktionen zur Verschlüsselung. Zum Zwecke der Evaluierung erfolgt eine Analyse der Testumgebung und es werden experimentelle Messungen durchgeführt und deren Ergebnisse dargestellt und diskutiert. Es wird gezeigt, dass Hardware-Offloading dabei eine bessere Performance erzielt als die Nutzung von speziellen CPU-Instruktionen, aber die Nutzung von IPsec immer noch deutliche Performanceeinbuße hat. Mögliche Ursachen für eine Verbesserung oder Verschlechterung des Durchsatzes, sowie Defizite der SmartNIC und des Hardware-Offloading, werden erläutert. Es werden außerdem die vorgenommenen Modifikationen und Konfigurationen aufgeführt, die durch die Messungen identifiziert wurden, um einen möglichst optimalen Durchsatz zu erreichen.

Inhaltsverzeichnis

1	Einleitung	1
2	Grundlagen	3
2.1	Ethernet-Netzwerke	3
2.2	Internet Protocol	5
2.3	Virtuelle Private Netzwerke	7
2.4	IPsec	9
2.4.1	Security Associations und Security Policies	9
2.4.2	IKEv2 Protokoll	10
2.4.3	Encapsulating Security Payload	12
2.5	Implementierung von IPsec in Linux	14
2.5.1	StrongSwan's IKE-Daemon charon	15
2.5.2	XFRM Framework	15
2.6	AES-GCM Verschlüsselung	15
2.6.1	Advanced Encryption Standard	15
2.6.2	Galois/Counter Mode	18
2.7	Hardware-Offloading mit IPsec	20
3	Messplan	22
3.1	Aufbau und Erläuterung der Testumgebung	22
3.1.1	Spezifikation der Testcomputer	22
3.1.2	Spezifikation des Testgeräts	23
3.1.3	Aufbau des Netzwerkes	24
3.1.4	Funktionsweise des Systems	25
3.2	Limitierungen der Testumgebung	26
3.3	Analyse des Testsystems	27
3.4	Messwerte	29
3.5	Parameter und Faktoren	29
3.6	Durchgeführte Messungen	31
4	Messergebnisse und Auswertung	34
4.1	Variable Senderate	34
4.2	Vergleich verschiedener AES-GCM Implementierungen	37
4.2.1	Verarbeitung durch IPsec	41
4.2.2	Verschlüsselung mit AES-GCM	41
4.2.3	Besonderheiten bei IP-Fragmentierung	42
4.2.4	Tests mit IMix	43
4.3	Parallelisierung	45

5	Aufgetretene Probleme	51
5.1	Inkompatibilitäten mit alter Hardware	51
5.2	Unterschiedliche Treiber Versionen	51
5.3	Überhitzung der Karten	52
5.4	Nicht standardmäßige MTU	52
5.5	Loopback-Port mit IPv6-Adressen	53
6	Fazit und Ausblick	54
A	StrongSwan Konfiguration	56
B	Weitere Messergebnisse	58
	Abbildungsverzeichnis	60
	Tabellenverzeichnis	61
	Glossar	62
	Literaturverzeichnis	64

1 Einleitung

Mit zunehmender Digitalisierung wächst auch die Menge der über das Internet übertragenen Daten immer weiter an. Um diese stetig steigende Datenmenge übertragen zu können, sind Netzwerkschnittstellen mit Übertragungsraten von bis zu 100 Gbit/s nötig. Es wird allerdings auch, insbesondere mit Hinblick auf die Datensicherheit und den Datenschutz, immer wichtiger, diese Daten auch gesichert zu übertragen. Zu diesem Zweck werden Daten verschlüsselt übertragen. Eine Möglichkeit dies zu realisieren ist durch Virtuelle Private Netzwerke (VPNs). IPsec ist eine standardisierte Architektur, bestehend aus mehreren Komponenten, welche häufig für den Aufbau von Virtuellen Privaten Netzwerken genutzt wird.

Die Verschlüsselung von Daten ist allerdings aufwendig und schränkt damit die Menge an übertragbaren Daten stark ein. Insbesondere bei Routern die eine große Menge an Daten übertragen müssen oder denen nur eingeschränkte Ressourcen zur Verfügung stehen, ist die Verschlüsselung der Pakete mit IPsec eine große Einschränkung. Die Menge an übertragbaren Daten wird dadurch noch zusätzlich begrenzt.

Hier zeigt sich, dass Software-Implementierungen der Verschlüsselung zu langsam für den praktischen Gebrauch sind. Um sowohl die Anforderung nach einer schnellen und nach einer sicheren Übertragung großer Datenmengen zu gewährleisten gibt es spezielle Hardware, um die Verschlüsselung zu beschleunigen. Da Hardware-Implementierungen von Algorithmen für gewöhnlich schneller und effizienter sind als Implementierungen in Software, versprechen diese Implementierungen deutlich schneller als herkömmliche Software-Implementierungen zu sein.

Heutzutage stehen dabei besonders zwei Hardware-Implementierungen im Vordergrund: CPUs mit AES-NI und SmartNICs. CPUs mit AES-NI (Advanced Encryption Standard New Instructions) verfügen über besondere Instruktionen, welche die Verschlüsselung von Daten mit dem AES Verfahren in deutlich weniger CPU-Zyklen durchführen können, als herkömmliche Implementierungen ohne die Nutzung dieser Instruktionen. Bei SmartNICs (kurz für Smart Network Interface Cards) hingegen handelt es sich um Netzwerkkarten, welche über spezielle Hardware verfügen und somit die Verschlüsselung von Daten direkt übernehmen können.

Es gibt allerdings bisher nur wenig Daten, welche beide Implementierungen direkt miteinander vergleichen. Das Ziel dieser Arbeit ist es, die Performance bei der Übertragung von verschlüsselten Daten einmal mit Unterstützung durch eine SmartNIC und einmal durch die Nutzung von AES-NI zu vergleichen. Als SmartNIC wurde dafür die Mellanox ConnectX-6 Dx gewählt, welche unter anderem Verschlüsselungen mit dem AES-GCM Algorithmus für IPsec-Tunnel durchführen kann. Als Testplattform wurden Linux-PCs mit Intel i7 CPUs der 9. und 10. Generation gewählt. Beide genannten Prozessoren verfügen über den AES-NI Instruktionssatz.

Da die Mellanox ConnectX-6 Dx laut ihrem Datenblatt¹ Übertragungsraten von bis zu 100 Gbit/s ermöglicht, ist ein weiteres Ziel dieser Arbeit zu prüfen, ob diese Werte in der Praxis tatsächlich erreicht werden können und bei der Verschlüsselung von Paketen auch gehalten werden

¹<https://www.mellanox.com/files/doc-2020/pb-connectx-6-dx-en-card.pdf>

können.

Da bei der Übertragung von kleinen Datenpaketen in der Praxis häufig deutlich geringere Übertragungsraten erzielt werden, soll der Vergleich auch mit verschiedenen Größen von Datenpaketen erfolgen. Insbesondere soll deswegen überprüft werden, ob durch die Unterstützung durch SmartNICs konstantere Übertragungsraten erzielt werden können.

Um die gesetzten Ziele der Arbeit zu erfüllen, werden experimentelle Messungen durchgeführt. Zu diesem Zweck wird ein einfaches Testsystem aufgebaut, welches die Funktionalität eines realen Systems nachbildet. Im Testsystem wird unter anderem ein IPsec-Tunnel aufgebaut und Daten, welche durch Netzwerktester dynamisch erzeugt werden, mit dem AES-GCM Algorithmus verschlüsselt übertragen.

Kapitel 2 führt die benötigten Begriffe und Konzepte ein, die für das Verständnis der restlichen Arbeit notwendig sind. Dieses Kapitel ist besonders für Leser gedacht, die mit der Thematik von IP-Netzwerken und IPsec nicht vertraut sind.

Der Messplan, welcher unter anderem den Aufbau und die Konfiguration des Testsystems und eine Beschreibung der durchgeführten Messungen enthält, wird in Kapitel 3 erläutert. In diesem Kapitel wird auch die verwendete SmartNIC näher erläutert und die erhobenen Messdaten und variablen Faktoren der Messungen eingeführt. Hier wird außerdem eine erste Analyse des Testsystems erfolgen und die erwarteten Messergebnisse besprochen.

Die aus den Messungen erhobenen Daten werden anschließend in Kapitel 4 vorgestellt und ausgewertet. Dabei werden mögliche Gründe und Ursachen für Performancegewinne und -verluste erläutert.

Zusätzlich erfolgt in Kapitel 5 eine Darstellung von Problemen und Hindernissen die während der Bearbeitung der Problemstellung aufgetreten sind. Abschließend werden in Kapitel 6 die Ergebnisse der Arbeit zusammengefasst und ein Fazit zur Problemstellung gezogen. Außerdem wird hier ein Ausblick auf mögliche weitere Arbeiten und noch offene Probleme und Fragen gegeben.

2 Grundlagen

In diesem Kapitel wird das erforderliche Grundlagenwissen eingeführt, welches für das Verständnis der restlichen Arbeit nötig ist. Es wird dabei mit einer kurzen Einführung in Ethernet und IP-Netzwerke begonnen. Anschließend wird das Konzept der Virtuellen Privaten Netzwerke besprochen und in diesem Kontext kurz die Protokollsuite IPsec, zusammen mit der hier verwendeten Implementierung StrongSwan, vorgestellt. Am Ende gibt es eine kurze Einführung in die Idee des Hardware-Offloading, in der unter anderem Vorteile von Hardware-Offloading gegenüber Software-Implementierungen besprochen werden. Abschließend folgt eine kurze Einführung in den in dieser Arbeit verwendeten Verschlüsselungsalgorithmus AES-GCM.

2.1 Ethernet-Netzwerke

Ethernet ist eine Netzwerktechnologie, welche ursprünglich von PARC¹ (Palo Alto Research Center; ehemals als Xerox PARC bekannt) entwickelt wurde, die es ermöglicht mehrere Computer über einen Kabelanschluss zu einem Netzwerk zusammenzuschließen. So miteinander verbundene Computer können dann miteinander kommunizieren und Daten untereinander austauschen. Ethernet wurde erstmals 1983 unter dem Namen IEEE 802.3[iee84] standardisiert. Seitdem hat die Ethernet-Technologie viele Erweiterungen und Überarbeitungen erhalten. Die neueste Version wurde 2018 veröffentlicht und ist IEEE 802.3-2018[iee18] und erlaubt Übertragungsgeschwindigkeiten von bis zu 400 Gbit/s.

Jeder Computer (oder genauer jede Ethernet-Schnittstelle) in einem Ethernet-Netzwerk benötigt eine eigene Adresse, auch MAC-Adresse (media access control address) genannt. Jede MAC-Adresse muss dabei eindeutig eine Schnittstelle identifizieren können und darf somit in einem Netzwerk nicht doppelt vorkommen. MAC-Adressen sind meistens fest in die Hardware integriert und werden deswegen auch physische oder Hardware-Adresse genannt, allerdings gibt es auch Hardware bei denen die MAC-Adresse veränderbar ist. Eine MAC-Adresse ist dabei 48 Bit lang und wird typischerweise im Hexadezimal-Format, in 2er Paaren dargestellt. Ein Beispiel für eine MAC-Adresse ist E8:2A:44:94:4A:EF. (Es werden teils auch andere Trennsymbole verwendet.)

Die Kommunikation in einem Ethernet-Netzwerk erfolgt dabei durch Ethernet-Frames. Ein Ethernet-Frame besteht dabei aus einem Header (deutsch: Kopf), dem Payload (deutsch: Nutzdaten) und abschließend einer Prüfsumme. Der Payload enthält die zu übertragene Daten, meist sind dies Pakete, Frames oder Daten von wiederum anderen Protokollen. Der Header wiederum enthält die Sender- und Empfänger-MAC-Adresse, sowie die Länge des Payloads. Die Prüfsumme wird genutzt um den gesamten Ethernet-Frame auf Fehler in der Übertragung zu überprüfen. In Tabelle 2.1 ist eine Darstellung eines Ethernet-Frames zu finden.

In aktuellen Ethernet-Netzwerken werden die verschiedenen Schnittstellen durch sogenannte Hubs oder Switches verbunden. Eine Abbildung eines einfachen Ethernet-Netzwerkes ist in Ab-

¹<https://www.parc.com/>

Empfänger-MAC	Sender-MAC	Länge	Payload	Prüfsumme
6 Bytes	6 Bytes	2 Bytes	46-1500 Bytes	2 Bytes

Tabelle 2.1: Aufbau eines Ethernet-Frames im OSI-Layer 2 (Data link layer).

bildung 2.1 zu finden. Ethernet-Hubs leiten empfangene Frames an alle angeschlossenen Schnittstellen weiter. Wenn dabei ein Ethernet-Frame empfangen wird, bei dem die Empfänger-MAC-Adresse sich von der eigenen MAC-Adresse unterscheidet, wird der entsprechende Ethernet-Frame ignoriert. Hubs belasten allerdings das Ethernet-Netzwerk stark, da sie jeden Frame für jede angeschlossene Verbindung duplizieren und somit viel unnötigen Datenverkehr erzeugen. Ethernet-Switches wiederum agieren dabei intelligenter und leiten Ethernet-Frames nur an die Schnittstelle mit der übereinstimmenden MAC-Adresse weiter, insofern bekannt ist, an welchem Port des Switches diese angeschlossen ist. Zu diesem Zweck lesen Switches die Absender-MAC-Adressen von allen eingehenden Frames und speichern dabei, von welchem Port welche MAC-Adresse kam. Wenn später ein Frame mit einer bereits bekannten Empfänger-MAC-Adresse beim Switch ankommt, kann das Paket nun an den korrekten Port weitergeleitet werden, anstatt an alle Ports.

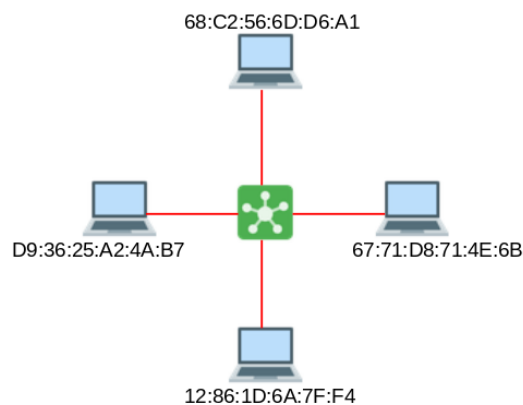


Abbildung 2.1: Eine Darstellung eines einfachen Ethernet-Netzwerkes, mit vier Computern (außen), welche durch einen Ethernet-Hub (mitte) verbunden sind. Statt eines Ethernet-Hubs könnte auch ein Ethernet-Switch verwendet werden.

Ethernet besitzt auch die Möglichkeit die Übertragungsgeschwindigkeit bei einer Überlastung zu reduzieren. Dies wird auch Flusskontrolle genannt. Dies ist besonders relevant, wenn ein Computer mehr Daten empfängt als er aktuell verarbeiten kann. Im Normalfall würden Frames, die er nicht mehr zwischenspeichern kann, einfach verloren gehen. Mit der Einführung der Flusskontrolle kann ein Empfänger aber ein sogenanntes Pause-Frame verschicken, welches dem Sender der Daten mitteilt, die Übertragung von Frames für eine kurze Zeit zu pausieren. Dadurch erhält der Empfänger mehr Zeit die empfangenen Frames zu verarbeiten und die Anzahl der verlorenen Frames wird reduziert. Die gewünschte Dauer der Unterbrechung wird vom Empfänger im

Pause-Frame eingetragen.

2.2 Internet Protocol

Das Internet Protocol (kurz: IP) dient dazu, Daten zwischen verschiedenen Netzwerken zu versenden. Es entwickelte sich ursprünglich aus dem unter [CK74] vorgestellten Protokoll für den Austausch von Netzwerkpaketen zwischen Computern. Es gibt zwei Versionen vom Internet Protocol die heute üblicherweise genutzt werden: IPv4, welches aktuell im RFC 791[Pos81] (mit dem letzten Update im RFC 6864[Tou13]) standardisiert ist, und das neuere IPv6, welches aktuell im RFC 8200[DH17] standardisiert ist. Die Versionen 1 bis 3, sowie Version 5, sind alle Experimentell und haben in der Praxis bisher keine Verwendung gefunden.

Das Internet Protocol versendet Daten in IP-Paketen. IP-Pakete enthalten neben den Daten die sie transportieren ebenfalls einen Header, welcher unter anderem die Sender- und Empfänger-Adresse (siehe unten) enthält. Eine Darstellung der Header der IPv4-Pakete und IPv6-Pakete findet sich in Abbildung 2.2.

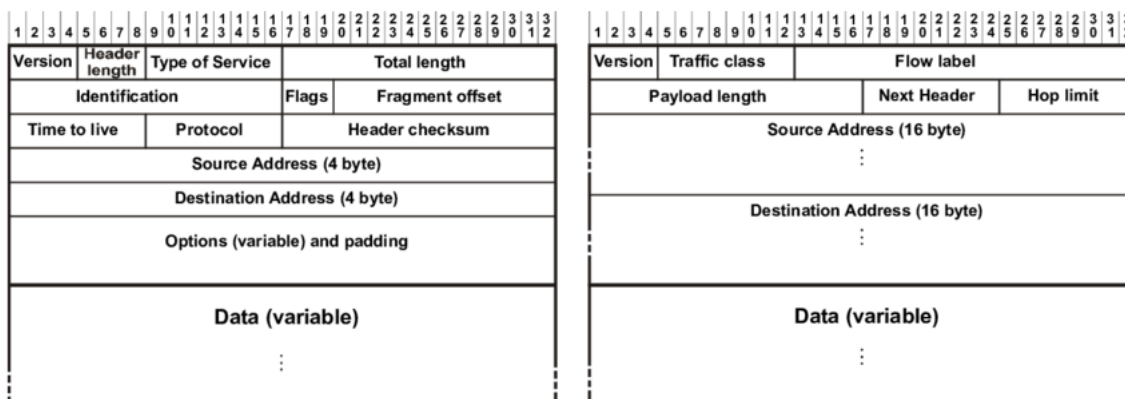


Abbildung 2.2: Eine Darstellung von IPv4 (links) und IPv6 (rechts) Paketheader.

Quelle:

https://www.researchgate.net/figure/IPv4-and-IPv6-packet-format_fig5_34930048, Zugriffen am 16. April 2022, 14:04

Im Internet Protocol wird jeder Computer (oder genauer jedes Interface) durch eine eigene IP-Adresse identifiziert. In IPv4 ist eine IP-Adresse eine 32 Bit (4 Byte) Zahl und erlaubt damit rund $2^{32} \approx 4,3 \cdot 10^9$ Adressen. Da sich dies mit der Zeit allerdings als zu wenig herausgestellt hat, wurde die Größe von Adressen in IPv6 auf 128 Bit und der damit verfügbare Adressraum auf $2^{128} \approx 3,4 \cdot 10^{38}$ Adressen angehoben. Für die Darstellung einer IPv4-Adresse werden die vier Byte als Dezimalzahlen, getrennt durch Punkte, angegeben. In IPv6 wird die Adresse in 8 Zahlen, getrennt durch Doppelpunkte, aufgeteilt, welche jeweils 16 Bit entsprechen und im Hexadezimalformat angegeben werden. Ein Beispiel für eine IPv4-Adresse ist 192.168.178.32 und für eine IPv6-Adresse 2001:16b8:4636:a700:b68a:626f:eeb:6524. Anders als bei Ethernet, ist die IP-Adresse aber meist nicht fest vorgegeben und eine Schnittstelle kann auch mehrere IP-Adressen zugewiesen haben.

Jede IP-Adresse ist dabei in einen Netzwerkteil und einen Hostteil (auch Geräteteil genannt)

aufgeteilt. Der Netzwerkteil identifiziert dabei ein Netzwerk (auch Subnetz genannt) und muss somit für alle Schnittstellen im gleichen Netzwerk identisch sein. In einem Netzwerk muss jede Schnittstelle aber einen eigenen Hostteil haben. Die Aufteilung von IP-Adressen nach Netzwerk- und Hostteil wird aktuell durch das Classless Inter-Domain Routing (CIDR) spezifiziert. Dieses löst die älteren Netzklassen ab, die ursprünglich mit IP verwendet wurden, da diese sich als zu in-flexibel herausgestellt haben. Die neuste Version von CIDR ist im RFC 4632[FL06] dokumentiert.

Um zu signalisieren welcher Teil der IP-Adresse der Netzwerkteil ist, wird neben der IP-Adresse auch noch die Subnetz-Maske benötigt. Die Subnetz-Maske ist eine Bitmaske, die genauso lang wie die entsprechende IP-Adresse ist, hat aber die zusätzliche Einschränkung, dass 1er Bits nur am Anfang (auf der linken Seite, bei den höchstwertigsten Bits) genutzt werden dürfen. Sie wird außerdem wie IP-Adressen angegeben. Eine Subnetz-Maske für eine IPv4-Adresse ist zum Beispiel 255.255.255.0. Meist wird die Subnetz-Maske häufig nur durch die Anzahl der 1er Bits am Anfang angegeben und als Suffix an die IP-Adresse, getrennt durch einen Backslash, angefügt. Ein Beispiel für eine IPv4-Adresse mit obiger Subnetz-Maske als Suffix ist 192.168.178.32/24. Die Subnetz-Maske gibt an, welche Bits der IP-Adresse zum Netzwerkteil gehören; der Rest der IP-Adresse ist entsprechend der Hostteil. Im Beispiel oben wären also die ersten 24 Bits der IP-Adresse (also 192.168.178) der Netzwerkteil und somit die letzten 8 Bit (also 32) der Hostteil. In diesem Beispiel können im Netzwerk also bis zu $2^8 = 256$ IP-Adressen vergeben werden und somit bis zu 256 Schnittstellen gleichzeitig existieren (von diesen sind allerdings zwei Adressen für die sogenannte Netzwerkadresse und Broadcast-Adresse reserviert und können nicht an Hosts vergeben werden).

Wenn Daten an eine Schnittstelle im gleichen Netzwerk gesendet werden sollen, also an eine IP-Adresse mit identischem Netzwerkteil, wird das IP-Paket direkt an die entsprechende Schnittstelle gesendet.

Wenn der Netzwerkteil einer Empfänger-IP-Adresse nicht mit dem Netzwerkteil des eigenen Netzwerkes übereinstimmt, muss das IP-Paket entsprechend an das andere Netzwerk weitergeleitet werden. Diese Aufgabe übernehmen in IP-Netzwerken Router (in manchen Kontexten auch Gateways genannt). Die Router verschiedener Netzwerke können miteinander verbunden werden und kommunizieren ebenfalls über das Internet Protocol miteinander. Router verbinden somit verschiedene Netzwerke und sind die Schnittstelle, um Daten an andere Netzwerke zu senden. Um ein IP-Paket an ein anderes Netzwerk zu schicken, wird das IP-Paket zuerst an einen Router im eigenen Netzwerk gesendet. Dieser leitet das IP-Paket dann wiederum an andere Router weiter, bis es bei einem Router im entsprechenden Ziel-Netzwerk ankommt und von diesem dann an den finalen Empfänger geschickt werden kann. Im Unterschied zu Ethernet ermöglicht das Internet Protocol damit auch die Kommunikation zwischen verschiedenen Netzwerken miteinander. Abbildung 2.3 zeigt eine Darstellung eines einfachen IP-Netzwerkes.

Um zu entscheiden an welchen Router das Paket weitergeleitet werden soll, hat jeder Router eine Routing-Tabelle. Mithilfe der Routing-Tabelle kann für jede IP-Adresse der nächste benötigte Router (häufig auch Next-Hop genannt) ermittelt werden. Die Routing-Tabelle kann statisch festgelegt werden oder dynamisch durch diverse Routing-Protokolle (wie zum Beispiel Open Shortest Path First, kurz: OSPF, definiert in RFC 2328[Moy98]) erzeugt werden.

Die IP-Adressen (genauer der Hostteil) der Geräte können ebenfalls statisch festgelegt werden oder dynamisch durch ein Protokoll wie DHCP (Dynamic Host Configuration Protocol, definiert in RFC 2131[Dro97]) erzeugt und vom eigenen Router vergeben werden.

Das Internet Protocol baut heutzutage typischerweise auf dem Ethernet-Protokoll auf und ver-

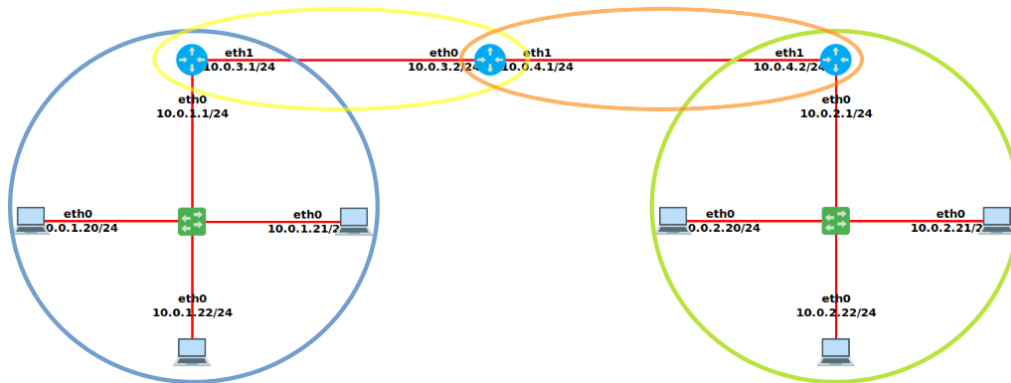


Abbildung 2.3: Eine Darstellung eines einfachen IP-Netzwerkes. Zu sehen sind vier verschiedene IP-Netzwerke (farbigen Kreise), welche durch drei Router (oben) verbunden sind. Jeder Router ist Teil von zwei IP-Netzwerken und hat zwei Schnittstellen (eth0 und eth1). Jedes Netzwerk hat einen eigenen Hostteil. Daten können vom blauen in das grüne Netzwerk gesendet werden, indem sie über die Router durch das gelbe und orangene Netzwerk gesendet werden.

sendet IP-Pakete innerhalb von Ethernet-Frames. Ein IP-Netzwerk (oder Subnetz) ist somit identisch zu einem Ethernet-Netzwerk. Um die MAC-Adresse einer Schnittstelle im eigenen Netzwerk anhand der IP-Adresse zu finden, wird das Address Resolution Protocol (kurz: ARP, definiert in RFC 826[Plu82]) genutzt. Wenn ein IP-Paket mehrere Geräte (bzw. Router) passieren muss, wird das IP-Paket dabei für jede genutzte Verbindung in einem neuen, eigenen Ethernet-Frame versendet.

2.3 Virtuelle Private Netzwerke

Virtuelle Private Netzwerke (VPNs) bezeichnen einen Zusammenschluss mehrerer Netzwerke oder Geräte miteinander. Die betroffenen Geräte bilden dabei gemeinsam ein virtuelles Netzwerk, welches nicht physisch existiert, und agieren dabei, als würden sie sich alle gemeinsam in diesem Netzwerk befinden. Dies ermöglicht es Geräten, die sich nicht im gleichen physischen Netzwerk befinden, trotzdem auf die Ressourcen der anderen Netzwerke oder der anderen Geräte zuzugreifen. Je nach Art des genutzten VPNs wissen Anwendungen nicht einmal unbedingt, dass sie in einem virtuellen Netzwerk agieren und behandeln es identisch zu physischen Netzwerken.

Wenn Daten zwischen Netzwerken übertragen werden müssen, werden diese meist verschlüsselt, um die Vertraulichkeit der Daten auch außerhalb des VPNs sicherzustellen. Zusätzlich werden übertragene Daten meist auch durch kryptographisch geschützte Prüfsummen und Integritätschecks beim Empfang vor Änderungen geschützt und die Sender der Daten authentifiziert. Das bedeutet, andere Geräte, welche nicht Teil des VPN sind, können die übertragenen Daten nicht lesen oder unbemerkt Änderungen durchführen und auch selbst keine Daten senden. Die Daten im VPN sind also vor Manipulation von außerhalb des VPN geschützt und somit privat.

Es gibt verschiedene Formen von VPNs. Bei Site-to-Site (deutsch: Netzwerk-zu-Netzwerk) VPNs handelt es sich um VPNs, welche mehrere Netzwerke oder Subnetze miteinander verbin-

den, sodass diese dann als ein einziges Netzwerk agieren können. Bei Host-to-Host VPNs handelt es sich um VPNs welche mehrere Endgeräte verbinden. Diese Endgeräte agieren dann, als würden sie sich alle in einem Netzwerk befinden. Zuletzt gibt es noch Host-to-Site VPNs, welche ein Endgerät in ein bereits vorhandenes Netzwerk einbinden.

Eine Darstellung von VPNs ist in Abbildung 2.4 zu finden. In dieser Arbeit werden nur Site-to-Site VPNs betrachtet.

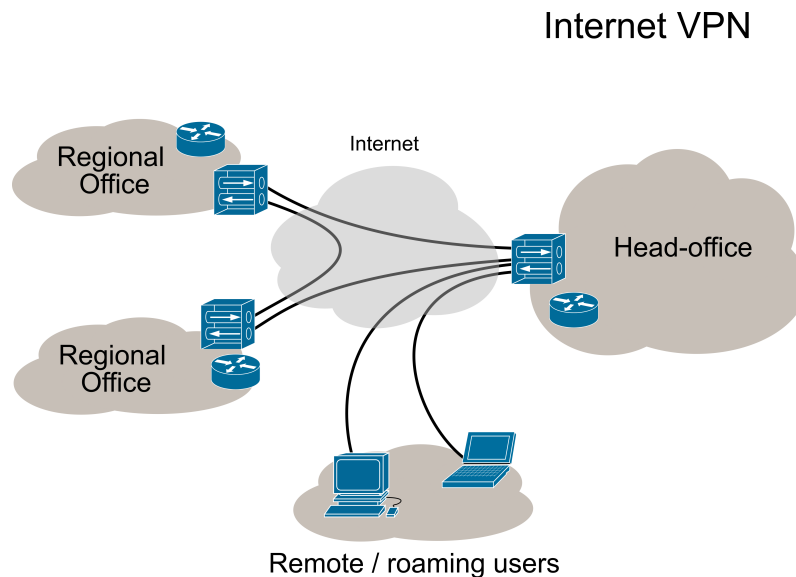


Abbildung 2.4: Darstellung von Site-to-Site VPNs (Verbindungen zwischen Remote Office und dem Head-Office) und Host-to-Site VPNs (für den Zugriff von Remote Nutzern auf das Netzwerk des Head-Office). Die Daten werden über das Internet übertragen.

Es gibt viele Möglichkeiten VPNs umzusetzen. Das OpenVPN Projekt² nutzt zum Beispiel das TLS Protokoll (Transport Layer Security, definiert in RFC 8446[Res18]) um VPNs zu realisieren. Eine häufig genutzte Implementierung von VPNs ist Cisco AnyConnect³, welche das DTLS Protokoll (Datagram Transport Layer Security, definiert in RFC 9147[RTM22]) nutzt. Eine relativ neue Implementierung ist WireGuard⁴, welche seit 2020 im Linux Kernel implementiert ist und sich als Ziel gesetzt hat, deutlich einfacher als andere VPN-Lösungen zu sein. In dieser Arbeit wird die IPsec Protokollsuite (siehe Abschnitt 2.4) genutzt um VPNs zu erstellen. Die genutzte Implementierung von IPsec ist StrongSwan (siehe Teilabschnitt 2.5.1) und das XFRM Framework im Linux Kernel (siehe Teilabschnitt 2.5.2).

²<https://openvpn.net/>

³<https://www.cisco.com/c/en/us/products/security/anyconnect-secure-mobility-client/index.html>

⁴<https://www.wireguard.com/>

2.4 IPsec

IPsec ist eine Sammlung von mehreren Protokollen, mit dem Ziel Virtuelle Private Netzwerke (VPNs) zur sicheren Übertragung von Daten zu erstellen. Die einzelnen Komponenten sind in unterschiedlichen RFCs spezifiziert und in RFC 4301[KS05] ist ein Gesamtüberblick von IPsec enthalten. Site-to-Site VPNs werden von IPsec durch IPsec-Tunnel zwischen zwei (kryptographischen) Endpunkten, welche auch IPsec-Gateways genannt werden, realisiert. ESP und AH operieren dafür im sogenannten Tunnel-Mode, weshalb die Nutzung von IPsec-Tunneln auch einfach Tunnel-Mode genannt wird. Auch für Host-to-Site VPNs wird der Tunnel-Mode genutzt, nur ist hierbei einer der kryptographischen Endpunkte auch ein Kommunikationspartner. Für Host-to-Host VPNs wird normalerweise der Transport-Mode genutzt, da beide kryptographischen Endpunkte auch die Kommunikationspartner sind und somit eine Änderung der IP-Header nicht nötig ist. Allerdings ist auch hier die Nutzung des Tunnel-Mode möglich. Für die Unterschiede beider Modi siehe auch Teilabschnitt 2.4.3. IPsec befindet sich dabei auf dem Internet Layer und arbeitet somit direkt auf IP-Paketen. Alle IP-Pakete, die einen IPsec-Tunnel passieren müssen, werden dabei von IPsec gekapselt (zu übertragende Pakete werden in die Payloads anderer Pakete geschrieben).

Zum Aufbau von Tunneln und dem Austausch des benötigten Schlüsselmaterials wird das Protokoll Internet Key Exchange (IKE) oder dessen Nachfolger IKEv2 genutzt. Zur eigentlichen Kapselung und Sicherung von IP-Paketen werden Encapsulating Security Payloads (ESP) oder Authentication Headers (AH) genutzt. Beide können dabei in einem Tunnel- oder Transport-Mode arbeiten. Zum Speichern von Schlüsseln und weiteren benötigten Informationen für die IPsec-Tunnel werden Security Associations (SA) und Security Policies (SP) genutzt.

In diesem Kapitel werden die Grundlagen von IPsec und die erwähnten Protokolle kurz erläutert. In Abschnitt 2.5 folgt anschließend eine Beschreibung der Implementierung von IPsec im Linux Kernel.

2.4.1 Security Associations und Security Policies

Security Associations (SAs) und Security Policies (SPs) werden genutzt, um Informationen zum Virtuellen Privaten Netzwerk zu speichern. Sie enthalten alle notwendigen Informationen, um einen IPsec-Tunnel aufzubauen und die entsprechenden IP-Pakete zu sichern. Eine Security Association ist dabei immer nur unidirektional und eine Verbindung zwischen zwei Endpunkten braucht entsprechend zwei Security Associations, je eine für jede Richtung der Kommunikation. Um einen Tunnel erfolgreich aufzubauen müssen beide Security Associations auf beiden Endpunkten installiert werden. Security Associations werden durch das IKE (oder IKEv2) Protokoll verhandelt, erstellt und verwaltet. Security Policies wiederum werden nicht durch das IKE-Protokoll ausgehandelt und müssen manuell durch den Nutzer erstellt und verwaltet werden. Allerdings erstellen einige IKE-Daemons, wie auch `charon` (siehe auch Teilabschnitt 2.5.1), Security Policies automatisch, basierend auf der genutzten Konfiguration. Die Informationen in den Security Policies werden genutzt, um die zu sichernden Pakete herauszufiltern, und die Informationen in den Security Associations werden später durch das ESP oder AH Protokoll genutzt, um die IP-Pakete korrekt zu sichern.

Eine Security Association speichert Informationen wie mit IP-Paketen, welche durch einen Tunnel gesendet werden sollen, verfahren werden soll. Neben den IP-Adressen der Tunnelenden wird

auch das Protokoll, mit dem die IP-Pakete gekapselt werden sollen (ESP oder AH), sowie der zu verwendende Modus (Tunnel oder Transport) gespeichert (für Informationen zu den Protokollen und Modi siehe Teilabschnitt 2.4.3). Außerdem werden die zu verwendenden Algorithmen mit den entsprechenden Schlüsseln für die Verschlüsselung und Authentifizierung der Pakete gespeichert. Zuletzt enthalten Security Associations noch die aktuelle Sequenznummer der ESP oder AH-Pakete, welche genutzt wird, um Replay Angriffe zu erkennen und zu blockieren (siehe auch Teilabschnitt 2.4.3). Security Associations werden durch einen Security Parameter Index (SPI) identifiziert. Dieser, in Kombination mit dem verwendeten Protokoll und dem Modus, identifiziert eine Security Association eindeutig.

Eine Security Policy gibt an, welche Pakete durch IPsec-Tunnel versendet werden sollen. Entsprechende IP-Pakete werden meist anhand der IP-Adressen von Sender und Empfänger ausgewählt. Allerdings ist auch eine Auswahl anhand von anderen Kriterien, wie zum Beispiel der genutzten TCP und UDP-Ports, möglich. Die Security Policy speichert auch die Security Association, die dann für dieses Paket verantwortlich ist. Um eine Security Association eindeutig identifizieren zu können, wird der SPI, das zu nutzende Protokoll und der Modus gespeichert.

2.4.2 IKEv2 Protokoll

Das Internet Key Exchange v2 Protokoll (kurz: IKEv2), ist im RFC 7296[KHN⁺14] spezifiziert und ist für den Austausch von Schlüsselmateriale und die Verwaltung von Security Associations zuständig. Es entstand 2005 im RFC 4306[Kau05] und ist der Nachfolger von IKE (gelegentlich auch als IKEv1 bezeichnet), welches zuletzt im RFC 2406[HC98] standardisiert wurde. IKEv2 behebt und verbessert diverse Probleme die mit IKE aufkamen. So führt es zum Beispiel die Möglichkeit Algorithmen zwischen Endpunkten verhandeln zu lassen ein und reduziert den Kommunikationsaufwand beim Erstellen eines IPsec-Tunnels stark. Aus diesem Grund wird in dieser Arbeit nur IKEv2 betrachtet. Wenn folgend von IKE gesprochen wird, ist damit immer IKEv2 gemeint, sofern nicht anders angegeben.

Zur Kommunikation zwischen zwei Endpunkten nutzt das IKE Protokoll eigene Pakettypen auf dem Application Layer, hier IKE-Nachrichten oder IKE-Pakete genannt. IKEv2 unterscheidet dabei zwischen zwei Arten von Security Associations: IKE SA und Child SA. IKE SA sind nur für die Verschlüsselung von IKE-Paketen zuständig und Child SA dienen der Verschlüsselung aller IP-Pakete die einen IPsec-Tunnel passieren sollen. IKE SA werden somit genutzt um Schlüssel sicher auszutauschen und die Child SA zu verhandeln. Child SA können also immer nur unter einer bereits bestehenden IKE SA erzeugt werden und werden gelöscht, wenn ihre zugehörige IKE SA gelöscht wird. Neben der Generierung von IKE SA und Child SA ist das IKE Protokoll auch für das rechtzeitige Austauschen von Security Associations nach (oder kurz vor) abgelaufener Lebenszeit (auch Rekeying genannt) verantwortlich.

Der Aufbau eines IPsec-Tunnels erfordert bei IKEv2 normalerweise vier Nachrichten. Das erste Nachrichtenpaar sind dabei sogenannte `IKE_SA_INIT` Nachrichten, welche eine IKE SA zwischen beiden kryptographischen Endpunkten aufbauen. Die Schlüssel für die IKE SA werden durch einen Diffie-Hellman Schlüsselaustausch (siehe [DH76]) erzeugt. Dadurch müssen die Schlüssel nicht im Klartext über unsichere Netze übertragen werden, wodurch Schlüssel nicht von Dritten abgefangen und mitgelesen werden können. Dies funktioniert allerdings nur, wenn beide Endpunkte mit dem genutzten Algorithmus zur Verschlüsselung einverstanden sind. Ist dies nicht der Fall, sind weitere `IKE_SA_INIT` Nachrichten nötig, um den Algorithmus auszuhandeln. Alle wei-

teren IKE Nachrichten werden mithilfe der soeben erzeugten IKE SA verschlüsselt übertragen. Das zweite Nachrichtenpaar besteht aus `IKE_AUTH` Nachrichten und dient dazu, beide Endpunkte zu authentifizieren und zugleich eine erste Child SA zu erzeugen. Da die Schlüssel für die Child SA bereits durch die IKE SA verschlüsselt übertragen werden, ist kein weiterer Diffie-Hellman Schlüsselaustausch notwendig. Ein Ablaufdiagramm dieses Prozesses ist in Abbildung 2.5 zu sehen. Weitere benötigte Child SA, sowie das Rekeying, können ähnlich durch `CREATE_CHILD_SA` Nachrichtenpaare erzeugt werden.

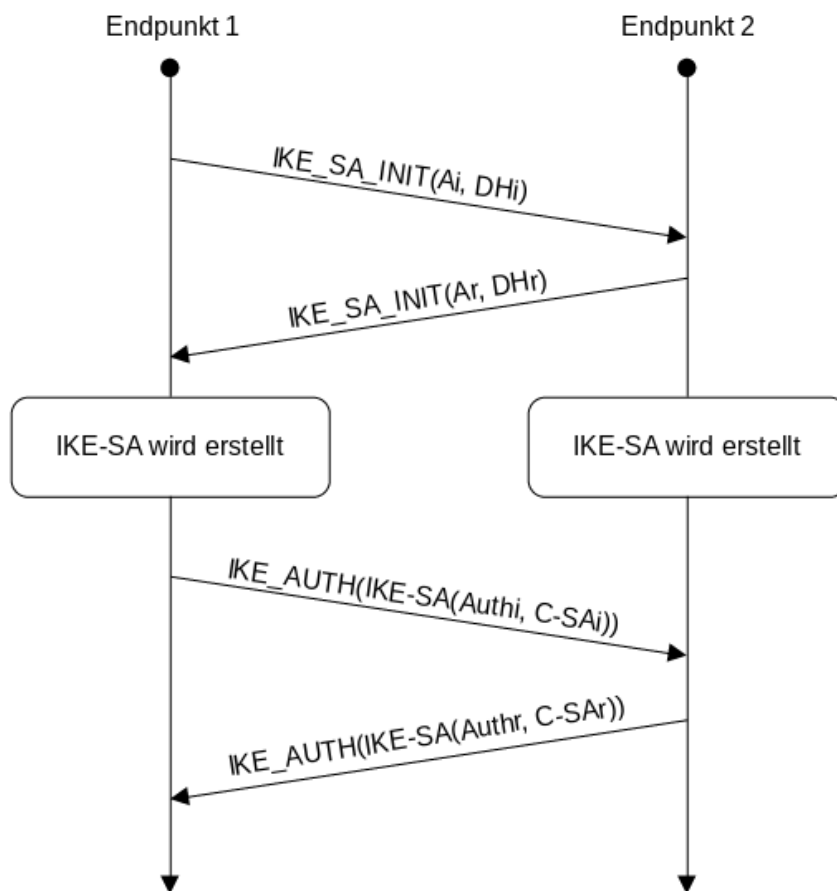


Abbildung 2.5: Stark vereinfachte Darstellung eines IKEv2 Verbindungsaufbaus. In den `IKE_SA_INIT` Nachrichten tauschen beide Endpunkte ihren bevorzugten Algorithmus (A_i und A_r) und je ihre Hälfte des Diffie-Hellman Schlüssels (DH_i und DH_r) aus. Wenn der gewählte Algorithmus identisch ist ($A_i = A_r$), erzeugen beide Endpunkte eine IKE-SA, mit dem gewählten Algorithmus und dem Schlüssel. Alle weiteren Nachrichten sind mit dieser IKE-SA abgesichert. In den `IKE_AUTH` Nachrichten authentifizieren beide Endpunkte sich gegenseitig ($Auth_i$ und $Auth_r$) und erstellen ihre erste Child-SA ($C-SA_i$ und $C-SA_r$).

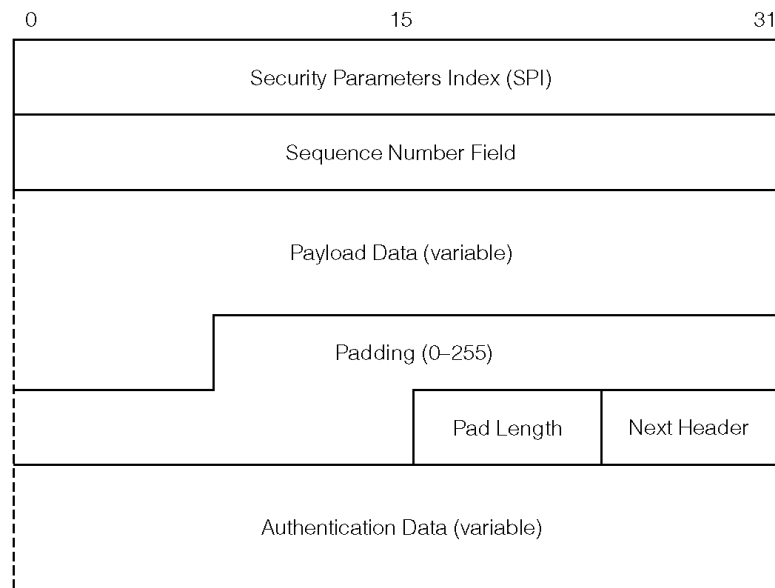


Abbildung 2.6: Darstellung eines Encapsulating Security Payloads.

Quelle:

<https://what-when-how.com/ipv6-advanced-protocols-implementation/encapsulating-security-payload-ipv6-and-ip-security/>, zugegriffen am 16. April 2022, 17:10

2.4.3 Encapsulating Security Payload

Encapsulating Security Payload (ESP) ist ein Protokoll zur Authentifikation, Integritätsbewahrung und Verschlüsselung auf dem Internet Layer. Es wurde vom US Naval Research Laboratory entwickelt und erstmals 1995 im RFC 1827[Atk95b] standardisiert. Die aktuelle Version findet sich im RFC 4303[Ken05b]. Es kann genutzt werden, um die Integrität und Vertraulichkeit von übertragenen Daten, sowie die Authentizität des Senders, zu gewährleisten.

Um die Integrität und Authentizität zu sichern enthält der ESP eine Integrity Check Value (ICV). Daneben enthält es auch ein Feld für den SPI der zu nutzenden SA und ein Feld für die Sequenznummer des Paketes, welche mit der Sequenznummer der SA verglichen wird, um Replay Angriffe zu erkennen und blockieren. Die SA speichert dabei außerdem die zu nutzenden Algorithmen für die Verschlüsselung und Integritätschecks, sowie die entsprechenden Schlüssel. Um die Vertraulichkeit der übertragenen Daten zu sichern wird außerdem der Payload verschlüsselt. Abbildung 2.6 zeigt den Aufbau eines Encapsulating Security Payloads.

Bei der Kapselung mit ESP wird immer erst die Verschlüsselung des Payloads und anschließend die Berechnung des ICV vorgenommen. Der Empfänger muss entsprechend erst die Integrität mithilfe der ICV überprüfen (und das Paket entsprechend verwerfen, wenn der Integritätscheck nicht erfolgreich war) und anschließend die Entschlüsselung vornehmen. (Die beiden Schritte können auch parallelisiert werden, solange das Paket trotzdem noch korrekt verworfen wird, wenn die ICV nicht korrekt ist.) Es ist auch möglich, nur einen der beiden Algorithmen zu nutzen. In diesen Fall erhält man allerdings keine Integritätschecks durch ICV oder keine Vertraulichkeit durch die Verschlüsselung des Payloads.

Die ICV wird durch den in der SA angegebenen Algorithmus berechnet. Für die Berechnung

wird das gesamte ESP-Paket (mit Ausnahme des Feldes für die ICV), inklusive (verschlüsseltem) Payload, genutzt. Um die Authentizität des Senders zu gewährleisten wird für die Berechnung des ICV zusätzlich ein geheimer Schlüssel benötigt. Wie der Schlüssel genutzt wird ist abhängig vom Algorithmus, in vielen Fällen wird der Schlüssel für die Berechnung einfach an das Paket angehängt. Der Schlüssel sollte nur dem Sender und dem Empfänger der AH-Pakete bekannt sein und wird nicht im ESP-Paket mit übertragen. Der Empfänger kann mithilfe des Schlüssels und dem gleichen Verfahren die Korrektheit und Echtheit der Nachricht verifizieren. Eine Nachricht gilt dabei als echt und vertrauenswürdig, wenn die errechnete ICV mit der übertragenen ICV übereinstimmt. Andernfalls muss das gesamte Paket verworfen werden.

ESP bietet außerdem die Möglichkeit, einen kombinierten Algorithmus zu nutzen, der die Verschlüsselung und die Integritätschecks gleichzeitig übernimmt. Solche Algorithmen werden auch Authenticated Encryption (kurz AE) oder Authenticated Encryption with Associated Data (kurz AEAD) genannt. Ein solcher Algorithmus ist zum Beispiel AES-GCM (für eine Erläuterung des Algorithmus siehe Teilabschnitt 2.6.2). Bei einem kombinierten Algorithmus ist das Feld für die ICV optional und dessen Nutzung vom verwendeten Algorithmus abhängig.

ESP kann in zwei verschiedenen Modi operieren: Transport-Mode und Tunnel-Mode. Im Transport-Mode wird das ESP-Paket dabei zwischen dem zu schützenden IP-Paket und seinem Payload positioniert. Hierbei muss darauf geachtet werden, dass einige Felder des IP-Paketes (wie die Länge des Payloads und die Angabe des nächsten Protokolls) entsprechend angepasst werden müssen. Im Tunnel-Mode wird das gesamte zu schützende IP-Paket in einem ESP-Paket gekapselt. Dieses ESP-Paket wiederum wird in einem neuen IP-Paket versendet. Die Werte der Felder des neuen (äußeren) IP-Paketes müssen nicht mit dem des inneren IP-Paketes übereinstimmen. Das äußere IP-Paket kann also auch an ein anderes Gerät adressiert sein. Im Kontext von IPsec ist das äußere IP-Paket meistens an einen Tunnelendpunkt adressiert und das innere IP-Paket an den finalen Empfänger der Nachricht. Eine Darstellung der beiden verschiedenen Modi ist in Abbildung 2.7 zu sehen.

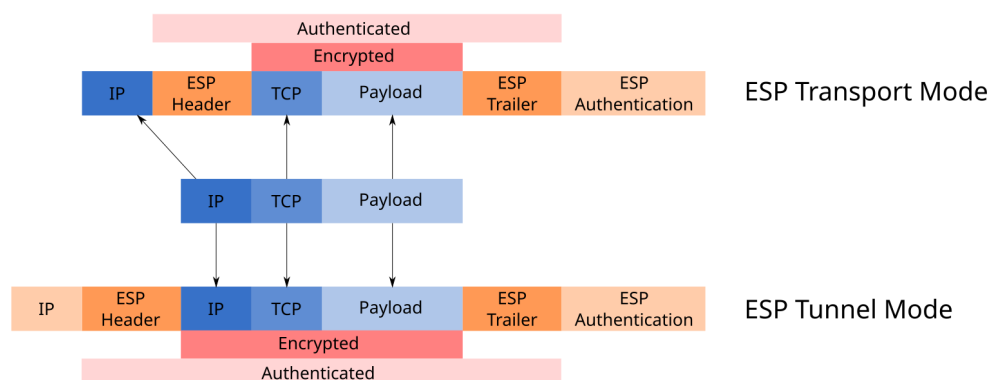


Abbildung 2.7: Kapselung eines IP-Paketes mit TCP-Paket (mitte) mit Encapsulating Security Payload im Transport-Mode (oben) und Tunnel-Mode (unten).

Quelle: https://en.wikipedia.org/wiki/IPsec#Encapsulating_Security_Payload, zugegriffen am 16. April 2022, 16:50

Neben dem Encapsulating Security Payload bietet IPsec auch an, Pakete in einem Authenti-

cation Header (AH) zu kapseln. Dieser wurde erstmal im RFC 1826[Atk95a] standardisiert und die aktuelle Version findet sich im RFC 4302[Ken05a]. AH funktioniert ähnlich wie ESP, bietet allerdings keine Vertraulichkeit, da keine Verschlüsselung stattfindet. Aus diesem Grund wird die Kapselung mit ESP generell der Nutzung von AH vorgezogen. Der Hauptvorteil von Authentication Header gegenüber dem Encapsulating Security Payload liegt in dem Fakt, dass Authentication Header auch die Echtheit des äußeren IP-Paketes gewährleisten können. Entsprechend werden Authentication Header gelegentlich auch genutzt, um ESP-Pakete zusammen mit dem äußeren IP-Paket zu sichern, was ESP alleine nicht kann.

2.5 Implementierung von IPsec in Linux

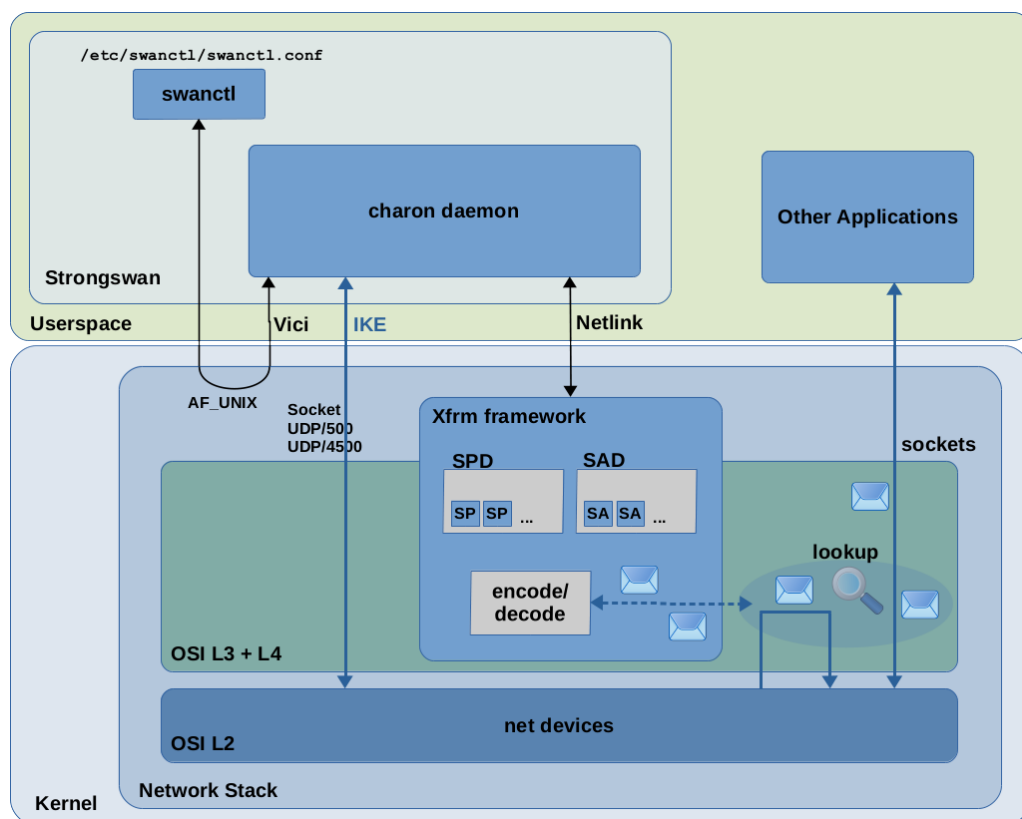


Abbildung 2.8: Zusammenhang der Komponenten von StrongSwan und IPsec im User- und Kernel-Space, sowie deren Kommunikationswege miteinander.

In diesem Kapitel wird die Implementierung von IPsec in Linux besprochen. Linux implementiert die IPsec-Suite dabei in zwei Teilen. Ein Userspace Prozess ist für die Implementierung des IKE Protokolles verantwortlich und die Verschlüsselung durch AH und ESP findet im Kernel statt. In dieser Arbeit wird als Implementierung StrongSwan⁵ genutzt, welches den IKE-Daemon cha-

⁵<https://www.strongswan.org/>

ron bereitstellt.

2.5.1 StrongSwan's IKE-Daemon charon

charon ist für die Aushandlung von Security Associations und die Verwaltung von IPsec Tunneln zuständig. charon speichert und verwaltet dabei IKE SA selbst, ohne Unterstützung vom Kernel. Die Child SA und Security Policies allerdings werden in der Security Policy Database (SPD) und der Security Association Database (SAD) im Kernel abgespeichert.

Die Interaktion mit charon erfolgt dabei über externe Tools, die über eigene Interfaces mit charon kommunizieren. Die von StrongSwan bereitgestellten Tools sind dabei ipsec, welches das ältere Stroke Interface nutzt, und swanctl, welches das neuere VICI Interface (Versatile IKE Control Interface) nutzt und deshalb generell eher empfohlen wird. In dieser Arbeit wurde für die Konfiguration von charon nur swanctl genutzt.

2.5.2 XFRM Framework

Das XFRM Framework ist Teil des Linux Kernel Network Stacks. Zu seinen Aufgaben gehören die Modifikation von eingehenden und ausgehenden Paketen. Genauer gesagt die Verschlüsselung (und Kapselung) von IP-Paketen in ESP oder AH-Pakete und ebenso die Entschlüsselung von eingehenden ESP oder AH-Paketen in IP-Pakete.

Um die Verschlüsselung durchzuführen nutzt das XFRM Framework die Security Associations und die Security Policies in den eigenen Datenbanken (SAD und SPD). In der Praxis werden diese häufig von IKE-Daemons erstellt und installiert, aber Security Associations und Security Policies können auch manuell erstellt und installiert werden. Die Kommunikation mit dem XFRM Framework erfolgt mithilfe von Netlink Sockets.

Es ist auch möglich, manuell auf das XFRM Framework zuzugreifen. Dies geschieht mit dem `ip xfrm` Befehl. Security Associations können dabei mit dem `ip xfrm state` Befehl angesehen und verändert werden. Für Security Policies kann der `ip xfrm policy` Befehl verwendet werden.

Abbildung 2.9 zeigt eine Visualisierung des Paketflusses, mit den Stellen an denen das XFRM Framework Pakete abfängt und modifiziert.

2.6 AES-GCM Verschlüsselung

In diesem Kapitel wird das Verschlüsselungsverfahren AES (Advanced Encryption Standard) besprochen. AES kann nur Daten mit einer festen Länge von 16 Byte verschlüsseln. Um mehr Daten zu verschlüsseln werden sogenannte Betriebsmodi genutzt. Ein Betriebsmodus gibt dabei an, wie das Verschlüsselungsverfahren auf längeren Daten eingesetzt wird. In diesem Kapitel wird außerdem der Betriebsmodus GCM (Galois/Counter Mode) vorgestellt.

2.6.1 Advanced Encryption Standard

Der Advanced Encryption Standard (kurz: AES) wurde 2001 vom U.S. National Institute of Standards and Technology in [oSN01] als Nachfolger des Data Encryption Standard (kurz: DES) eingeführt. Er basiert auf dem 1999 in [DR99] vorgestellten Rijndael Verschlüsselungsverfahren. Er

Packet flow in Netfilter and General Networking

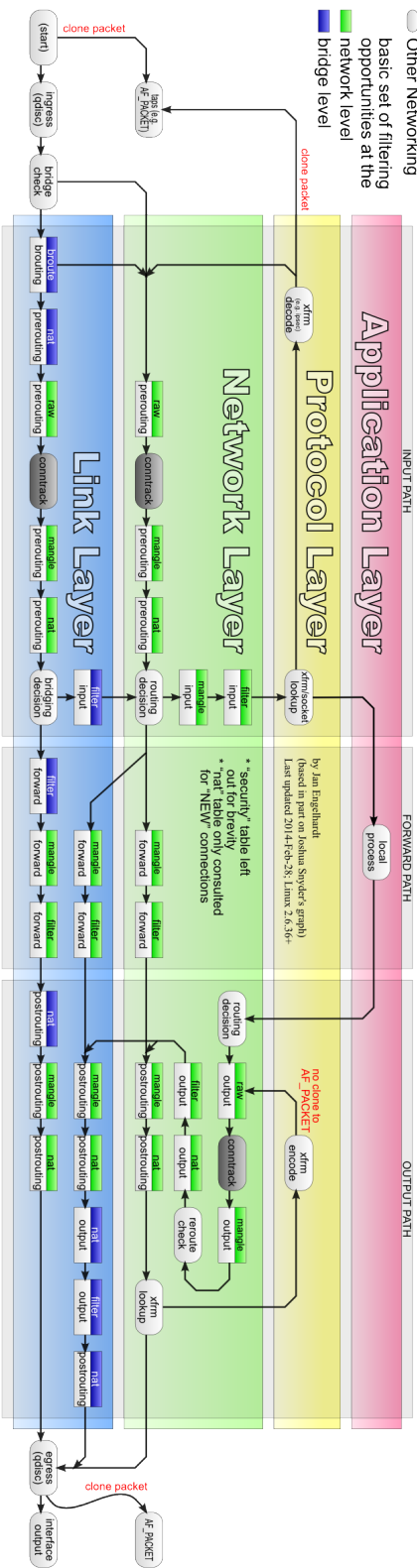


Abbildung 2.9: Eine Darstellung des Packetflusses durch den Linux Network Stack mit Netfilter Hooks und XFRM Hooks. Pakete werden an den xfrm Lookup Hooks mit den Security Policies verglichen und bei einem Treffer umgeleitet. Bei xfrm decode und xfrm encode werden die Pakete mit den Informationen aus den Security Associations ver- oder entschlüsselt.
Quelle: <https://inai.de/images/nf-packet-flow.png>, Zugriffen am 19. April 2022, 13:57

gilt aktuell als einer der sichersten bekannten Verschlüsselungsverfahren und wird aufgrund seiner hohen Geschwindigkeit in vielen Anwendungsbereichen genutzt.

AES ist eine symmetrische Blockchiffre. Das bedeutet, AES nutzt für die Ver- und Entschlüsselung von Daten den gleichen Schlüssel und kann nur Daten mit einer festen Größe von 16 Byte verschlüsseln. Als Schlüsselgrößen kann AES mit drei verschiedenen Varianten arbeiten: 128 Bit (16 Byte), 192 Bit (24 Byte) und 256 Bit (32 Byte).

AES stellt die 16 Bytes (hier b_0 bis b_{15}) für die Verschlüsselung in einer 4*4 Tabelle oder Matrix, wie folgt dar:

$$\begin{bmatrix} b_0 & b_4 & b_8 & b_{12} \\ b_1 & b_5 & b_9 & b_{13} \\ b_2 & b_6 & b_{10} & b_{14} \\ b_3 & b_7 & b_{11} & b_{15} \end{bmatrix}$$

Diese Matrix wird in diversen Runden transformiert. Die Anzahl der Runden unterscheidet sich dabei je nach Schlüsselgröße und kann 10 Runden (128 Bit Schlüssel), 12 Runden (192 Bit Schlüssel) oder 14 Runden (256 Bit Schlüssel) sein. Eine Runde besteht dabei aus vier Operationen, die auf der Matrix durchgeführt werden.

Zuerst kommt die Substitution, auch SubBytes genannt. Hierbei werden alle Bytes mittels einer Substitutions-Tabelle (häufig S-Box genannt) ausgetauscht. Diese Substitution ist dabei nicht-linear, kann also nicht oder nur sehr schwer durch mathematische Formeln dargestellt werden. Dies soll Angriffe durch mathematische Analysen des Algorithmus verhindern. Die Substitution ist allerdings bijektiv und kann bei der Entschlüsselung entsprechend umgekehrt werden. Die Aufgabe der Substitution ist es sicherzustellen, dass selbst kleine Änderungen bei der Eingabe zu großen Änderungen bei der Ausgabe führen. Dies macht es nicht möglich, durch die Ausgabe Rückschlüsse auf die Eingabe zu ziehen. Diese Eigenschaft wird auch Konfusion genannt.

Die zweite Operation ist ShiftRows, in der die Zeilen der Matrix verschoben werden. Die zweite Zeile wird dabei eine Stelle nach links geschoben, die dritte Zeile zwei Stellen und die vierte Zeile drei Stellen. Die oberste Zeile der Matrix bleibt unverändert.

Die dritte Operation ist MixColumns. Hierbei wird jede Spalte der Matrix einer Matrixmultiplikation mit einer festen Matrix M (2.1) unterzogen. Bei der Entschlüsselung kann eine Multi-

$$M = \begin{bmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{bmatrix} \quad (2.1)$$

Gleichung 2.1: Matrix M , wie sie für den MixColumns Schritt verwendet wird. M ändert sich dabei nicht (selbst wenn andere Schlüssel genutzt werden) und ist immer gleich. Für die Entschlüsselung wird die Matrix M^{-1} verwendet.

pplikation mit dem Inversen der Matrix M durchgeführt werden, um die MixColumns-Operation umzukehren. Die Operationen ShiftRows und MixColumns bewirken zusammen eine Durchmischung der Matrix, auch Diffusion genannt. Dies bewirkt, dass alle Bytes in der Eingabe auch alle Bytes in der Ausgabe beeinflussen.

Die letzte Operation ist AddRoundKey und ist eine XOR-Operation mit einem sogenannten Rundenschlüssel. AES nutzt für jede Anwendung von AddRoundKey einen eigenen Runden-

schlüssel, welcher 128 Bit lang ist. Die benötigten Rundenschlüssel werden mithilfe der oben beschriebenen S-Box und einer Shift-Operation aus dem initialen Schlüssel erzeugt. Wie viele Rundenschlüssel erzeugt werden müssen, hängt dabei von der gewählten Schlüsselgröße des initialen Schlüssels ab.

Es gibt allerdings zwei Ausnahmen zu den angewendeten Operationen: Vor der ersten Runde wird bereits einmal eine AddRoundKey Transformation gemacht und in der finalen Runde wird auf die Anwendung von MixColumns verzichtet.

2.6.2 Galois/Counter Mode

Blockchiffren können immer nur eine feste Anzahl an Bits verschlüsseln. Diese Bits sind der sogenannte Block, der durch die Blockchiffre verschlüsselt wird. Betriebsmodi geben an wie verfahren werden soll, um auch eine größere Menge an Daten sicher zu verschlüsseln. Die Wahl guter Betriebsmodi ist wichtig um die Diffusion zu gewährleisten. Bei schlecht gewählten Betriebsmodi, wie zum Beispiel Electronic Codebook (ECB), bei der jeder Block einfach einzeln verschlüsselt wird, ist die Diffusion nicht gewährleistet, da identische Blöcke auch identisch verschlüsselt werden. Dies wiederum lässt Rückschlüsse auf die ursprünglichen Daten zu. [MvOV18, Anmerkung 7.12]

Der in dieser Arbeit genutzt Betriebsmodus ist Galois/Counter Mode. Dieser basiert auf dem Counter Mode (kurz: CTR Mode), welcher als vergleichsweise schnell gilt, und erweitert diesen um eine Authentifizierung der Daten durch eine Galois Authentifikation. Er wurde 2007 in [Dwo07] standardisiert und ist aktuell aufgrund seiner hohen Geschwindigkeit weit verbreitet. Eine Darstellung des Galois/Counter Mode ist in Abbildung 2.10 zu finden.

Die Grundidee des Counter Mode und damit auch des Galois/Counter Mode ist es, nicht die gewünschten Daten zu verschlüsseln, sondern stattdessen eine Kombination aus Zählvariable und Initialization Vector (kurz: IV) zu verschlüsseln. Die Zählvariable beginnt dabei bei 1. Der IV sollte dabei zufällig sein und bei jeder Verschlüsselung mit dem Galois/Counter Mode neu generiert werden. Im Fall von AES wird entsprechend ein 96 Bit langer IV, gefolgt von einer 32 Bit langen Zählvariable verschlüsselt. Das Ergebnis dieser Verschlüsselung wird folgend $E_K(IV||counter)$ genannt. (Sollte die IV nicht 96 Bit lang sein, kann eine 96 Bit lange Zahl aus der gegebenen IV generiert werden.) Die eigentlichen Daten werden dann durch ein XOR mit $E_K(IV||counter)$ verschlüsselt. Die Zählvariable wird daraufhin erhöht und das Verfahren für alle Datenblöcke wiederholt. Die Daten können durch eine weitere XOR-Operation mit $E_K(IV||counter)$ auch wieder entschlüsselt werden. Die Entschlüsselung kann gut parallelisiert werden, da alle Blöcke unabhängig voneinander entschlüsselt werden können, und ist deshalb besonders schnell. Die Verschlüsselung mit XOR hat auch den positiven Nebeneffekt, dass die verschlüsselten Daten die gleiche Länge wie die unverschlüsselten Daten haben.

Bei einem zufällig gewählten IV ist $E_K(IV||counter)$ nicht ohne Kenntnis des Schlüssels bestimmbar, selbst wenn sowohl der IV, als auch die Zählvariable bekannt sind. Ohne $E_K(IV||counter)$ zu kennen ist es nicht möglich, den Klartext aus den verschlüsselten Datenblöcken abzuleiten, da für die Umkehrung der XOR-Operation $E_K(IV||counter)$ benötigt wird. Die Nutzung eines IV vermeidet es, für jede übertragene Nachricht einen neuen Schlüssel aushandeln zu müssen. Dafür muss der IV aber auch bei der Entschlüsselung bekannt sein, er muss also beim Versenden der verschlüsselten Daten mitgesendet werden. Außerdem darf ein IV nicht doppelt genutzt werden, da sonst die Diffusion nicht gewährleistet ist.

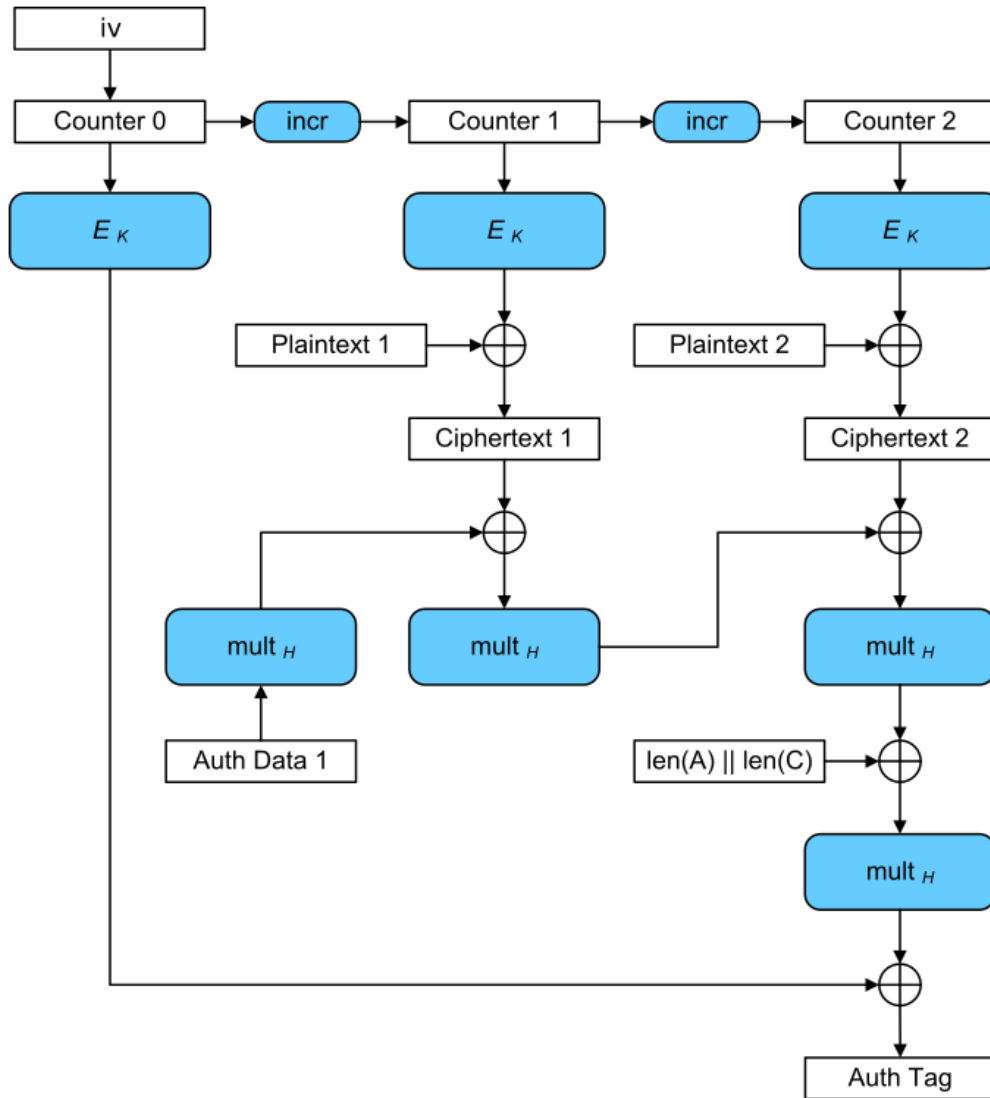


Abbildung 2.10: Eine Darstellung der Verschlüsselung mit Galois/Counter Mode. Bei E_K handelt es sich um den Verschlüsselungsalgorithmus E mit Schlüssel K . Counter 0 ist eine Kombination des IV und der Zahl 1. Bei Auth Data handelt es sich um Daten, welche nur authentifiziert, aber nicht verschlüsselt werden sollen. mult_H ist die Multiplikation mit dem Fixpunkt H . $\text{len}(A)$ und $\text{len}(C)$ sind die Bitlängen der gesamten Auth Data und Ciphertext.

Quelle: https://en.wikipedia.org/wiki/Galois/Counter_Mode, Zugriffen am 25. April 2022, 17:37 Uhr

Der Galois/Counter Mode authentifiziert die verschlüsselten Daten auch noch zusätzlich. Für die Authentifikation wird ein Authentifikationstag (auch Integrity Check Value, kurz ICV, genannt) errechnet und mitgesendet, welcher sich hauptsächlich aus den verschlüsselten Datenblöcken ergibt. Der Authentifikationstag wird dabei durch XOR-Operationen und Multiplikationen mit einem Fixpunkt H errechnet (siehe auch Abbildung 2.10). H entspricht dabei der Verschlüsselung der 0 ($H = E_K(0)$). Der Fixpunkt H und der sich daraus ergebende Authentifikationstag sind somit stark vom gewählten Schlüssel K abhängig und der Authentifikationstag kann ohne Kenntnis des Schlüssels nicht gefälscht werden. Außerdem ist der Authentifikationstag stark von den verschlüsselten Daten abhängig, sodass eine Änderung in den verschlüsselten Daten durch einen fehlerhaften Authentifikationstag auffallen würde. Der Authentifikationstag kann vom Empfänger auf die gleiche Weise errechnet und mit dem übertragenen Tag verglichen werden. Stimmen die Tags nicht überein, liegt ein Fehler oder eine Änderung bei den übertragenen Daten vor.

2.7 Hardware-Offloading mit IPsec

Hardware-Offloading beschreibt die Verarbeitung von diversen Funktionalitäten auf eigens dafür hergestellter Hardware, anstatt, wie üblich, in Software auf General-Purpose-Hardware, wie zum Beispiel CPUs. Da die dafür verwendete Hardware nur wenige spezielle Aufgaben hat, kann auf viele Funktionalitäten von General-Purpose-Hardware, die für diese Aufgaben nicht benötigt werden, verzichtet werden. Dadurch ist die verwendete Hardware meist deutlich effizienter und arbeitet somit schneller als herkömmliche CPUs, ist aber auch deutlich weniger flexibel und kann nur für bestimmte Aufgaben verwendet werden. Hardware-Offloading wird meist für Aufgaben, die besonders häufig auftreten oder besonders zeitintensiv sind, verwendet. Außerdem kann durch die Nutzung von Hardware-Offloading die CPU entlastet werden.

Netzwerkarten, die über Offloading Funktionalitäten verfügen, werden auch SmartNICs genannt. Für diese Arbeit besonders interessant ist das IPsec-Offloading (in manchen Kontexten auch ESP-Offloading oder XFRM-Offloading genannt). Dabei wird die Ver- und Entschlüsselung von IP-Paketen auf die Netzwerkkarte ausgelagert. Dazu gehören unter anderem die Suche nach passenden Security Associations, die Verschlüsselung des IP-Paketes (im Tunnel-Mode) oder des Payloads (im Transport-Mode) und die Kapselung in einem ESP-Paket und, im Tunnel-Mode, äußeren IP-Paket.

Im Windows Server ist eine API für das IPsec-Offloading seit der Version 2008 unter dem Namen "IPsec Task Offload" implementiert, wird inzwischen aber offiziell nicht mehr unterstützt oder weiterentwickelt.⁶ Im Linux Kernel ist eine API für die Nutzung von IPsec-Hardware-Offloading seit 2017 mit Version 4.12⁷ implementiert. Diese kann bei der Kompilierung durch die Flags `CONFIG_XFRM_OFFLOAD` und `CONFIG_INET_ESP_OFFLOAD` aktiviert werden. IPsec-Offloading für Linux wurde erstmals 2017 auf der NetDev Conference 1.2 (im IPsec Workshop⁸) vorgestellt. Im weiteren Verlauf der Arbeit wird nur noch die Linux Version betrachtet.

Um das Hardware-Offloading mit einer SmartNIC nutzen zu können, muss ein Treiber entspre-

⁶Für mehr Informationen siehe auch: <https://techcommunity.microsoft.com/t5/core-infrastructure-and-security/why-are-we-deprecating-network-performance-features-kb4014193/ba-p/259053>

⁷genauer durch Commit d77e38e612a017480157fe6d2c1422f42cb5b7e3

⁸<https://legacy.netdevconf.info/1.2/session.html?steffen-klassert>

chende Funktionen zum Hinzufügen und Entfernen von Security Associations implementieren.⁹ Wenn ein IP-Paket durch den Tunnel versendet werden soll, überprüft der Treiber zunächst, ob Hardware-Offloading für dieses Paket genutzt werden kann. Ist dies der Fall, wird das Paket und die Netzwerkkarte vom Treiber für die Verschlüsselung vorbereitet und anschließend das Paket an die Netzwerkkarte übergeben, wo es verschlüsselt und in einem ESP-Paket gekapselt und anschließend versendet wird. Kann das Paket nicht auf der Netzwerkkarte verschlüsselt werden (ein häufiger Grund dafür ist, dass das Paket dadurch die MTU überschreiten würde und fragmentiert werden müsste, was viele Netzwerkkarten nicht können), dann wird es stattdessen normal auf der CPU in Software verschlüsselt und gekapselt und nur das finale Paket an die Netzwerkkarte zum Versenden übergeben. Wenn ein ESP-Paket empfangen wird, wird es - sofern möglich - direkt auf der Netzwerkkarte entschlüsselt und der Treiber informiert den Kernel über die Entschlüsselung (dies ist wichtig für das Aktualisieren der Sequenznummer und für die korrekte Anwendung von Filterregeln). Wenn die Netzwerkkarte das Paket nicht entschlüsseln kann, wird das Paket normal auf der CPU entschlüsselt. Diese beiden Fallbacks auf die Software-Implementierung im Linux Kernel werden auch Slow-Path genannt.

Um Hardware-Offloading nutzen zu können, müssen die SA bei der Erstellung auch auf der Netzwerkkarte installiert werden. Wenn Security Associations manuell mit dem `ip xfrm` Befehl erstellt werden, kann dies durch die Option `offload dev DEVICE dir [in|out]` erreicht werden. In StrongSwan kann Hardware-Offloading einfach über die Flag `hw_offload = no | yes | auto` aktiviert werden (`auto` lässt, anders als `yes`, einen Software-Fallback zu).

⁹siehe auch: https://www.kernel.org/doc/html/latest/networking/xfrm_device.html

3 Messplan

In diesem Kapitel werden die Testumgebung und die darauf durchgeführten Messungen vorgestellt. Es wird zuerst der Aufbau der Testumgebung und die sich daraus ergebenden Limitierungen erläutert. Zusätzlich erfolgt eine erste Analyse des Testsystems, in der mögliche Bottlenecks identifiziert werden. Anschließend werden die erhobenen Messwerte, sowie die konstanten Parameter und variablen Faktoren der Messungen eingeführt. Am Ende des Kapitels werden die durchgeführten Messungen erläutert.

Das Ziel der Messungen ist es, die Performance verschiedener IPsec-Verschlüsselungsoptionen miteinander zu vergleichen. Von besonderem Interesse ist hierbei, wie sich die Nutzung von Hardware-Offloading durch smarte Netzwerkkarten (auch SmartNICs genannt) auf die Performance auswirkt. Insbesondere in Hochgeschwindigkeits-Netzwerken mit Übertragungsraten von bis zu 100 Gbit/s, wie sie in einigen Bereichen bereits eingesetzt werden, ist es wichtig, den Performanceverlust durch Verschlüsselungen mit IPsec möglichst gering zu halten. Zu diesem Zweck unterstützt die in dieser Arbeit verwendete Hardware Übertragungsraten von bis zu 100 Gbit/s und die Tests werden, sofern dies ohne Performanceverluste möglich ist, mit Übertragungsraten von bis zu 100 Gbit/s durchgeführt.

3.1 Aufbau und Erläuterung der Testumgebung

Die Testumgebung besteht aus zwei x86-64-Linux-PCs (hier auch Testobjekt oder Testcomputer genannt) und einem Testgerät, welche alle über ein IP-Netzwerk miteinander verbunden sind. In diesem Kapitel werden die Testcomputer und das Testgerät genauer erklärt und anschließend das aufgebaute Testnetzwerk erläutert.

3.1.1 Spezifikation der Testcomputer

Die beiden Testcomputer sind ein Dell OptiPlex 7070 Tower und ein Dell Optiplex 7090 Tower. Im OptiPlex 7070 ist ein Intel i7-9700¹ als CPU eingebaut, welcher über 8 Kerne verfügt und eine maximale Taktfrequenz von 4,7 GHz erreicht. Im OptiPlex 7090 ist ein Intel i7-10700² als CPU eingebaut, welcher ebenfalls über 8 Kerne verfügt und eine maximale Taktfrequenz von 4,8 Ghz erreicht. Der i7-10700 unterstützt allerdings die Intel Hyper-Threading Technologie und hat damit 16 virtuelle / logische Kerne. Beide CPUs unterstützen den Intel AES-NI Instruktionssatz.

Der Intel AES-NI Instruktionssatz enthält sechs Instruktionen, welche die Ver- und Entschlüsselung von Daten mit AES beschleunigen sollen. Zwei der Instruktionen dienen dabei der Generierung von Round-Keys und die restlichen vier Instrktionen führen je eine Runde der Ver- oder

¹<https://www.intel.de/content/www/de/de/products/sku/191792/intel-core-i79700-processor-12m-cache-up-to-4-70-ghz/specifications.html>

²<https://www.intel.de/content/www/de/de/products/sku/199316/intel-core-i710700-processor-16m-cache-up-to-4-80-ghz/specifications.html>

Entschlüsselung mit AES durch. Da diese Instruktionen im Microcode der CPU implementiert sind, sind sie deutlich schneller als herkömmliche Software-Implementierungen von AES. Für weitere Informationen zur Anwendung und Performance von AES-NI siehe auch [Gue10] und [ADF⁺10].

In beide Testcomputer wurde als Netzwerkkarte eine Mellanox ConnectX-6 Dx³ eingebaut. Die in dieser Arbeit genutzte Version der Karten³ haben je zwei Netzwerkports, welche beide Übertragungsraten von 100 Gbit/s unterstützen. Außerdem verfügen beide Karten über die Möglichkeit IPsec-Verschlüsselungen mit ESP und AES-GCM durchzuführen. Die Netzwerkkarten sind dabei in einen PCIe 3.0 Slot der Mainboards der Testcomputer eingebaut, was eine Kommunikation zwischen Karte und Computer von bis zu 128 Gbit/s ermöglicht. Beide Computer sind über diese Netzwerkkarten miteinander verbunden. Um die erforderlichen Geschwindigkeiten von 100 Gbit/s zu erreichen werden QSFP28 Ethernet-Glasfaserkabel als Übertragungsmedium genutzt.

Als Betriebssystem kommt nur Linux zum Einsatz, da neuere Windows Versionen die nötigen Offloading-Funktionalitäten nicht mehr offiziell unterstützen. Außerdem sind die benötigten Treiber, um das Offloading nutzen zu können, nur für Linux verfügbar. Auf dem OptiPlex 7070 wurde als Betriebssystem Ubuntu 18.04 LTS in der Desktop-Version mit Linux-Kernel 5.4 und auf dem OptiPlex 7090 wurde Ubuntu 22.04 LTS mit Linux-Kernel 5.15 installiert. Insofern nicht anders angegeben wurde keine extra Software installiert und keine Einstellungen am System verändert.

Auf beiden Systemen wurde außerdem der MLNX_OFED (Version 5.6) Linux Treiber installiert. Dieser enthält unter anderem die notwendigen Funktionalitäten, um die Verschlüsselung und Kapselung mit ESP auf die Netzwerkkarte auszulagern.

Zum Aufbau von IPsec-Verbindungen zwischen beiden Computern wurde auf beiden Testcomputern StrongSwan (Version 5.9) installiert und der IKEv2-Daemon `charon` aktiviert. Die verwendete Konfigurationsdatei ist in Kapitel A zu finden. Da IKEv1 inzwischen veraltet ist und von IKEv2 abgelöst wurde, werden in dieser Arbeit keine Tests mit IKEv1 durchgeführt.

3.1.2 Spezifikation des Testgeräts

Für die im Rahmen dieser Arbeit vorgenommenen Messungen wurde ein VeEX RXT-1200-Testgerät mit dem RXT-6200-Testmodul⁴ verwendet. Das Testmodul hat zwei QSFP28/56 Ports, mit welchen Messungen mit bis zu je 100 Gbit/s durchgeführt werden können. Die Tests können dabei auf OSI-Layer 2, 3 oder 4 durchgeführt werden.

Das Testgerät verfügt über zwei Testmodi, welche für diese Arbeit relevant sind: Der erste ist ein RFC 2544 Test, welcher auf den Spezifikationen aus RFC 2544[BM99] und RFC 1242[Bra91] basiert und unter anderem Durchsatz, Latenz und verlorene Frames für verschiedene Framegrößen messen kann. Der zweite Test ist ein Durchsatztest, welcher eine festlegbare Anzahl an Frames senden kann und die Anzahl an empfangenen Frames misst und daraus den Durchsatz ermittelt.

Hierbei ist anzumerken, dass der Durchsatztest nicht der Definition aus RFC 1242[Bra91, Abschnitt 3.17] folgt und keinen verlustfreien Durchsatz ermittelt. Um einen verlustfreien Durchsatz zu ermitteln schreibt der RFC 2544[BM99, Abschnitt 16.1] einen Binary-Search Algorithmus vor. Der Durchsatztest verzichtet allerdings auf einen Binary-Search Algorithmus, was in der Praxis

³<https://www.nvidia.com/en-us/networking/ethernet/connectx-6-dx/>, MPN: MCX623106AC-CDAT, <https://store.nvidia.com/en-us/networking/store/product/MCX623106AC-CDAT/NVIDIAMCX623106ACCDATConnectX6DxENAdapterCard100GbECryptoEnabled/>

⁴<https://www.veexinc.com/products/rxt1200> und <https://www.veexinc.com/products/rxt6200module>

dazu führt, dass der gemessene Durchsatz im Durchsatztest oft um einige Prozent höher ist als beim RFC 2544 Test.

RFC 2544 sieht außerdem keine Tests mit verschiedenen Framegrößen (auch Frame-, Paket-mixe oder IMix, kurz für Internet Mix, genannt) vor (siehe auch [BM99, Abschnitt 18]) und der Test ermöglicht somit auch keine Messungen von Paketmischen. Auch Messungen mit verschiedenen Datenströmen, welche alle unterschiedliche Adressen nutzen, können mit dem verfügbaren RFC 2544 Test nicht durchgeführt werden. Dies ist besonders relevant, da moderne Netzwerkkarten über mehrere Queues verfügen, welche von mehreren Cores gleichzeitig abgearbeitet werden können und somit eine parallele Verarbeitung von Paketen ermöglichen. Die Pakete werden meist anhand ihrer Adressen (und wenn vorhanden TCP- / UDP-Ports) auf diese Queues aufgeteilt. Eine Nutzung von nur einem Stream ermöglicht somit keine Aufteilung der Pakete auf mehrere Queues und somit auch keine Parallelisierung, was zu Performanceverlusten führen kann.

Das RXT-6200 Modul besitzt außerdem keine Multi-Port Tests. Das bedeutet alle Tests werden nur auf einem Port durchgeführt und Daten müssen auf dem gleichen Port empfangen wie auch gesendet werden. Das Senden der Daten auf einem Port und Empfangen auf dem anderen Port ist nicht möglich. Allerdings hat der RXT-6200 eine Loopback Funktion, welche den Aufbau von Tests vereinfacht: Wenn die Loopback Funktion auf einem Port genutzt wird, werden alle empfangenen Daten auf diesem Port wieder zurückgesendet. Der Loopback kann dabei so konfiguriert werden, dass alle empfangenen Frames komplett unverändert oder mit getauschten Sender- und Empfänger-Adressen (betrifft sowohl MAC-, als auch IP-Adresse) zurückgesendet werden. In letzterem Fall werden die Daten also wieder an den Absender zurückgeschickt.

3.1.3 Aufbau des Netzwerkes

Beide Testcomputer sind über einen Port der Mellanox Netzwerkkarte miteinander verbunden. Außerdem sind beide Computer mit dem anderen Port auch mit dem Testgerät verbunden. Der Aufbau ist außerdem in Abbildung 3.1 zu sehen.

Das Netzwerk enthält drei Subnetze: $10.0.0.0/24$, $10.0.1.0/24$ und $10.0.2.0/24$. Das Subnetz $10.0.0.0/24$ wird dabei für die Kommunikation zwischen beiden Computern genutzt und die anderen beiden Subnetze für die Kommunikation mit dem Testgerät.

Die beiden Computer fungieren als Router und das Testgerät simuliert weitere Geräte in den beiden Subnetzen $10.0.1.0/24$ und $10.0.2.0/24$, welche miteinander über die Router kommunizieren (im einfachsten Fall wird also je ein Gerät im Subnetz $10.0.1.0/24$ und ein Gerät im Subnetz $10.0.2.0/24$ simuliert, welche miteinander kommunizieren). Beide Testcomputer haben zu diesem Zweck entsprechende Routing-Regeln eingetragen, sodass Daten entsprechend an den anderen Testcomputer weitergeleitet werden können. Je nach Test wird zwischen beiden Computern stattdessen ein (oder mehrere) IPsec-Tunnel aufgebaut, welcher alle Daten der Subnetze $10.0.1.0/24$ und $10.0.2.0/24$ verschlüsselt.

Das Testgerät verfügt über zwei Ports. Der erste Port (P1) ist dabei für das Versenden und Empfangen von Testdaten zuständig und wird in dieser Arbeit auch Testport genannt. Der Testport kann je nach Test auch über bis zu acht IP-Adressen verfügen, um mehrere Geräte gleichzeitig zu simulieren. Der zweite Port (P2) nutzt die Loopback-Funktion des Testgerätes und wird in dieser Arbeit auch Loopback-Port genannt. Der Loopback-Port schickt alle Daten mit vertauschten MAC- und IP-Adressen zurück.

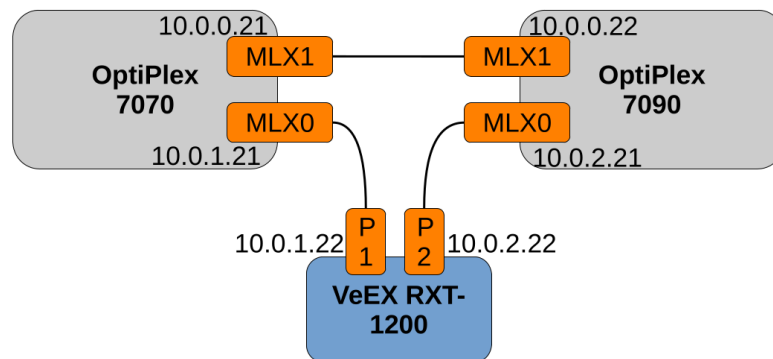


Abbildung 3.1: Der Aufbau des Testnetzwerkes. Die orangefarbenen Felder zeigen die jeweiligen Ethernet-Ports. Daneben findet sich die IP-Adresse, die von diesem Port genutzt wird. P1 ist der Testport und P2 ist der Loopback-Port des Testgeräts. MLX0 und MLX1 sind die Netzwerkinterfaces der Mellanox Karten.

Auf die Nutzung von Switches oder zusätzlichen Routern wurde verzichtet, da diese die Übertragungszeiten nur unnötig erhöhen würden. Außerdem würde die Nutzung von Hardware mit zu geringen Übertragungsraten den Durchsatz limitieren. Beides würde die Messergebnisse verfälschen und schlechtere Ergebnisse hervorbringen.

3.1.4 Funktionsweise des Systems

Die Testdaten werden auf dem Testgerät erzeugt und auf dem Testport versendet. Die Daten sind dabei an den Loopback-Port (10.0.2.22) adressiert. Sie erreichen den ersten Testcomputer über den ersten Port der Netzwerkkarte (MLX0) und werden hier an den zweiten Testcomputer, über den zweiten Port der Netzwerkkarte (MLX1), weitergeleitet. Auf dem zweiten Computer werden die Daten entsprechend (am Port MLX1) empfangen und (über Port MLX0) an den Loopback-Port des Testgeräts weitergeleitet. Dieser sendet die Daten zurück, jetzt an den Testport (meist 10.0.1.22) adressiert. Die Daten werden auf umgekehrtem Weg zurückgeschickt und erreichen den Testport. Ein Paket wird also, bis es wieder beim Testport ankommt, von jedem Testcomputer zwei mal empfangen und weitergeleitet und muss so insgesamt vier mal verarbeitet werden.

Wenn für einen Test ein IPsec-Tunnel aufgebaut werden soll, verhandeln die IKEv2-Daemons *charon*, die auf beiden Computern installiert sind, die entsprechenden Schlüssel und bauen einen IPsec-Tunnel zwischen den beiden Computern (zwischen 10.0.0.21 und 10.0.0.22) auf. Dafür installiert *charon* die entsprechenden Security Policies und Security Associations des Tunnels in der Security Policy Database (SAP) und der Security Association Database (SAD). Beide Computer agieren nun als kryptographische Endpunkte für den IPsec-Tunnel. Der IPsec-Tunnel verschlüsselt alle Daten aus den Subnetzen 10.0.1.0/24 und 10.0.2.0/24, also alle Daten die zwischen dem Testport und dem Loopback-Port versendet werden, mit AES-GCM als Verschlüsselungs- und Authentifizierungsalgorithmus. (Siehe auch Kapitel A.)

Die Testcomputer erkennen zu verschlüsselnde IP-Pakete anhand von Metadaten, wie IP-Adressen, Protokolle und Ports. In dieser Arbeit werden als Erkennungsmerkmal die IP-Adressen genutzt. Das Abfangen und Verschlüsseln der Pakete erfolgt im Linux-Network-Stack durch das

XFRM-Framework. Wenn ein Paket vom Testgerät empfangen wird (egal ob es vom Testport oder Loopback-Port kommt) und weitergeleitet werden muss, wird zuerst mit den Security Policies in der SPD geprüft ob dieses Paket durch den IPsec-Tunnel versendet werden soll. Ist dies der Fall, wird die benötigte Security Association aus der SAD ausgelesen und mit deren Informationen das IP-Paket verschlüsselt und anschließend an den anderen Testcomputer gesendet. Dort wird das Paket empfangen und mit der SA wieder entschlüsselt. Das nun wiederhergestellte ursprüngliche IP-Paket kann nun an den Zielport des Testgerätes gesendet werden. Da ein Paket vom Loopback-Port wieder zurückgesendet wird, muss jedes IP-Paket den IPsec-Tunnel insgesamt zweimal passieren und somit auch zweimal ver- und entschlüsselt werden.

IP-Pakete, welche nicht den IPsec-Tunnel passieren sollen, werden bei der Übertragung und Verarbeitung nicht behindert und normal weitergeleitet. Pakete, welche laut der SPD verschlüsselt sein müssten, dies aber nicht sind, werden vom empfangenden Testcomputer verworfen. Beide Fälle treten bei den in dieser Arbeit durchgeführten Tests aber nicht auf.

Bei der Verarbeitung von IP-Paketen können verschiedene Fehler auftreten, die dazu führen können, dass ein Paket verworfen wird. Dies passiert zum Beispiel, wenn die Ringbuffer der Netzwerkkarte voll sind und ein einkommendes Paket nicht mehr angenommen werden kann. Dies tritt insbesondere dann auf, wenn zu viele Pakete empfangen werden und die Testcomputer diese nicht schnell genug abarbeiten können. Da beide Testcomputer immer von beiden Ports gleichzeitig Pakete empfangen müssen, kann es passieren, dass dabei Pakete von einem der Ports gar nicht mehr empfangen werden können, da die Testcomputer bereits mit der Abarbeitung des anderen Ports voll ausgelastet sind. In diesem Fall kommt die Kommunikation komplett zum Erliegen. Ein einfacher Workaround für dieses Problem ist es, die Senderate bei den Tests zu reduzieren und weniger Pakete zu versenden.

Eine weitere Fehlerquelle ist die Ver- und Entschlüsselung. Wenn ein IP-Paket fehlerhaft ver- oder entschlüsselt wird, ist das resultierende IP-Paket fehlerhaft und meist kein gültiges IP-Paket mehr und kann somit nicht weiter verarbeitet werden. Dies tritt aber nur bei schwerwiegenden Implementierungsfehlern im IKE-Daemon, Linux-Kernel oder Treiber der Netzwerkkarte auf und ist somit in der Praxis nicht zu erwarten.

3.2 Limitierungen der Testumgebung

Aufgrund der verwendeten Hardware und dem Aufbau des Testnetzwerkes gibt es einige Limitationen, welche Tests durchgeführt werden können.

Da alle Daten die den IPsec Tunnel passieren vertraulich gehalten werden sollen, werden auch keine Tests mit Authentication Header durchgeführt, sondern nur mit Encapsulating Security Payload.

Die verwendeten Netzwerkkarten können nur bestimmte Algorithmen offloaden und schränken dadurch die Wahl der nutzbaren Verschlüsselungsalgorithmen ein. Insbesondere kann die Mellanox ConnectX-6 Dx nur Verschlüsselungen mit AES-GCM mit 128 oder 256 Bit Schlüsseln offloaden. Dies wird, neben der NULL-Encryption, entsprechend der einzige Verschlüsselungsalgorithmus sein, mit dem getestet wird.

Es ist möglich, IPsec im oben beschriebenen Netzwerk sowohl im Tunnel- als auch im Transport-Mode zu betreiben. Die Verwendung von Transport-Mode ist allerdings in der Praxis ebenfalls nur selten vertreten, da sich der Transport-Mode nicht für die Kommunikation und Absicherung der

Pakete zwischen Subnetzen eignet. Der Grund hierfür ist, dass der Transport-Mode den originalen IP-Header behält und größtenteils unverändert lässt und somit auch keine IP-Adressen ändert. Damit müssten beim Transport-Mode alle Router zwischen den beiden kryptographischen Endpunkten ebenfalls beide Subnetze kennen und über entsprechende Routing-Regeln verfügen. (Dies kann zwar mit NAT-Regeln behoben werden, würde aber einen größeren Overhead und damit geringere Performance bedeuten.) Im Tunnel-Mode erhält dagegen jedes Paket einen neuen IP-Header und damit müssen Router zwischen den Endpunkten nur noch die Endpunkte selbst kennen und benötigen keine Informationen mehr über die Subnetze. Außerdem schützt der Tunnel-Mode damit auch die Vertraulichkeit der originalen IP-Header und die darin enthaltenen Informationen, wie zum Beispiel die IP-Adressen der Geräte in den Subnetzen. Aus diesen Gründen wird in dieser Arbeit nur mit dem Tunnel-Mode getestet.

Das Testgerät selbst kann nur Ethernet-Frames und IP-Pakete generieren. Aus diesem Grund werden keine Tests mit InfiniBand als zugrundeliegendes Übertragungsmedium durchgeführt.

Das Testgerät ist in der Lage sowohl IPv4-Paket, als auch IPv6-Pakete, zu erzeugen. Allerdings kann dem Loopback-Port keine IPv6-Adresse zugewiesen werden, was es schwierig macht IPv6-Pakete an diesen zu senden. Aus diesem Grund werden keine Tests mit IPv6 durchgeführt und alle Tests nutzen IPv4.

Mellanox bietet neben Treibern für Linux auch Treiber für verschiedene Windows Versionen an. Diese Treiber haben allerdings nicht die benötigten Offloading-Funktionalität implementiert. Aus diesem Grund erfolgen alle Tests nur mit Linux als Betriebssystem.

Alternative Frameworks zur Arbeit mit Netzwerkkarten, wie zum Beispiel DPDK⁵ (Data Plane Development Kit), verfügen ebenfalls über Libraries zum Offloading der Verschlüsselung. Allerdings hat Mellanox bisher keine Treiber für diese bereitgestellt, sodass alternative Frameworks in dieser Arbeit nicht weiter betrachtet werden können. Die Verschlüsselung erfolgt somit nur durch den im Linux-Kernel integrierten Network-Stack.

3.3 Analyse des Testsystems

Im beschriebenen Testsystem gibt es zwei mögliche Situationen, bei denen Performanceverluste erwartet werden können. Einmal sind Performanceverluste durch den Aufbau und die Verwaltung des IPsec-Tunnels möglich. Während ein IPsec-Tunnel aufgebaut wird, können keine Daten über diesen Tunnel versendet werden. Dies wird zu Paketverlusten und damit Performanceeinbrüchen führen. Da die Security Associations für einen IPsec-Tunnel immer nur eine begrenzte Lebensdauer haben und anschließend neu aufgebaut werden müssen (dieser Prozess wird auch Rekeying genannt), kann auch während eines Tests die Lebenszeit eines Tunnels ablaufen. In diesem Fall muss der Tunnel erst neu aufgebaut werden bevor er wieder verwendet werden kann, was zu Störungen im Paketfluss führen kann. Ähnliches passiert wenn die IKE-Verbindung, deren SA ebenfalls nur eine begrenzte Lebensdauer haben und dann neu ausgehandelt werden müssen, zwischen beiden kryptographischen Endpunkten ausläuft.

Bei einer korrekten Konfiguration des IKEv2-Daemons ist allerdings nicht zu erwarten, dass dies messbare Effekte hat. Dafür gibt es mehrere Gründe. Einerseits werden auslaufende Tunnel und IKE-Verbindungen bereits ausgetauscht bevor ihre Lebensdauer abläuft. Dieser Austausch findet dann in-place statt. Es werden also neue SA verhandelt, aber die alten SA erst gelöscht

⁵<https://www.dpdk.org/>

sobald die neuen SA für die Verschlüsselung genutzt werden können.[KHN⁺14, Abschnitt 2.8] Damit kann die neue Verbindung direkt genutzt werden und es kommt nicht zu Ausfällen bei der Verfügbarkeit der IPsec-Tunnel. Der einzige Fall in dem dies nicht möglich ist, ist bei einer Neuauthentifizierung der IKE-SA. Hier muss die alte Verbindung tatsächlich beendet werden und anschließend eine neue IKE-SA mit neuen Child SA aufgebaut werden.

Ein weiterer Grund für die geringe Relevanz dieses Problems ist, dass der Aufbau einer neuen SA meist sehr schnell geht: Das Erstellen und Rekeyen einer Child SA, sowie das Rekeyen einer IKE-SA, benötigt ein einziges Nachrichtenpaar, bevor die neue SA genutzt werden kann. Das Erstellen einer IKE-SA und ersten Child-SA benötigt zwei Nachrichtenpaare. All dies setzt allerdings voraus, dass es bei der Aushandlung der SA keine Probleme gibt und alle vorgeschlagenen Einstellungen sofort vom Responder der IKE-Nachricht angenommen werden.[KHN⁺14, Abschnitt 1.2 und 1.3] Da auf beiden Testcomputern aber der gleiche IKEv2-Daemon (Strong-Swan's charon) installiert ist und auf beiden Computern die gleiche Konfiguration eingestellt ist, treten Uneinigkeiten bei der Aushandlung in dieser Arbeit nicht auf. Bedenkt man außerdem, dass beide Testcomputer direkt miteinander verbunden sind, sollte jede Kommunikation zwischen ihnen nur geringe Latenzen haben und die Aushandlung neuer SA somit keine nennenswerte Zeit in Anspruch nehmen.

Außerdem ist in der Praxis die Lebensdauer von SA vergleichsweise hoch (sehr oft deutlich höher als die in dieser Arbeit genutzte Testdauer von 5 Minuten) und eine Erneuerung der Schlüssel findet entsprechend nur selten statt. Bei langer Inbetriebnahme ist der Schlüsselaustausch entsprechend vernachlässigbar und stört den Normalbetrieb nicht weiter. Aus all diesen Gründen ist durch den Schlüsselaustausch kein messbarer Performanceverlust zu erwarten und er wird in dieser Arbeit (und bei den Messungen) nicht mehr mit betrachtet.

Weitaus größere Performanceverluste sind bei der Ver- und Entschlüsselung der Pakete zu erwarten, wenn diese den IPsec-Tunnel betreten oder verlassen müssen. Bei der Verschlüsselung muss dabei sowohl die SPD, als auch die SAD, angefragt werden, ob und wie ein Paket verschlüsselt werden soll. Bei der Entschlüsselung muss ebenfalls erst die SAD angefragt werden, wie das Paket entschlüsselt werden soll, und anschließend die SPD, ob das entschlüsselte Paket tatsächlich aus dem korrekten Subnetz kam. Dies führt zu Verzögerungen bei der Paketverarbeitung, welche, unabhängig von den gewählten Algorithmen, immer dazu kommen, sobald IPsec aktiviert ist.

Dazu kommt noch der extra Zeitaufwand durch die Kapselung in einem ESP-Paket und die Anwendung des Ver- und Entschlüsselungsalgorithmus, sowie des Authentifizierungsalgorithmus. Allerdings ist AES-GCM ein kombinierter Verschlüsselungs- und Authentifizierungsalgorithmus und ist im Vergleich mit anderen Algorithmen relativ schnell, da er gut parallelisiert werden kann.

Bei der Nutzung der Offloading-Funktionalität sind außerdem deutliche Performancegewinne zu erwarten, da in Hardware implementierte Algorithmen meist besser zeitlich optimiert sind, als gleiche Algorithmen in einer Software-Implementierung auf General-Purpose Hardware, wie zum Beispiel einer CPU.

In der Praxis können auch noch weitere Ursachen für Performanceverluste vorhanden sein, wie zum Beispiel die Filterung von Paketen mit nftables. Diese haben allerdings nur wenig mit IPsec zu tun und werden entsprechend nicht in dieser Arbeit betrachtet.

3.4 Messwerte

Der wichtigste Messwert, der bei den Messungen erhoben wird, ist der erzielte Durchsatz. Mithilfe des Durchsatzes kann ermittelt werden, ob und in welchem Ausmaß die Verschlüsselung die Verarbeitungsgeschwindigkeit der Pakete und damit den übertragbaren Datenverkehr beeinflusst. Der Durchsatz wird in Gbit/s angegeben und ergibt sich aus den Payloads der Ethernet-Frames, die erfolgreich (also ohne Fehler) wieder beim Testgerät ankommen. Gemessen wird dabei die Größe des gesamten Ethernet-Frames, inklusive Header und Prüfsumme. Ethernet-Frames, welche nicht Teil des Tests sind und nicht vom Testgerät erzeugt wurden, beispielsweise Pause-Frames oder ARP-Anfragen, werden nicht dazu gezählt. Es ist zu erwarten, dass solche Nicht-Test Frames nur selten auftreten und den Durchsatz nicht messbar beeinflussen.

Die Latenz wird gemessen, um zu ermitteln, wie viel Zeit die Verschlüsselung pro Paket zusätzlich in Anspruch nimmt. Die Angabe erfolgt dabei in Mikrosekunden (μs). Die Latenz folgt dabei der Definition aus RFC 1242[Bra91, Abschnitt 3.8]:

The time interval starting when the end of the first bit of the input frame reaches the input port and ending when the start of the first bit of the output frame is seen on the output port.

Die Latenz wird als Round-Trip Delay (kurz: RTD) gemessen. Das bedeutet, es wird die Zeit gemessen, bis ein Frame wieder beim Testport des Testgerätes ankommt.

Um zu ermitteln, wenn und wie viele Pakete fehlerhaft oder gar nicht verschlüsselt werden, wird die Anzahl der fehlerhaften und verlorenen Pakete gemessen. Ebenso werden Paketdopplungen und Out-of-Order-Pakete (Pakete welche in der falschen Reihenfolge empfangen werden) gemessen, die durch Fehler in der Übertragung entstehen können. Fehlerhafte, verlorene, duplizierte und Out-of-Order-Pakete können, je nachdem wie die genutzte Anwendung auf solche Pakete reagiert, alle zu Performanceproblemen und sogar Fehlern in der Anwendung führen. Aus diesem Grund sind solche Pakete generell unerwünscht und es ist wichtig, ihre entsprechende Häufigkeit zu kennen, um die Auswirkung möglicher Probleme abschätzen zu können. Im Optimalfall erzeugt ein System überhaupt keine fehlerhaften Pakete. Beim genutzten Testaufbau werden nur wenige Punkt-zu-Punkt-Verbindungen genutzt (und keine Switches oder ähnliche Ethernet-Geräte), weshalb Paketdopplungen und fehlerhafte Pakete eher unwahrscheinlich sind.

Insbesondere beim Offloading der Verschlüsselung auf die Netzwerkkarte wird die CPU entlastet. Um die entstehenden CPU-Lasten durch die Verschlüsselung zu vergleichen, wird auch die durchschnittliche CPU-Auslastung gemessen. Für die Messung der CPU-Auslastung wird der Befehl `mpstat` verwendet. Die Messung erfolgt dabei auf dem OptiPlex 7070.

3.5 Parameter und Faktoren

Es werden zwischen System- und Workloadparametern unterschieden. Systemparameter sind dabei Einstellungen, welche an den beiden Testcomputern und am Testaufbau vorgenommen werden und das Verhalten der Testcomputer ändern. Workloadparameter wiederum sind Einstellungen des Workloads, werden also am Testgerät vorgenommen und haben Einfluss auf die versendeten Testdaten. Parameter welche zwischen den Messungen geändert werden, werden auch Faktoren genannt.

Beim Testsystem sind die einflussreichsten Parameter bei der Konfiguration der IPsec-Tunnel zu erwarten. Am relevantesten ist hierbei der gewählte Verschlüsselungsalgorithmus, welcher in dieser Arbeit entweder eine NULL-Encryption oder eine AES-GCM Verschlüsselung ist. Wenn AES-GCM als Verschlüsselungsalgorithmus gewählt wird, wird auch dessen Schlüsselgröße (welche entweder 128 oder 256 Bit sein kann) variiert. AES schreibt bei der Nutzung von 256 Bit langen Schlüsseln mehr Verschlüsselungsrunden vor als mit 128 Bit langen Schlüsseln (14 gegen 10 Runden), weshalb bei einer höheren Schlüsselgröße eine schlechtere Performance zu erwarten ist. Der gewählte Algorithmus mit Schlüsselgröße ist deshalb ein variabler Faktor.

Ein weiterer wichtiger Parameter bei der Konfiguration der IPsec-Tunnel ist die Nutzung von Hardware-Offloading. Bei aktiviertem Hardware-Offloading ist eine bessere Performance zu erwarten, als bei einer Verschlüsselung auf der CPU. Entsprechend ist die Nutzung von Hardware-Offloading ein weiterer Faktor.

Außerdem ist der gewählte Modus der ESP-Pakete, also Tunnel- oder Transport-Mode, ein weiterer Systemparameter. Dieser wird in dieser Arbeit aber nicht variiert und ESP-Pakete nutzen immer den Tunnel-Mode. Weitere Features von IKEv2, wie das Rekeying und Reauthentication nach einer bestimmten Zeitspanne und die Dead-Peer-Detection, sind bei allen Messungen deaktiviert.

Die genutzte MTU (Maximum Transmission Unit) ist dabei meist standardmäßig auf 1500 Bytes eingestellt. Nur wenn mit Paketen, die größer als 1500 Bytes sind, getestet wird (also Framegrößen von 4096 oder 9000 Bytes, siehe unten), wird die MTU auf den beiden Testcomputern auf 9000 Bytes erhöht.

Bei den Workloadparametern ist zu erwarten, dass der einflussreichste Parameter die Framegröße ist. Diese beeinflusst, wie viele Pakete verschlüsselt werden müssen. (Bei größerer Framegröße werden bei gleicher Senderate weniger Frames versendet.) Die Framegröße ist deshalb ein Workload-Faktor und nimmt dabei hauptsächlich folgende Werte an: 68, 128, 256, 512, 1024, 1400 und 1518 Bytes. Einige der Messungen werden aber auch mit Framegrößen von 4096 und 9000 Bytes durchgeführt. Außerdem wird auch mit dem IMix aus Tabelle 3.1 gemessen.

Framegröße	68	100	373	570	1300	1518
absolute Gewichtung	3	26	6	5	6	16
relative Gewichtung	4,8%	41,9%	9,7%	8,0%	9,7%	25,8%

Tabelle 3.1: Der für die Messungen verwendete Internet-Mix. Alle Frames in diesem Mix haben eine zufällige Größe, folgen aber insgesamt der angegebenen Wahrscheinlichkeit. Ein Frame in diesem Mix ist im Durchschnitt 464 Bytes groß. Die relative Gewichtung ist auf eine Nachkommastelle gerundet.

Bei Nutzung einer Standard MTU von 1500 Bytes kommt es bei Messungen mit einer Framegröße von 1518 oder mehr Bytes zur IP-Fragmentierung (ein IP-Paket, welches größer als die MTU ist, wird in mehrere kleinere IP-Pakete aufgeteilt, welche alle kleiner oder gleich der MTU sind) nach der Verschlüsselung, da durch die Kapselung in einem ESP-Paket die MTU überschritten wird.⁶ Die Framegröße von 1400 Bytes wurde entsprechend gewählt, um möglichst große Frames zu erzeugen, bei denen allerdings keine Fragmentierung nach der Kapselung auftritt.

⁶Die übertragenen IP-Pakete sind nach der Kapselung mit ESP um mindestens 54-57 Bytes größer als das ursprüngliche IP-Paket. Dies setzt sich zusammen aus 34-37 Bytes, welche durch ein ESP-Paket mit AES-GCM hinzukommen

Neben der Framegröße sind auch die verwendeten Protokolle der Frames ein Workload-Parameter. In allen Messungen dieser Arbeit wurden nur Ethernet-Frames mit IPv4-Paketen verwendet. Messungen mit IPv6 oder Layer 4 Protokollen (UDP oder TCP) erfolgen nicht.

Neben der Framegröße ist auch die Senderate ein Workload-Faktor. Sie wird in Gbit/s angegeben und bestimmt wie viele Frames pro Sekunde verschickt werden. Die Senderate enthält dabei nicht nur den Payload der Frames, sondern auch den Frame-Header und den Inter-Frame-Gap, also alle Bits die über Ethernet übertragen werden. Sie hat einen vergleichsweise kleinen Einfluss auf die Performance, kann aber ebenfalls die Performance beeinflussen, wenn sie höher als der erzielte Durchsatz ist. In diesem Fall arbeitet das System unter stark erhöhter Last, was zu Verlusten in der Performance führen kann. Um Überlastung zu vermeiden wird die Senderate deswegen in den meisten Messungen gering gehalten. Eine geringe Senderate begrenzt allerdings den maximal erreichbaren Durchsatz. Es muss bei der Wahl der Senderate entsprechend auch darauf geachtet werden, dass der Durchsatz nicht negativ beeinflusst wird.

Zuletzt ist auch die Anzahl der verschiedenen Streams und IPsec-Tunnel relevant. Unterschiedliche Streams nutzen dabei unterschiedliche IP-Adressen, werden aber vom System gleich behandelt, werden also an die gleichen Schnittstellen geleitet. Auf dem Testgerät können bis zu acht Streams gleichzeitig konfiguriert werden. Die Nutzung mehrerer Streams und IPsec-Tunnel ermöglicht es, diese parallel und auf mehreren Cores abzuarbeiten, was sich positiv auf die Performance auswirken kann. Entsprechend ist die Anzahl der Streams und IPsec-Tunnel ein weiterer Faktor.

Wie hoch der Performancegewinn durch die Parallelisierung ist, hängt von der Aufteilung der Interrupts auf die verschiedenen Cores ab. Normalerweise ist die Empfehlung von Mellanox, immer einen Interrupt pro Core zuzuweisen, aber das IRQ-Balancing verändert die Zuweisungen konstant und folgt dieser Empfehlung somit nicht. Entsprechend kann aktiviertes IRQ-Balancing einen starken Einfluss auf die Performance bei der Parallelisierung haben und ist ein weiterer Faktor.

3.6 Durchgeführte Messungen

In diesem Kapitel werden die durchgeführten Messexperimente vorgestellt. Die dabei erzielten Messergebnisse sind im nachfolgenden Kapitel zu finden. Alle Messungen, die mit dem Durchsatztest durchgeführt werden, haben eine Dauer von 5 Minuten und werden 3 mal wiederholt. Messungen mit dem RFC 2544 Test haben pro Schritt eine Dauer von 1 Minute und werden ebenfalls 3 mal wiederholt. Da bei diesen Messungen nur ein sehr geringer Zufallsgrad besteht sind mehr Wiederholungen nicht nötig.

Experiment 1: variable Senderate

Im ersten Experiment wird der Effekt der Senderate auf den Durchsatz ermittelt. Hierbei soll untersucht werden, ob die Netzwerkkarte auch unter hoher Last noch ohne Verluste bei der Performance arbeiten kann.

(für Details siehe RFC 4106[VM05, Abschnitt 7]), und, da in dieser Arbeit der Tunnel-Mode genutzt wird, weiteren 20 Bytes für einen zusätzlichen IPv4-Header. Bei der Verschlüsselung mit AES-GCM bleibt die Länge der Daten identisch und der Payload somit von gleicher Größe.

Dafür wird ein IPsec-Tunnel zwischen beiden Testcomputern aufgebaut und der dafür genutzte Verschlüsselungsalgorithmus ist AES-GCM mit 128 Bit Schlüsseln. Der Test wird dabei je einmal mit Hardware-Offloading und einmal ohne Offloading durchgeführt. Die Senderate wird dabei zwischen 10% und 100% in je 10% Schritten variiert. Außerdem wird auch die Framegröße zwischen 68, 128 und 1400 Byte variiert. Insgesamt werden also $2 * 10 * 3 = 60$ Messungen durchgeführt.

Um zu verhindern, dass die Senderate durch Pause-Frames beeinflusst wird, wird für diese Messungen das Versenden von Pause-Frames auf den beiden Testcomputern deaktiviert.

Für die Messungen des Durchsatzes werden Durchsatztests verwendet. Zusätzlich wird parallel auch die CPU-Auslastung mit gemessen.

Experiment 2: Vergleich AES-GCM-Implementierungen

In diesen Messungen werden die verschiedenen Implementierungen von AES-GCM verglichen. Der Großteil der gemessenen Daten in dieser Arbeit wurden in diesem Experiment erhoben.

Bei diesen Messungen wird zwischen sechs verschiedenen Modi variiert. Um eine Baseline zu bilden wird einmal ein Test komplett ohne die Nutzung von IPsec-Tunneln durchgeführt. Die Übertragung der Pakete erfolgt dann nur durch Routing Regeln. Zusätzlich wird einmal ein Test mit NULL-Encryption als Verschlüsselungsalgorithmus durchgeführt. Daraus kann ermittelt werden, wie viel Performance durch die Verwendung von IPsec verloren geht. Die in diesen Messungen verlorene Performance kann auch durch Offloading nur sehr schwer bis gar nicht verringert werden. Die letzten vier Tests nutzen alle AES-GCM als Verschlüsselungsalgorithmus. Dabei werden je zwei Tests mit 128 Bit und zwei Tests mit 256 Bit Schlüsseln durchgeführt. In beiden Fällen wird je ein Test ohne Hardware-Offloading und ein Test mit Hardware-Offloading durchgeführt.

In jedem Tests wird die verwendete Framegröße variiert. Die genutzten Framegrößen sind dabei: 68, 128, 256, 512, 1024, 1400 und 1518 Byte. Außerdem wird mit dem obigen IMix (Tabelle 3.1) getestet. Zusätzlich werden auch Tests mit Framegrößen von 4096 und 9000 Bytes durchgeführt, allerdings wird hierfür die MTU zwischen den Testcomputern und dem Testgerät auf 9000 Byte erhöht. Insgesamt müssen also $6 * 10 = 60$ Messungen durchgeführt werden.

Es werden in diesem Experiment alle vier oben erwähnten Messwerte erhoben. Der Durchsatz und die CPU-Auslastung werden durch Durchsatztests ermittelt und die Latenz sowie Lost-Frame durch RFC 2544 Tests.

Experiment 3: Parallelisierung

In diesem Experiment ist das Ziel, die Effekte der Parallelisierung herauszufinden und zu prüfen, ob ein Performancegewinn durch die Aufteilung der übertragenen Daten auf mehrere Streams möglich ist. Zusätzlich soll überprüft werden, ob die Nutzung mehrerer IPsec-Tunnel parallel einen Performancegewinn ermöglicht und, wenn ja, ob es eine optimale Anzahl an IPsec-Tunneln gibt.

Es wird mit AES-GCM Verschlüsselung mit Schlüsselgrößen von 128 Bit getestet.

Hierbei werden alle Tests in diversen Varianten durchgeführt. Zuerst wird je einmal mit einer Framegröße von 68 Byte und mit einer Framegröße von 1400 Byte getestet. Auch wird je einmal mit aktivierten IRQ-Balancing und einmal ohne IRQ-Balancing getestet. Wenn IRQ-Balancing deaktiviert ist, wird ein 1:1-Mapping von Interrupt-Queues auf zuständige Cores genutzt. (Jeder Core

ist also nur für genau eine Queue zuständig.) Außerdem werden alle Tests je einmal mit nur einem IPsec-Tunnel durchgeführt, der für alle Daten zuständig ist, und mit mehreren IPsec-Tunneln (Tunnelanzahl entspricht dabei der Streamanzahl), zwischen denen die Daten gleichmäßig aufgeteilt werden. Zusätzlich werden alle Tests einmal mit und einmal ohne aktiviertes Hardware-Offloading durchgeführt.

Die verwendeten Streamanzahlen sind dabei: 1, 2, 4 und 8. Insgesamt müssen also $2^4 * 4 = 64$ Messungen durchgeführt werden.

Gemessen werden hierbei der Durchsatz und die CPU-Auslastung, ebenfalls mithilfe von Durchsatztests.

4 Messergebnisse und Auswertung

In diesem Kapitel werden die erhobenen Messergebnisse aus den Experimenten ausgewertet. Der Fokus liegt dabei auf dem Effekt, den Offloading auf den Durchsatz hat. Außerdem werden in diesem Kapitel mögliche Ursachen für Performanceverluste erläutert.

4.1 Variable Senderate

Zuerst wird der Effekt der Senderate Netzwerktesters auf den Durchsatz ermittelt. Dafür wird die Senderate zwischen 1 und 100 Gbit/s variiert und der erreichte Durchsatz ermittelt. Hierbei ist insbesondere der prozentuale Unterschied zwischen verschiedenen Senderaten interessant. Für diesen Test ist IPsec aktiviert und alle Pakete werden mit AES-GCM (Schlüssellänge 128 Bit) verschlüsselt. Dieser Test wird für verschiedene Framegrößen wiederholt, um Unterschiede zwischen Framegrößen feststellen zu können. Außerdem wird jeder Test je einmal ohne und einmal mit Unterstützung durch Hardware-Offloading durchgeführt.

Der erreichte Durchsatz ist in Abbildung 4.1 zu sehen, außerdem ist eine Darstellung des relativen Durchsatzes für jede Testvariante in Abbildung 4.3 zu sehen. Eine tabellarische Darstellung ist in Tabelle B.1 zu finden. Es ist zu erkennen, dass der Durchsatz bei einer zu hohen Senderate einbricht. Ab wann der Durchsatz einbricht ist allerdings bei jeder Framegröße unterschiedlich und je höher die Framegröße ist, umso später bricht der Durchsatz ein. Bei einer Framegröße von 68 Byte beginnt der Einbruch ab einer Senderate von 30 Gbit/s, bei einer Framegröße von 128 Byte allerdings erst ab einer Senderate von 50 Gbit/s. Bei Framegrößen von 256 Byte und mehr gibt es keinen Einbruch mehr. Da auch bei einer Framegröße von 1400 Byte kein Einbruch mehr festzustellen ist, scheint der Durchsatz nur bei kleinen Frames einzubrechen. Da bei kleineren Framegrößen mehr Frames versendet werden müssen, um die gleiche Senderate zu erreichen, kann gefolgert werden, dass der Einbruch des Durchsatzes erst ab einer bestimmten Anzahl an versendeten Frames pro Sekunde auftritt.

Sowohl bei einer Framegröße von 68 Byte, als auch bei 128 Byte, findet der Einbruch mit Hardware-Offloading zum gleichen Zeitpunkt statt. Bei beiden Framegrößen ist der Einbruch mit Hardware-Offloading ähnlich stark, wie ohne Hardware-Offloading: Bei einer Framegröße von 68 Bytes und einer Senderate von 100 Gbit/s wird in beiden Fällen nur rund 29,4% des maximalen Durchsatzes erreicht und bei einer Framegröße von 128 Byte nur 51%. Hardware-Offloading hat also keinen Effekt darauf, wann der Einbruch stattfindet und wie stark er ist.

Diese Einbrüche sind ein bereits bekanntes Problem bei Mellanox und lassen sich auf einen Fehler in der verwendeten Linux-Kernel Version zurückführen.¹ Der besagte Fehler wurde in Kernel-Version 5.8² allerdings behoben, sollte bei aktuellen Systemen also nicht mehr auftreten. Ob dies

¹siehe auch: <https://docs.nvidia.com/networking/display/MLNXOFEDv561033/Known+Issues>, Internal Ref. Number: 2143007

²durch Commit 94579ac3f6d0820adc83b5dc5358ead0158101e9

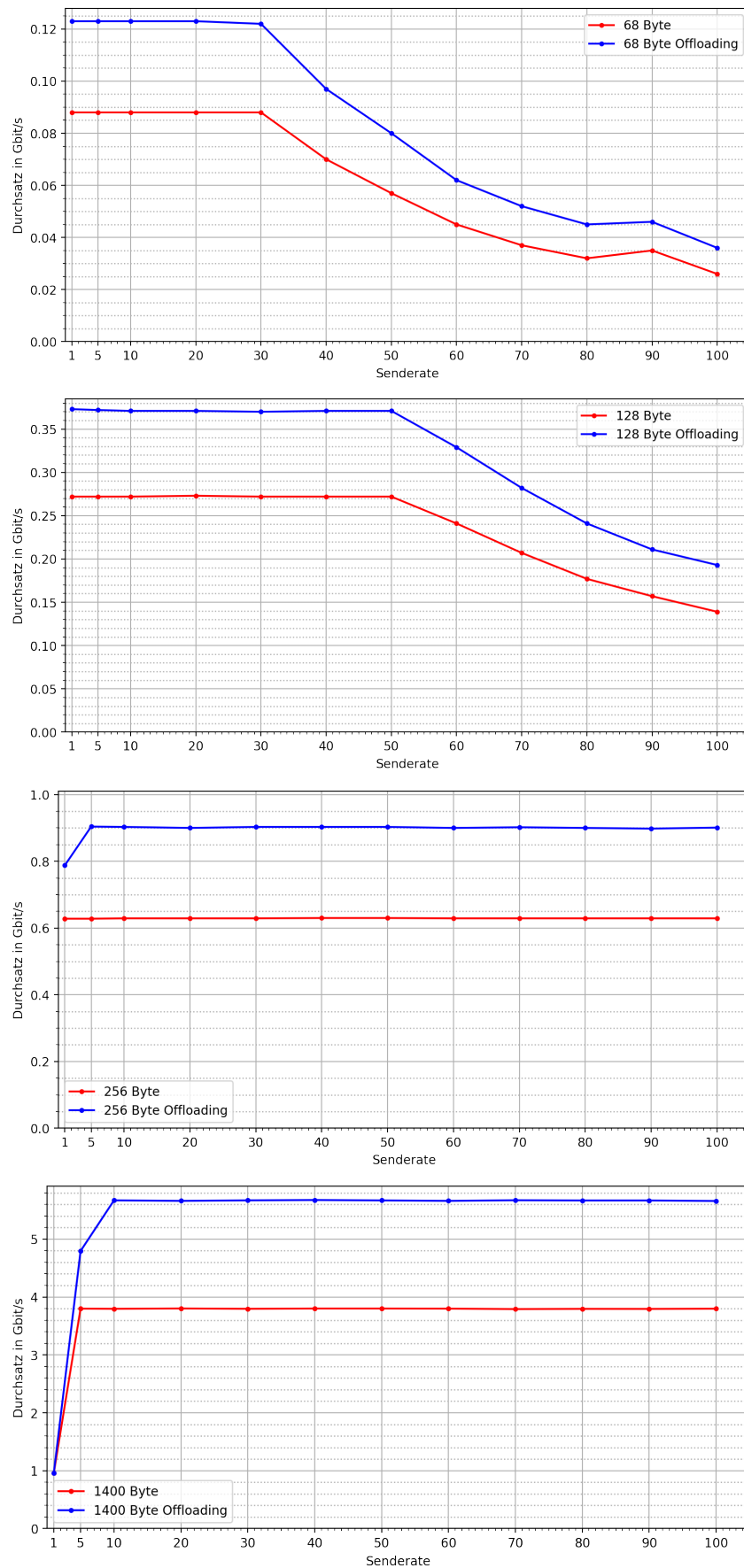


Abbildung 4.1: Der absolute erreichte Durchsatz bei verschiedenen Senderaten und Framegrößen, in Gbit/s. IPsec ist aktiviert und der gewählte Verschlüsselungsalgorithmus ist AES-GCM.

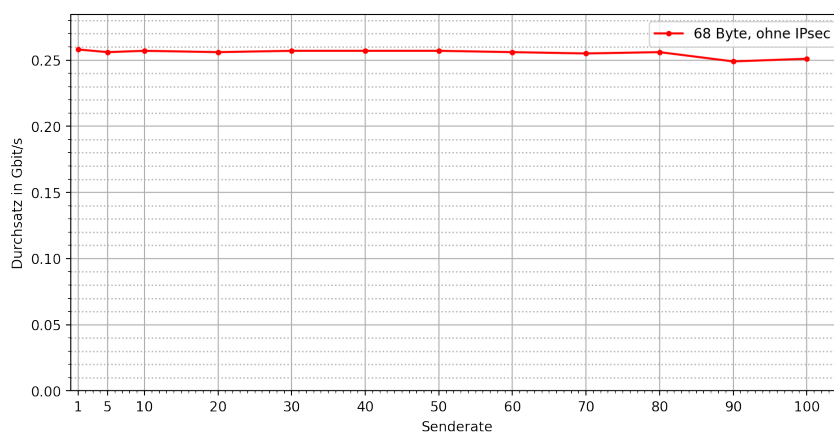


Abbildung 4.2: Der absolute erreichte Durchsatz bei verschiedenen Senderaten, in Gbit/s. Die Framegröße ist 68 Byte und IPsec ist nicht aktiviert.

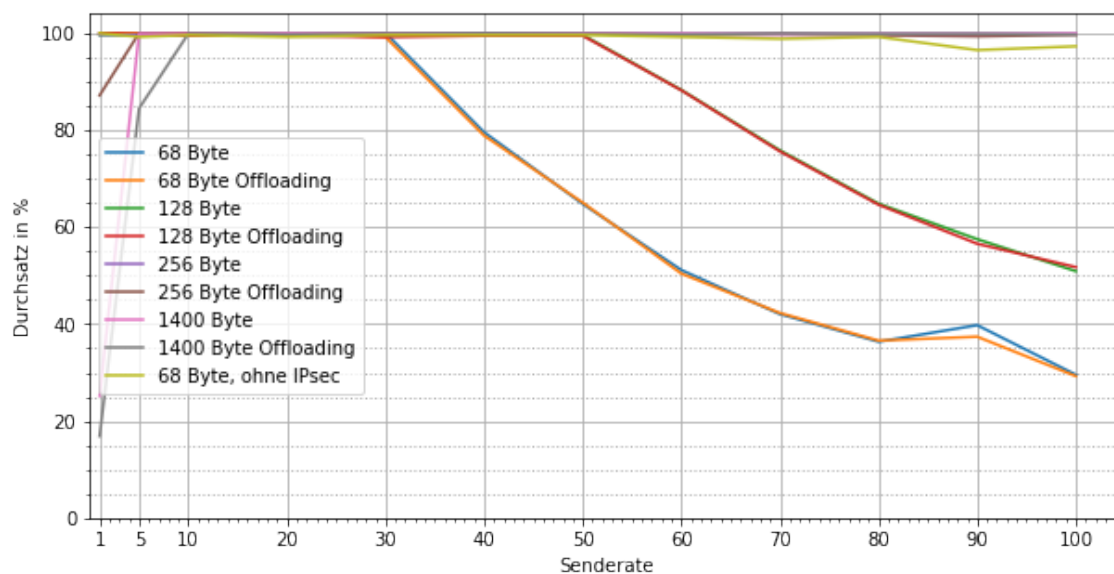


Abbildung 4.3: Der relative erreichte Durchsatz bei verschiedenen Senderaten und Framegrößen. Die Angabe des Durchsatzes ist relativ zum maximal erreichten Durchsatz für diese Variante (Framegröße + Offloading). Die Unterschiede zwischen aktivierten und deaktivierten Offloading sind teils so gering, dass die Graphen aufeinander liegen.

tatsächlich der Fall ist, konnte im Rahmen dieser Arbeit nicht mehr getestet werden. Wie im Fehlerbericht angemerkt tritt der Fehler aber nur bei der Nutzung von IPsec auf. Messungen ohne die Nutzung von IPsec sind in Abbildung 4.2 zu sehen und zeigen keinen Rückgang des Durchsatzes.

Eine Besonderheit tritt bei einer Framegröße von 68 Byte und einer Senderate von 90 Gbit/s (sowohl mit, als auch ohne Offloading) auf: Hier ist der erreichte Durchsatz leicht höher als erwartet. Vergleicht man den Durchsatz bei einer Senderate von 80 und 90 Gbit/s fällt auf, dass bei letzterem der Durchsatz um 1 bis 3 Mbit/s ansteigt. Bei einer Senderate von 100 Gbit/s ist der erreichte Durchsatz wiederum um rund 10 Mbit/s geringer. Bei anderen Framegrößen tritt dieser Effekt allerdings nicht auf. Woher diese Unregelmäßigkeit kommt konnte im Rahmen dieser Arbeit nicht geklärt werden.

4.2 Vergleich verschiedener AES-GCM Implementierungen

Der Hauptteil der Messungen setzte sich mit dem Vergleich der Performance mit und ohne Offloading der Verschlüsselung unter verschiedenen Framegrößen auseinander. Es wird dabei erst einmal der Durchsatz ohne eine Verschlüsselung mit IPsec und eine Verschlüsselung mit NULL-Encryption verglichen, um den Performanceverlust durch die Verarbeitung mit IPsec bestimmen zu können. Anschließend wird der Durchsatz bei einer NULL-Encryption mit dem Durchsatz bei einer AES-GCM Verschlüsselung verglichen. Dabei wird auch der Effekt des Hardware-Offloading untersucht. Abschließend werden Messergebnisse zum Effekt der IP-Fragmentierung und mit dem IMix vorgestellt.

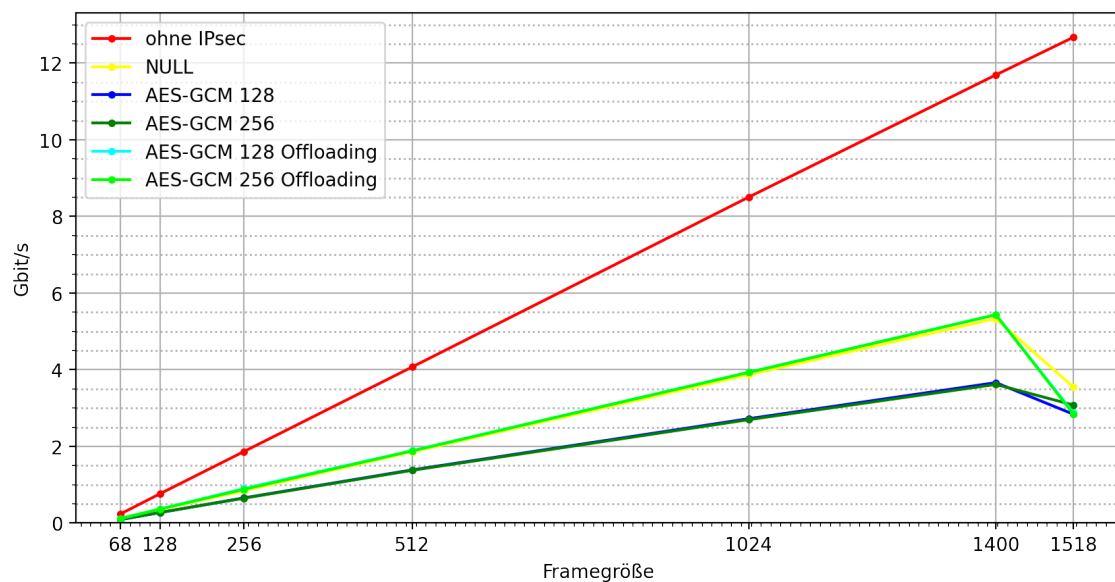


Abbildung 4.4: Vergleich des erzielten Durchsatzes (in Gbit/s) bei verschiedenen Framegrößen und für verschiedene Testversionen. Unterschiede zwischen den genutzten Schlüsselgrößen sind so gering, dass die jeweiligen Graphen teilweise aufeinander liegen.

Frame- größe	ohne IPsec	NULL	AES-GCM 128	AES-GCM 256	AES-GCM 128 Offloading	AES-GCM 256 Offloading
68	0.237	0.097	0.092	0.090	0.113	0.121
128	0.762	0.352	0.274	0.273	0.364	0.362
256	1.867	0.842	0.656	0.648	0.900	0.879
512	4.073	1.859	1.387	1.376	1.889	1.883
1024	8.505	3.872	2.722	2.698	3.934	3.930
1400	11.693	5.336	3.664	3.618	5.438	5.433
1518	12.676	3.546	2.840	3.079	2.874	2.843

Tabelle 4.1: Vergleich des erzielten Durchsatzes (in Gbit/s) bei verschiedenen Framegrößen und für verschiedene Testversionen. Die Framegrößen sind in Byte angegeben. Alle Werte sind auf drei Nachkommastellen gerundet.

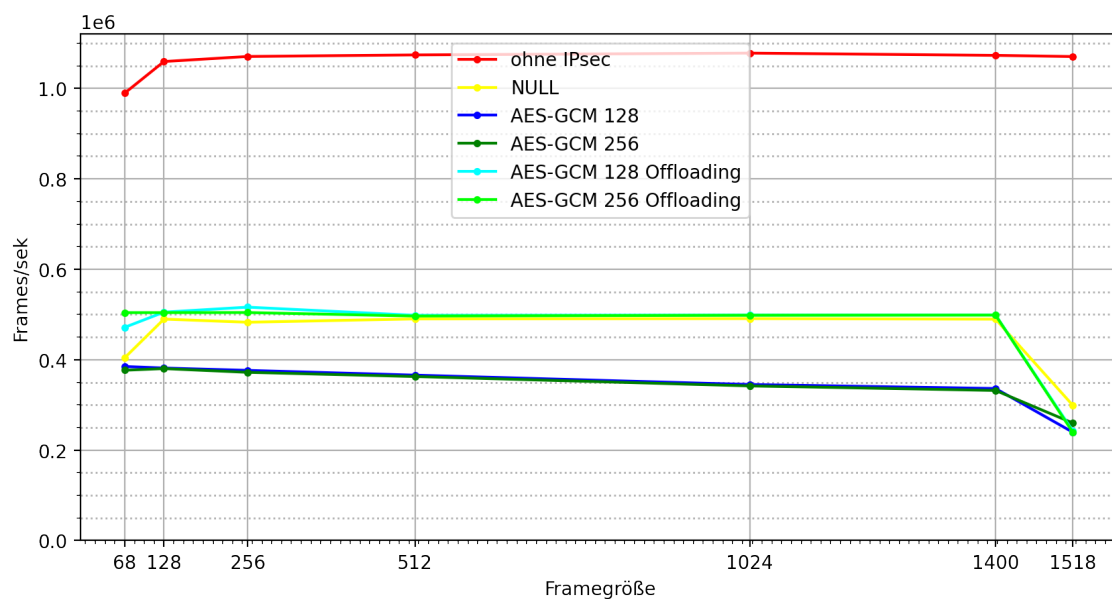


Abbildung 4.5: Vergleich der durchschnittlichen Frames pro Sekunde bei verschiedenen Framegrößen und für verschiedene Testversionen.

Frame- größe	ohne IPsec	NULL	AES-GCM 128	AES-GCM 256	AES-GCM 128 Offloading	AES-GCM 256 Offloading
68	989735	404407	384991	376738	471645	504143
128	1059556	489784	381574	380277	505198	504152
256	1070717	482857	376382	372070	516221	504324
512	1074160	490210	365836	362758	498157	496535
1024	1078146	490865	345134	342002	498770	498290
1400	1073153	489694	336266	331983	499053	498651
1518	1070568	299504	239830	260075	242756	240113

Tabelle 4.2: Vergleich der durchschnittlichen Frames pro Sekunde bei verschiedenen Framegrößen und für verschiedene Testversionen. Die Framegrößen sind in Byte angegeben. Alle Werte sind auf Ganzzahlen gerundet.

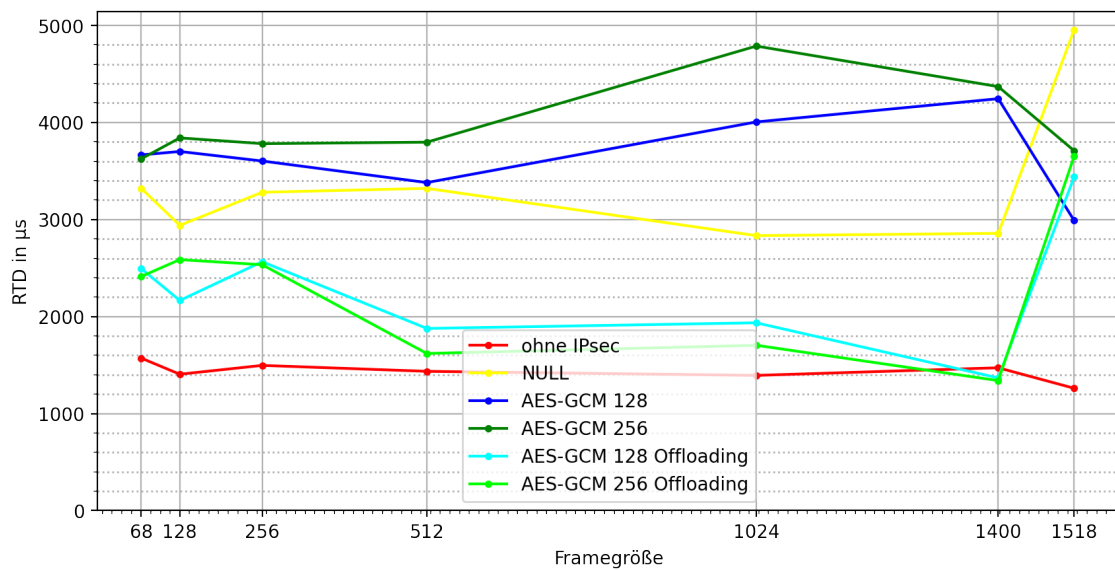


Abbildung 4.6: Round-Trip-Delay bei verschiedenen Framegrößen und für verschiedene Testversionen.

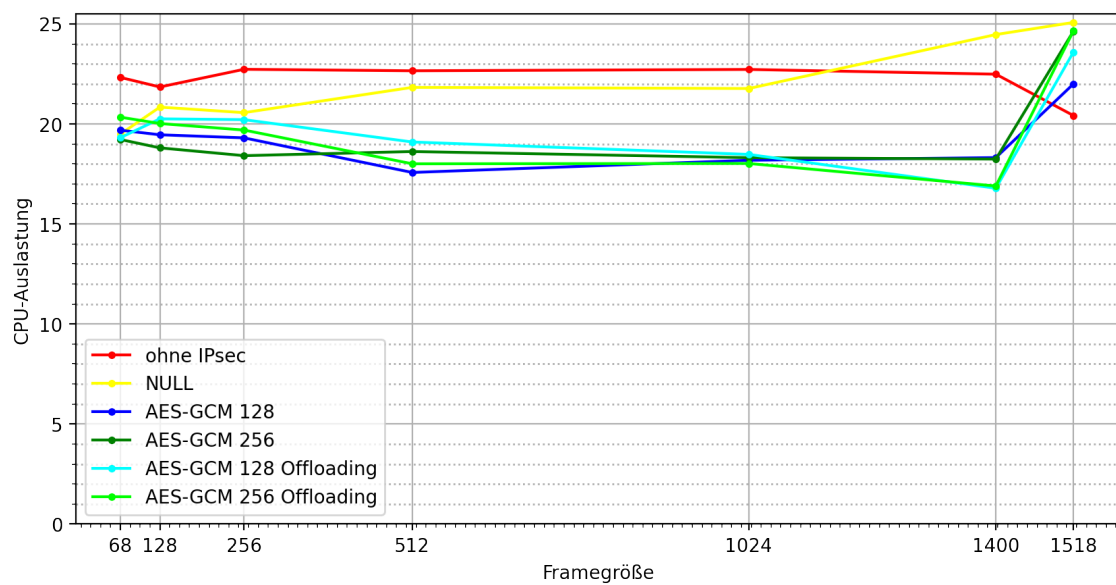


Abbildung 4.7: Durchschnittlich verwendete CPU-Zeit bei verschiedenen Framegrößen und für verschiedene Testversionen. Gemessen wurde die gesamte Zeit in der die CPU gerechnet hat, über alle Cores hinweg. (Einschließlich der benötigten Zeit für weitere Prozesse, die nichts mit der Übertragung von Paketen zu tun haben.) 100% würde entsprechend eine volle Auslastung aller Cores sein und 12,5% wäre eine Vollaustung eines Cores.

4.2.1 Verarbeitung durch IPsec

Zuerst werden zwei Baseline-Tests durchgeführt, um den maximal erreichbaren Durchsatz (ohne Verluste durch eine Verschlüsselung) zu ermitteln. Einmal wurde dabei IPsec komplett deaktiviert und einmal wurde IPsec mit einer NULL-Encryption und ohne Authentifizierung genutzt. Dadurch kann ermittelt werden, wie viel Zeit bereits durch die Verarbeitung eines Paketes mit IPsec (insbesondere das Vergleichen von Paketen mit den vorhandenen Security Policies, das Identifizieren der zuständigen Security Association und die Kapselung eines Paketes mit ESP) benötigt wird. In diesem Kapitel werden nur Framegrößen bis 1400 Byte betrachtet.

Eine Darstellung des erzielten Durchsatzes ist in Abbildung 4.4 und in Tabelle 4.1 zu sehen. In Abbildung 4.5 und Tabelle 4.2 ist der Durchsatz in Frames/sek dargestellt. Es zeigt sich, dass der Durchsatz um bis zu 55% einbricht, sobald IPsec aktiviert wird. Außerdem ist zu erkennen, dass der Durchsatz in beiden Fällen linear mit der Paketgröße ansteigt. Ohne IPsec wird ein Durchsatz von bis zu 11,7 Gbit/s erreicht und mit einer NULL-Encryption bis zu 5,3 Gbit/s (jeweils mit einer Framegröße von 1400 Byte). Die Anzahl der Frames, die pro Sekunde versendet werden, bleibt allerdings in beiden Fällen konstant. Ohne IPsec werden bis zu 1,1 Millionen Pakete pro Sekunde versendet, bei einer NULL-Encryption allerdings nur noch 490.000. Es ist also erneut ein Verlust von knapp 55% festzustellen. Dies bedeutet, dass der Durchsatz hauptsächlich durch die Anzahl der Frames, die bearbeitet werden können, limitiert ist. Die Framegröße selbst scheint für die Verarbeitungsdauer eines Paketes vergleichsweise irrelevant zu sein.

Diese Vermutung bestätigt sich beim Round-Trip-Delay. Dieser ist in Abbildung 4.6 zu sehen. Der Round-Trip-Delay ist mehr als doppelt so groß und steigt von bis zu $1571\mu\text{s}$ auf bis zu $3321\mu\text{s}$ an. Über die Framegrößen hinweg bleibt er aber weitestgehend konstant. Ein großer Teil der Rechenzeit bei der Übertragung der Pakete wird also bereits für die Verarbeitung von IPsec und nicht für die Verschlüsselung der Pakete verwendet.

In keiner der beiden Varianten traten fehlerhafte Pakete auf. Allerdings wurden im Durchschnitt 10 Nicht-Test Pakete gemessen. Eine genauere Analyse mit Wireshark zeigte, dass es sich hierbei um ARP-Anfragen handelte. Bei Linux ist ein Eintrag im ARP-Cache standardmäßig für 15 bis 45 Sekunden gültig. Bei einer Testdauer von 5 Minuten sind damit auch im Durchschnitt 10 ARP-Anfragen zu erwarten, die Messungen zeigen hier also keine Besonderheiten.

Die verwendete CPU-Zeit ist, mit 20% bis 25%, bei beiden Varianten annähernd identisch. Die Auslastung der CPU ist in Abbildung 4.7 zu sehen. Dies lässt vermuten, dass die Paketverarbeitung hauptsächlich durch die Leistung der CPU limitiert ist. Wenn keine Limitierung durch die CPU vorliegen würde, würde ein Rückgang des erreichten Durchsatzes auch ein Rückgang der verwendeten CPU-Zeit bedeuten. Es wird allerdings in keiner der beiden Varianten eine CPU-Auslastung von 100% erreicht. Dies ist auf eine fehlende Parallelisierung bei der Verarbeitung der Pakete zurückzuführen und wird in Abschnitt 4.3 genauer erläutert.

4.2.2 Verschlüsselung mit AES-GCM

Anschließend wurden zwei weitere Tests durchgeführt, in denen die Pakete mit AES-GCM (einmal mit einer Schlüssellänge von 128 und einmal 256 Bit) verschlüsselt wurden. Der Durchsatz in Gbit/s ist zu sehen in Abbildung 4.4 und Tabelle 4.1. Der Durchsatz in Frames pro Sekunde wiederum findet sich in Abbildung 4.5 und Tabelle 4.2. Der Durchsatz (sowohl in Gbit/s als auch in Frames/s) sinkt im Vergleich zur NULL-Encryption noch einmal um bis zu 33% ab und es wird

ein maximaler Durchsatz von 3,7 Gbit/s erreicht. Gleichzeitig steigt der Round-Trip-Delay für jedes Paket um durchschnittlich 50%, auf bis $4787\mu\text{s}$. Der Durchsatz ohne die Nutzung von IPsec ist in etwa drei mal so groß, wie der Durchsatz mit einer Verschlüsselung mit AES-GCM. Die Verschlüsselung der Pakete benötigt also in etwa noch einmal ähnlich viel Zeit, wie die gesamte restliche Verarbeitung mit IPsec. Ähnlich wie bei den vorherigen Tests auch, bleibt die Anzahl der Frames pro Sekunde für beide Schlüsselgrößen annähernd konstant auf rund 380.000. Die Unterschiede zwischen den beiden Schlüsselgrößen sind generell sehr gering. Dies ist insbesondere interessant, da eine Verschlüsselung mit einem 256 Bit langen Schlüssel vier Verschlüsselungsrunden mehr vorschreibt. Diese zusätzlichen Verschlüsselungsrunden scheinen aber, anders als erwartet, keinen großen Einfluss auf die Performance des Gesamtsystems zu haben.

Zuletzt wurden zwei Tests durchgeführt, in denen das Hardware-Offloading für die Verschlüsselung aktiviert wurde. Der Verschlüsselungsalgorithmus selbst ist weiterhin AES-GCM mit einer Schlüssellängen von einmal 128 und einmal 256 Bit. Verglichen mit den beiden Tests ohne Offloading ist der Durchsatz hier stark erhöht. Es wird ein Durchsatz von bis zu 5,4 Gbit/s (bei einer Framegröße von 1400 Byte) erreicht. Dieser übersteigt damit sogar den obigen Durchsatz bei einer NULL-Encryption leicht. Dies trifft sogar auf alle getesteten Framegrößen bis 1400 Byte zu. Das ist ein überraschendes Ergebnis, kann jedoch dadurch erklärt werden, dass bei der Nutzung von Hardware-Offloading auch die Suche nach den benötigten Security Associations und die Kapselung der IP-Pakete in einem ESP-Paket auf der Netzwerkkarte durchgeführt werden. Beide Vorgänge sind somit ebenfalls schneller, als wenn sie durch eine Software-Implementierung, hier dem Linux-Kernel, durchgeführt werden würden. Zusätzlich ist der RTD (zu sehen in Abbildung 4.6), mit Werten von $1339\mu\text{s}$ bis $2586\mu\text{s}$, oft nur leicht höher als komplett ohne die Nutzung von IPsec.

Es ist also zu erkennen, dass Offloading einen hohen positiven Effekt hat und den erreichten Durchsatz deutlich erhöhen kann. Hierbei sind Steigungen von mindestens 22,5% (bei einer Framegröße von 68 Byte und einer Schlüsselgröße von 128 Bit), bis hin zu 50,1% (bei einer Framegröße von 1400 Byte und einer Schlüsselgröße von 256 Bit), möglich. Generell ist zu erkennen, dass der Durchsatz für verschiedene Framegrößen bei der Nutzung von Offloading stärker ansteigt, als ohne die Nutzung von Hardware-Offloading. Damit werden höhere Performancegewinne bei der Verschlüsselung größerer Framegrößen erreicht, als bei kleineren Framegrößen. Dies kann durch den prozentual geringeren Overhead bei großen Frames erklärt werden. Bei einem hohen Overhead pro Frame können weniger Daten verschickt werden und es müssen mehr Frames versendet werden, um den gleichen Durchsatz zu erreichen.

In keiner der Varianten wurden fehlerhafte Pakete gemessen und die Anzahl der Nicht-Test Pakete blieb unverändert auf durchschnittlich 10 Pakete.

Die benötigte CPU-Auslastung bleibt bei allen vier Varianten relativ konstant, auf Durchschnittlich 19%. Da aber bei den zwei Offloading-Varianten ein höherer Durchsatz erreicht wird, kann mit der gleichen CPU-Last auch deutlich mehr Pakete verarbeitet werden. Wenn sich die Anzahl der übertragenen Pakete bei der Aktivierung von Hardware-Offloading also nicht ändert, ist auch eine geringere CPU-Auslastung zu erwarten.

4.2.3 Besonderheiten bei IP-Fragmentierung

Wenn ein IP-Paket eine bestimmte Größe erreicht, sodass es nach der Kapselung in einem ESP und IP-Paket die MTU übersteigt, muss das IP-Paket vor dem Versenden noch fragmentiert werden. Dieser Effekt tritt in dieser Arbeit bei Tests mit einer Framegröße von 1518 Byte (also einer

IP-Paketgröße von 1500 Byte) auf. Die Fragmentierung benötigt zusätzliche Zeit für jedes Paket, weshalb der erreichte Durchsatz bei diesen Messungen deutlich geringer ausfällt, als normalerweise erwartet wird. Um den Effekt der Fragmentierung abzuschätzen, kann der Durchsatz zwischen den Framegrößen 1400 und 1518 Byte verglichen werden. Siehe dafür insbesondere Tabelle 4.1.

Zu sehen ist, dass bei einer Verschlüsselung mit AES-GCM (Schlüssellänge 128 Bit) der Durchsatz von 3,7 Gbit/s auf 2,8 Gbit/s abfällt. Dies entspricht einem relativen Verlust von rund 22%. Bei der Nutzung der NULL-Encryption ist der Verlust sogar noch größer: Der Durchsatz fällt von 5,3 Gbit/s auf 3,5 Gbit/s, was einem Abfall von 35% entspricht. Warum der Abfall bei der NULL-Encryption soviel größer ist, als bei einer Verschlüsselung mit AES-GCM, ist allerdings unklar.

Noch schlechtere Ergebnisse sind zu sehen, sobald Offloading aktiviert wird. Hier fällt der Durchsatz von 5,4 Gbit/s auf 2,8 Gbit/s. Der Durchsatz sinkt somit um bis zu 48%. Bei der Nutzung von Hardware-Offloading wird also ein sehr ähnlicher Durchsatz, wie bei einer Verschlüsselung ohne Hardware-Offloading, erreicht. Dies lässt vermuten, dass eine Verschlüsselung mit Fragmentierung nicht auf die Hardware ausgelagert werden kann und die gesamte Verarbeitung der Pakete auf die Software-Implementierung zurückfallen muss.

Um optimale Ergebnisse zu erreichen sollte also darauf geachtet werden, dass eine IP-Fragmentierung nicht nötig ist. Wenn TCP genutzt wird kann dies zum Beispiel erreicht werden, indem die Maximum Segment Size (MSS) entsprechend reduziert wird (zum Beispiel durch MSS Clamping). Dadurch sollten keine TCP-Pakete mehr versendet werden, die bei der Verschlüsselung zu einer Fragmentierung führen würden. Eine Alternative wäre es, die MTU in den beiden Subnetzen zu reduzieren. Dies würde alle IP-Pakete (und nicht nur TCP-Pakete) betreffen und zu große IP-Pakete würden entsprechend vor dem Versenden, also auch vor der Verschlüsselung, fragmentiert werden. Wenn dies auf viele IP-Pakete im Netzwerk zutrifft, würde dadurch aber die Anzahl der versendeten IP-Pakete im Netzwerk, aufgrund der vielen IP-Fragmente, stark ansteigen. Da der Durchsatz hauptsächlich durch die Anzahl und nicht die Größe der IP-Pakete limitiert ist, kann dies bei vielen IP-Paketen aber auch den Durchsatz negativ beeinflussen. Je nach genutzter Anwendung können eventuell auch noch weitere Anpassungen getroffen werden, um eine Fragmentierung der IP-Pakete zu verhindern.

4.2.4 Tests mit IMix

Zuletzt wurden noch Tests mit dem oben vorgestellten IMix durchgeführt. Die Gesamt-Senderate für diese Tests beträgt dabei 15,5 Gbit/s. Der verwendete IMix hat eine durchschnittliche Framegröße von 644,8 Bytes. Basierend auf den vorherigen Messergebnissen ist der damit zu erwartende Durchsatz, ohne die Nutzung von IPsec, rund 5 bis 6 Gbit/s. Mit Nutzung von IPsec ist ein entsprechend geringerer Durchsatz zu erwarten, der den obigen Verhältnissen folgt.

In den Messungen mit dem IMix zeigte sich allerdings ein deutlich geringerer Durchsatz, zu sehen in Abbildung 4.8 und Tabelle 4.3: Der Durchsatz ohne die Nutzung von IPsec ist gerade einmal 1,3 Gbit/s. Mit einer NULL-Encryption verringert sich der Durchsatz auf rund 0,45 Gbit/s. Interessanterweise ist der Unterschied zwischen einer NULL-Encryption und den anderen Verschlüsselungsverfahren nur sehr gering. AES-GCM ohne Offloading erreicht 0,36 Gbit/s und mit Offloading steigt der Durchsatz auf 0,42 Gbit/s an. Auch der Unterschied zwischen einer Verschlüsselung mit und ohne Offloading ist also sehr gering.

Um die Ursache dieses niedrigen Durchsatzes zu finden, wurde die Anzahl der empfangenen

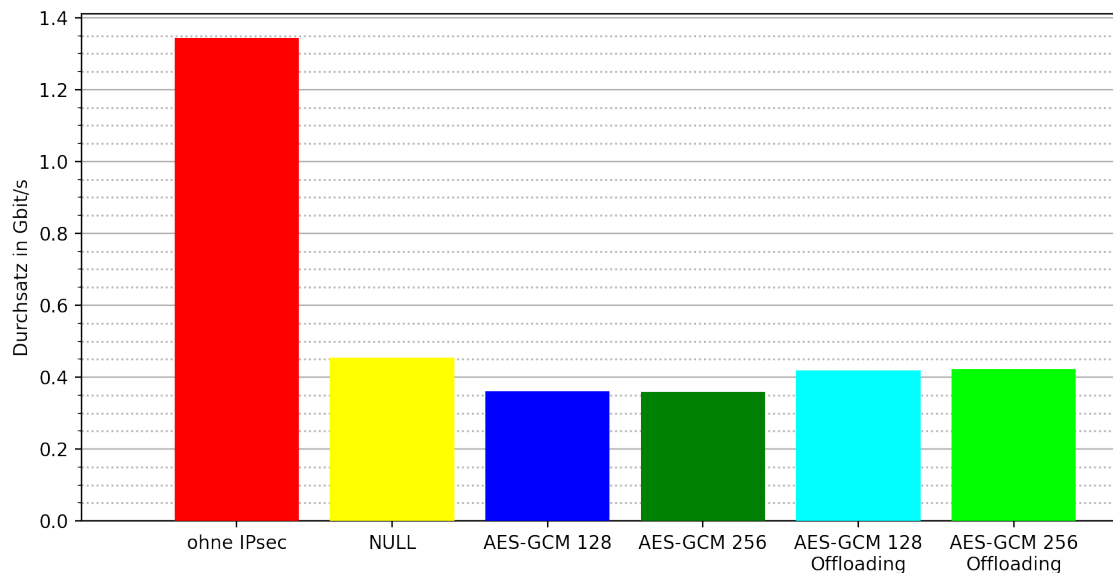


Abbildung 4.8: Vergleich des erzielten Durchsatzes (in Gbit/s) mit dem IMix, für verschiedene Testversionen.

Frame- größe	ohne IPsec	NULL	AES-GCM 128	AES-GCM 256	AES-GCM 128 Offloading	AES-GCM 256 Offloading
68	0.030	0.013	0.006	0.006	0.013	0.013
100	0.398	0.176	0.089	0.089	0.178	0.180
373	0.152	0.067	0.034	0.034	0.068	0.068
570	0.134	0.059	0.030	0.030	0.040	0.040
1300	0.170	0.075	0.038	0.037	0.076	0.077
1518	0.460	0.064	0.163	0.163	0.044	0.044
Gesamt	1.343	0.455	0.360	0.359	0.419	0.422

Tabelle 4.3: Vergleich des erzielten Durchsatzes (in Gbit/s) mit dem IMix, für verschiedene Testversionen. Es ist einmal der Gesamtdurchsatz zu sehen und einmal aufgeteilt nach den entsprechenden Framegrößen. Die Framegrößen sind in Byte angegeben. Alle Werte sind auf drei Nachkommastellen gerundet.

Frame- größe	Versendet	ohne IPsec	NULL	AES-GCM 128	AES-GCM 256	AES-GCM 128 Offloading	AES-GCM 256 Offloading
68	4.839	11.757	12.060	11.528	11.525	12.230	12.232
100	41.935	74.820	76.711	73.244	73.223	77.787	77.799
373	9.677	5.289	5.423	5.181	5.179	5.498	5.499
570	8.065	2.939	3.014	2.887	2.886	2.036	2.033
1300	9.677	1.574	1.611	1.536	1.535	1.636	1.637
1518	25.806	3.621	1.181	5.624	5.653	0.812	0.799

Tabelle 4.4: Vergleich der übertragenen Frames mit dem IMix, für verschiedene Testversionen. Angegeben ist jeweils die relative Häufigkeit der Framegröße (in Prozent). Es sind einmal die Häufigkeit der Versendeten Frames und für jeden Test die Häufigkeit der empfangenen Frames zu sehen. Die Framegrößen sind in Byte angegeben. Alle Werte sind auf drei Nachkommastellen gerundet.

Frames jeder Framegröße mit der Anzahl der gesendeten Frames verglichen. Zu sehen ist dies in Tabelle 4.4. Dabei ist deutlich zu erkennen, dass kleinere Frames bei der Übertragung deutlich bevorzugt werden. Besonders deutlich ist dies bei den beiden kleinsten Framegrößen 68 und 100 Byte zu erkennen: Obwohl insgesamt nur 46,7% der versendeten Frames diese Framegrößen hatten, waren mindestens 84,7% aller empfangenen Frames von einer dieser beiden Framegrößen. Damit ändert sich auch der Durchschnitt der Größe übertragenen Frames und der neue Durchschnitt variiert je nach Test zwischen 151,6 und 194,7 Bytes. Der zu erwartende durchschnittliche Durchsatz bei diesen Framegrößen ist deutlich geringer und stimmt mit den Messergebnissen überein. Warum das Testsystem kleinere Framegrößen bei der Übertragung bevorzugt konnte allerdings nicht geklärt werden.

Wenn die Verteilung der Frames für die verschiedenen Testvarianten verglichen werden fällt außerdem auf, dass bei einer Verschlüsselung ohne Offloading deutlich mehr Frames mit einer Größe von 1518 Bytes übertragen werden, als bei einer Verschlüsselung mit Offloading oder mit NULL-Encryption. (Bei einer NULL-Encryption sind dies 1,2% der Pakete, bei AES-GCM ohne Offloading 5,6% und bei AES-GCM mit Offloading 0,8%.) Dies lässt sich dadurch erklären, dass bei einer NULL-Encryption und einer Verschlüsselung mit Offloading der Durchsatz bei einer Framegröße von 1518 Bytes (aufgrund der Fragmentierung) deutlich stärker reduziert ist, als bei einer Verschlüsselung ohne Offloading (siehe auch Teilabschnitt 4.2.3: 35% für NULL-Encryption, 22% für AES-GCM ohne Offloading und 48% für AES-GCM mit Offloading). Verglichen mit den anderen Framegrößen ist der Performanceverlust bei AES-GCM ohne Offloading also deutlich geringer als bei den anderen Versionen. Aus diesem Grund werden bei den anderen beiden Versionen deutlich weniger Frames mit einer Größe von 1518 Byte versendet, als bei einer Verschlüsselung ohne Offloading.

4.3 Parallelisierung

Zuletzt wurde untersucht, ob eine Verbesserung des Durchsatzes durch eine Parallelisierung erreicht werden kann. Dafür werden mehrere Datenströme gleichzeitig übertragen und der Gesamtdurchsatz sowie die Gesamt-CPU-Auslastung gemessen. Jeder Datenstrom nutzt dabei eine andere IP-Adresse zwischen 10.0.0.21 und 10.0.0.29 (mit Ausnahme von 10.0.0.22) als Absender-

Adresse. Die Empfänger-Adresse ist jedoch immer 10.0.0.22, da der Loopback-Port des Testgerätes nicht mehr als eine IP-Adresse annehmen kann. Die Anzahl der parallelen Datenströme ist dabei 1, 2, 4 oder 8. Als Verschlüsselungsalgorithmus wird dabei AES-GCM mit einer Schlüssellänge von 128 Bit verwendet und es wird je einmal mit und ohne Unterstützung durch Hardware-Offloading getestet. Außerdem wird je einmal mit einer Framegröße von 68 Byte und einmal mit einer Framegröße von 1400 Byte getestet. Die Senderaten wurden gewählt, sodass eine Überlastung durch zu viele Frames zu vermeiden wird: Bei einer Framegröße von 68 Byte wurde eine Senderate von 0,25 Gbit/s gewählt und bei einer Framegröße von 1400 Byte wurde eine Senderate von 3 Gbit/s pro Datenstrom gewählt. (Siehe auch Abschnitt 4.1.)

Der Test wird in drei verschiedenen Varianten durchgeführt:

Variante 1 Es wird ein IPsec-Tunnel aufgebaut und alle Datenströme laufen über diesen Tunnel.

Variante 2 Es werden mehrere IPsec-Tunnel aufgebaut und über jeden Tunnel läuft genau ein Datenstrom.

Variante 3 Wie Variante 2, zusätzlich ist aber noch IRQ-Balancing (Interrupt Request Balancing) aktiviert.

Im Normalfall ist IRQ-Balancing deaktiviert und die verschiedenen Interrupt-Queues gleichmäßig auf alle Cores aufgeteilt, um eine möglichst gleichmäßige Lastenverteilung zu ermöglichen und so einen besseren Durchsatz zu erreichen.³ In diesen Tests wurden die Interrupts mithilfe des von Mellanox mitgelieferten `set_irq_affinity.sh` Skriptes aufgeteilt, wodurch jeder Core für genau eine Interrupt-Queue zuständig ist. In Linux wird das IRQ-Balancing durch den `irq_balancer` Service übernommen. Wenn dieser aktiviert ist, wird die Aufteilung der Interrupt-Queues durch diesen Service übernommen, weshalb die Lastenverteilung nicht mehr unbedingt gleichmäßig ist. Um zu testen, ob dies einen Effekt auf den Durchsatz hat, wurde für Variante 3 IRQ-Balancing aktiviert.

Der Gesamtdurchsatz ist in Abbildung 4.9 zu sehen, in Tabelle B.2 ist eine tabellarische Darstellung der Daten. Zu erkennen ist, dass der Durchsatz (bis auf wenige Ausnahmen) bei mehr Datenströmen auch ansteigt, oder zumindest gleich bleibt.

Bei Variante 1 ist nach 4 parallelen Datenströmen allerdings kein Performancegewinn mehr feststellbar. Wenn Offloading deaktiviert ist, sinkt der Gesamtdurchsatz bei 8 parallelen Datenströmen sogar wieder ab.

Für Variante 2 ist durch eine Parallelisierung ein deutlicher Performancegewinn messbar, der sogar Variante 1 übersteigt. Es scheint also, dass durch die Aufteilung der Kommunikation auf mehrere IPsec-Tunnel, der Durchsatz erhöht werden kann. Bereits ohne die Nutzung von Offloading und für Framegrößen von 68 Byte, steigt der Durchsatz von rund 73 Mbit/s (für einen Datenstrom) auf bis zu 200 Mbit/s (für 8 Datenströme) an, was einer Steigerung von rund 270% entspricht. Mit der Nutzung von Offloading werden sogar noch bessere Werte erreicht und der Durchsatz steigt von 85 Mbit/s auf 295 Mbit/s an, was einer Steigerung von fast 345% entspricht. Bei Framegrößen von 1400 Byte ist ebenfalls ein deutlicher Anstieg messbar, hier allerdings nur eine Verdopplung des Durchsatzes. Insgesamt hat die Parallelisierung damit aber nur eine relativ

³Für weitere Informationen zur Verteilung der Interrupt-Queues siehe auch: <https://support.mellanox.com/s/article/what-is-irq-affinity-x>

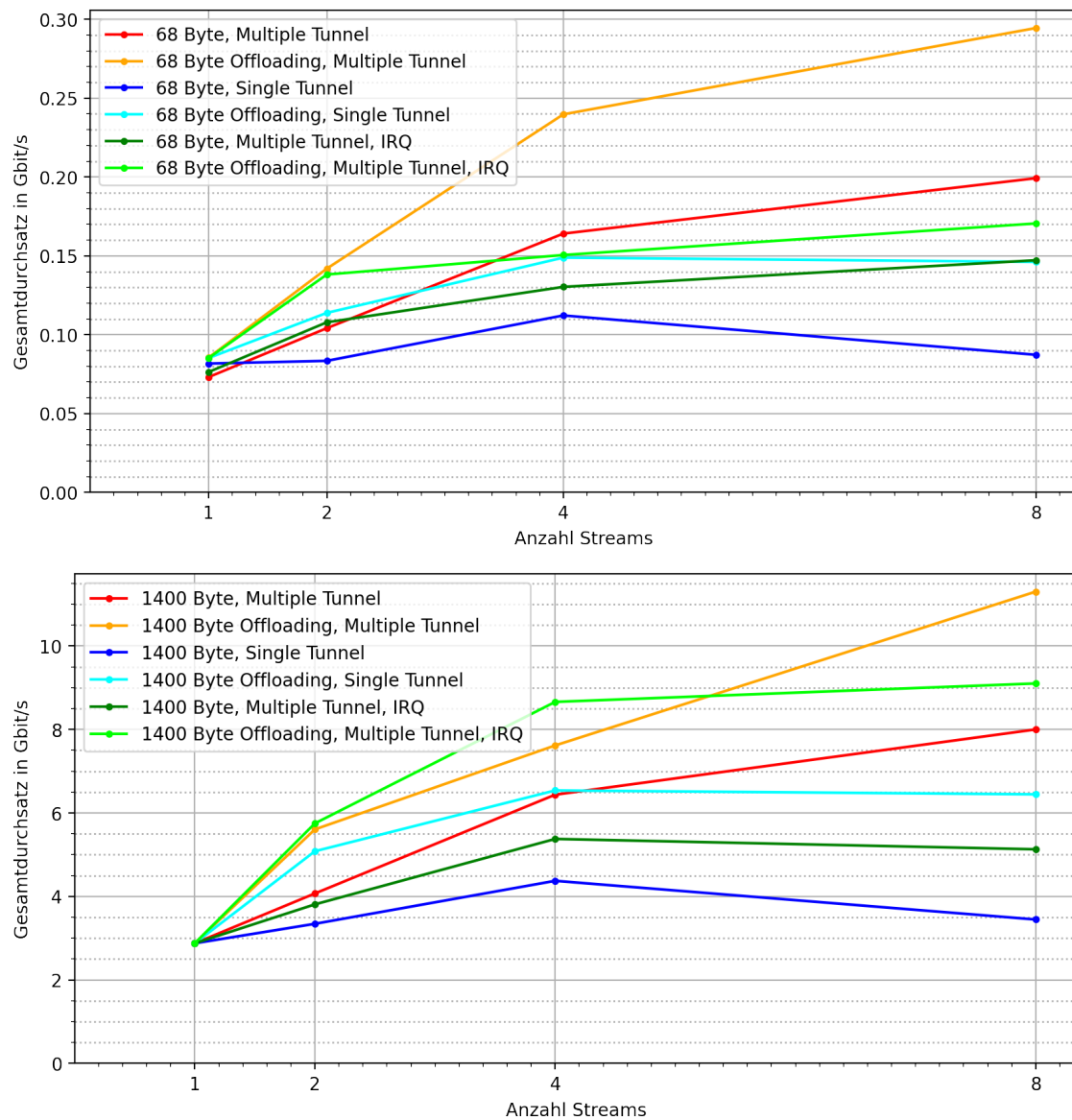


Abbildung 4.9: Der Gesamtdurchsatz (in Gbit/s) bei parallelen Datenströmen, je einmal für eine Framengröße von 68 Byte (oben) und 1400 Byte (unten).

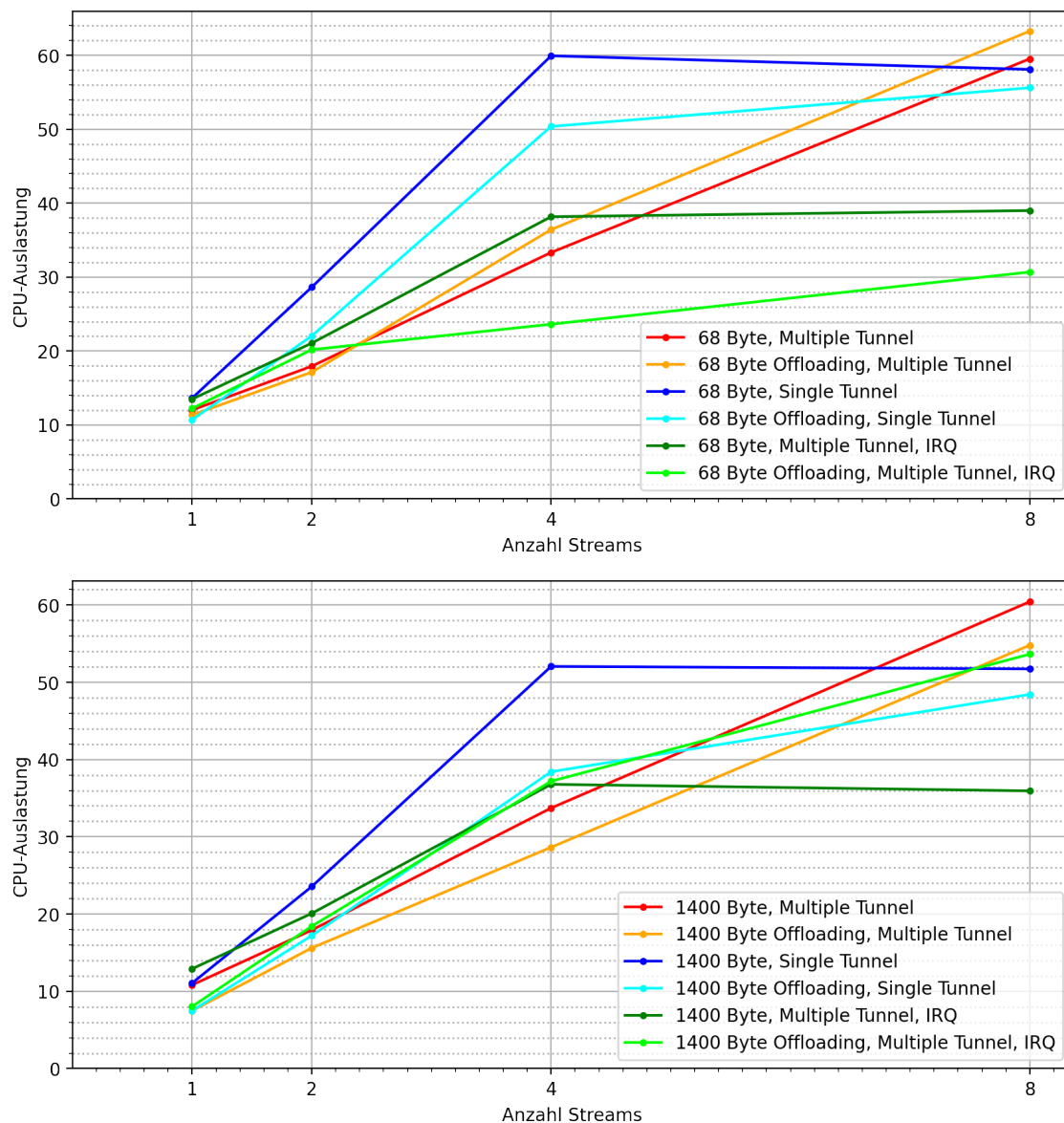


Abbildung 4.10: Die gesamte benötigte CPU-Auslastung während der Übertragung bei parallelen Datenströmen, je einmal für eine Framegröße von 68 Byte (oben) und 1400 Byte (unten). Gemessen wurde die gesamte Zeit in der die CPU gerechnet hat, über alle Cores hinweg. (Einschließlich der benötigten Zeit für weitere Prozesse, die nichts mit der Übertragung von Paketen zu tun haben.) 100% würde entsprechend eine volle Auslastung aller Cores sein und 12,5% wäre eine Volllastung eines Cores.

geringe Effizienz⁴ und bei einer Nutzung von 8 Datenströmen und einer Framegröße von 1400 Byte ist gerade einmal eine Effizienz von $2/8 = 25\%$ erreichbar. Auch mit Hardware-Offloading ist bei 1400 Byte großen Frames keine Steigerung der Effizienz möglich. Bei einer Framegröße von 68 Byte ist die Effizienz mit 34%, bzw. 43% mit Offloading, nur geringfügig höher. Außerdem ist die Effizienz einer geringeren Anzahl von Datenströmen ebenfalls höher und bei nur zwei Datenströmen wird mit einer Framegröße von 68 Byte und Offloading sogar eine Effizienz von 83% erreicht.

In einigen Fällen werden in Variante 3, mit aktiviertem IRQ-Balancing, ähnliche Ergebnisse erzielt wie in Variante 2, ohne die Nutzung von IRQ-Balancing. Insbesondere bei einer kleinen Anzahl von Datenströmen (ein oder zwei Datenströme) sind die Unterschiede meist nur sehr gering. In nur einem Fall (Framegröße 1400 Byte, 4 Datenströme, Offloading aktiviert) ist der Durchsatz mit IRQ-Balancing deutlich besser als ohne: Mit IRQ-Balancing werden bis zu 8,66 Gbit/s erreicht, ohne IRQ-Balancing allerdings nur 7,6 Gbit/s. Wenn 8 verschiedene Datenströme genutzt werden, verschlechtert das IRQ-Balancing aber den Durchsatz in jedem Fall deutlich. Da zwischen den Subnetzen häufig mehrere Geräte gleichzeitig miteinander kommunizieren, sollte IRQ-Balancing für eine optimale Performance deaktiviert werden.

Selbst ohne IRQ-Balancing waren die einzelnen Datenströme nicht gleichmäßig auf alle Kerne verteilt, was teils zu unterschiedlichen Durchsätzen bei den einzelnen Datenströmen geführt hat. Der Grund für diese Ungleichmäßigkeit liegt in Hash-Konflikten bei der Verteilung der IP-Pakete: Da kein Load-Balancing vorgenommen wird, sondern IP-Pakete einfach anhand ihrer IP-Adressen, mittels einer Hash-Table, zugeordnet werden⁵, kann es leicht zu Hash-Konflikten kommen. Diese Hash-Konflikte sorgen dafür, dass selbst Pakete mit unterschiedlichen Adressen dem gleichen Core zugewiesen werden, was die Parallelisierbarkeit der Übertragung beeinflusst. In der Praxis lassen sich solche Hash-Konflikte aber meist nicht vermeiden und eine bessere Parallelisierung kann nur durch mehr Kerne erreicht werden. Es ist denkbar, dass noch bessere Ergebnisse erzielt werden können, wenn mehr CPU-Kerne zur Verfügung stehen und die Last somit auch auf mehr Kerne verteilt werden kann.

Es ist außerdem anzumerken, dass der Durchsatz ohne IRQ-Balancing deutlich stabiler war, als mit IRQ-Balancing. Wenn IRQ-Balancing aktiviert wurde, schwankte der Durchsatz während der Messungen stark und es kam öfters vor, dass einige Datenströme kurzzeitig überhaupt keine Pakete übertragen konnten. Je mehr Datenströme es gab, umso stärker wurden die Schwankungen. Allerdings ist auch anzumerken, dass bei wenigen Datenströmen teils auch Schwankungen nach oben stattfanden, der Durchsatz also kurzzeitig deutlich erhöht ist. Aus diesem Grund ist der durchschnittliche Durchsatz mit und ohne IRQ-Balancing hier auch sehr ähnlich. Bei 8 Datenströmen ist dies aber nicht mehr der Fall und die Schwankungen verschlechtern den Durchsatz nur. Dies ist auch ein Grund für den schlechteren Durchsatz bei aktiviertem IRQ-Balancing bei 8 Datenströmen. Ohne IRQ-Balancing wurden keine Schwankungen festgestellt und alle Datenströme hatten einen annähernd konstanten Durchsatz. Dies ist ein weiterer Grund gegen die Nutzung von IRQ-Balancing.

Die benötigte CPU-Last ist in Abbildung 4.10 zu sehen. Hier ist zu erkennen, dass bis zu 4 Datenströmen die CPU-Last ebenfalls ansteigt. Variante 1 hat hier den größten Anstieg und hat

⁴Die Effizienz berechnet sich aus dem relativen Durchsatz (Verglichen mit dem Durchsatz bei nur einem Datenstrom) dividiert durch die Anzahl der Datenströme. Sei D_i der Durchsatz bei i Datenströmen und n die Anzahl der Datenströme. Dann ist die Effizienz definiert als: $E(n) = (D_n/D_1)/n$.

⁵Für mehr Informationen siehe auch: <https://support.mellanox.com/s/article/MLNX2-117-2982kn>

teils eine deutlich höhere CPU-Auslastung als andere Varianten. Ab vier Datenströmen ist der Anstieg bei Variante 1 und 3 allerdings deutlich gehemmt. Variante 2 hat dagegen wiederum einen gleichbleibenden, linearen Anstieg.

Bei 8 Datenströmen ist die CPU-Last bei Variante 2 am höchsten. Der Unterschied zwischen Variante 1 und 2 ist aber generell sehr gering, obwohl Variante 2 einen deutlich höheren Durchsatz erzielt. Es ist also zu erkennen, dass die Nutzung von mehreren IPsec-Tunneln nicht nur einen höheren Durchsatz erzielt, sondern auch die CPU deutlich effektiver nutzt. Bei Variante 3 ist die CPU-Auslastung aber teils deutlich geringer als bei Variante 2, insbesondere wenn Offloading deaktiviert ist. IRQ-Balancing kann also dafür genutzt werden, etwas CPU-Auslastung einzusparen.

5 Aufgetretene Probleme

Während der Bearbeitung dieser Arbeit traten diverse Probleme auf, die Anpassungen am Messplan und der Testumgebung notwendig machten. Die meisten der aufgetretenen Probleme hängen mit der Netzwerkkarte und dessen Treiber zusammen. In diesem Kapitel werden die aufgetretenen Probleme und, sofern bekannt, deren Ursachen dargestellt. Außerdem werden die Auswirkungen der Probleme und deren Einfluss auf die Messungen erläutert. Es wird außerdem erklärt, wie mit den Problemen umgegangen wurde und wie diese gelöst wurden.

5.1 Inkompatibilitäten mit alter Hardware

Der ursprüngliche Messplan sah vor, dass die Mellanox-Karten in einen bereits vorhandenen Entwicklungsserver eingebaut werden und dieser Server dann für die Messungen verwendet wird. Dies führte allerdings zu Problemen, da der Server nach dem Einbau der Karten nicht mehr erfolgreich booten konnte. Das Problem trat bereits beim Start des BIOS auf. Dieses schien bei der Initialisierung der PCIe-Geräte in einer Endlosschleife festzustecken und reagierte dadurch nicht mehr auf Nutzereingaben. Die Ursache dieses Problems lag hierbei in inkompatiblen PCIe Versionen. Die Mellanox ConnectX-6 Dx Karte ist nur mit PCIe 3.0 und 4.0 kompatibel, der verwendete Server verfügte allerdings nur über PCIe 1.0 Anschlüsse.

Als Alternative für die Fortführung der Arbeit wurde sich für die Nutzung der x64-PCs Dell OptiPlex 7050 und 7070 entschieden. Beide verfügen über PCIe 3.0 Anschlüsse und sind damit mit der Karte kompatibel. Die Netzwerkkarte funktionierte mit dem OptiPlex 7070 auch einwandfrei. Bei dem OptiPlex 7050 konnte das BIOS nach dem Einbau der Karte aber nicht mehr korrekt starten. Um Fehler in der vorhandenen Karte ausschließen zu können, wurden beide verfügbaren Netzwerkkarten mit beiden PCs getestet. Dies änderte allerdings nichts am Problem. Warum das Problem auch bei einem OptiPlex 7050 auftritt, nicht aber bei einem OptiPlex 7070, konnte im Verlauf der Arbeit nicht geklärt werden. Ob das Problem nur bei dem vorhandenen Computer oder generell bei OptiPlex 7050 PCs auftritt konnte ebenfalls nicht geklärt werden.

Um dieses Problem zu lösen wurde sich stattdessen für die Nutzung eines OptiPlex 7090 entschieden. Bei diesem funktionierte die Netzwerkkarte ohne den Boot-Prozess zu blockieren.

5.2 Unterschiedliche Treiber Versionen

Mellanox bietet zwei verschiedene Treiber für die Netzwerkkarten an: `MLNX_EN` und `MLNX_OFED`. Der `MLNX_EN` Treiber enthält dabei nur die Funktionalitäten für die Ethernet-Karten, wohingegen `MLNX_OFED` sowohl für die Ethernet als auch für die InfiniBand-Karten verwendet werden kann. Für die Messungen in dieser Arbeit wurden allerdings nur Ethernet-Karten verwendet. Aus diesem Grund wurde anfangs auch nur der entsprechende Ethernet-Treiber installiert. Bei diesem konnte

die Offloading Funktionalität allerdings nicht aktiviert werden, obwohl der Treiber laut Dokumentation diese ebenfalls enthalten müsste.¹

Deswegen wurde anschließend der `MLNX_OFED` Treiber installiert. Bei diesem konnte das Hardware-Offloading ohne weitere Probleme aktiviert werden. Warum die Funktionalität nur bei diesem Treiber verfügbar war, konnte allerdings im Verlauf der Arbeit nicht ermittelt werden.

5.3 Überhitzung der Karten

Es wurde schnell festgestellt, dass die Netzwerkkarten nach ungefähr 30 bis 60 Minuten nach dem Start des PC ausfallen. Dies geschieht auch dann, wenn die Karten nicht genutzt wurden, also im Ruhebetrieb sind. Das wiederum erschwert Messungen der Performance. Wenn eine der Karten ausgefallen ist, konnte sie nicht mehr angesprochen werden und Operationen, welche die Karte involvieren (wie zum Beispiel Selbsttests), scheiterten mit unterschiedlichen Fehlercodes. Außerdem konnte sie keine Nachrichten mehr versenden oder empfangen. Die Netzwerkinterfaces waren aber immer noch vorhanden und der Treiber blieb ebenfalls aktiv. Die Status-LEDs an der Karte fielen allerdings aus, was signalisiert, dass die Karte nicht mehr aktiv ist.

Um die Ursache dieses Problems zu finden wurde der Befehl `dmesg` genutzt, um die Nachrichten der Treiber mitzulesen. Hierbei zeigte sich, dass beim Ausfall einer Karte der Treiber einen unbekannten, kritischen Fehler meldet. Die Ursache dieses Fehlers wird allerdings nicht mit angegeben. In den Ausgaben der Treiber finden sich aber auch Warnungen aufgrund von hohen Temperaturen der Netzwerkkarte, welche nach ungefähr 15 bis 45 Minuten nach dem Start des Computers generiert wurden. Daraus lässt sich schließen, dass der Treiber die Netzwerkkarte aufgrund von stark erhöhter Temperatur ausgeschaltet hat, um eine Überhitzung und Beschädigung der Netzwerkkarte zu vermeiden.

Um dieses Problem zu lösen, wurde jeweils ein zusätzlicher Lüfter bereitgestellt, um die Karten zusätzlich zu kühlen. Dadurch trat keine Überhitzung mehr auf und die Karten fielen nicht mehr aus.

5.4 Nicht standardmäßige MTU

Es wurde auch versucht, Messungen mit Jumbo-Frames (Frames größer als 1518 Byte) durchzuführen. Dafür wurde die MTU auf allen Interfaces von 1500 auf 9100 erhöht, um Jumbo-Frames bis zu einer Größe von 9000 Byte testen zu können. Durch die Erhöhung der MTU sind aber Performanceverluste aufgetreten. Genauer wurden viele der Frames bei der Übertragung verworfen, was den Durchsatz stark reduziert hat. Selbst Frames, die weit unter der MTU lagen und kleiner waren als 1500 Byte, waren davon betroffen. Die erfolgten Messungen mit einer MTU von 9100 zeigen einen zusätzlichen Verlust beim Durchsatz von 45% bis 52%, unabhängig von der Framegröße. Das Problem hängt also nicht mit der verwendeten Framegröße, sondern mit der genutzten MTU zusammen.

Die genaue Ursache dieses Problems konnte im Verlauf der Arbeit nicht geklärt werden. Allerdings ist auch dies ein bereits bekanntes Problem bei Mellanox-Karten.²

¹ siehe auch: <https://docs.nvidia.com/networking/display/MLNXENV571020/IPsec+Crypto+Offload>

² siehe auch: <https://docs.nvidia.com/networking/display/MLNXOFEDv561033/Known+Issues>,
Internal Ref. Number: 2780436

Die Ergebnisse dieser Messungen sind entsprechend nicht vergleichbar mit den anderen Ergebnissen dieser Arbeit. Aus diesem Grund wurde sich dafür entschieden, keine weiteren Messungen mit Jumbo-Frames oder nicht-standardmäßigen MTUs durchzuführen.

5.5 Loopback-Port mit IPv6-Adressen

Ursprünglich sollte es ein Messexperiment geben, welches die Performance von Hardware-Offloading mit IPv4 und IPv6-Paketen vergleicht. Dem Loopback-Port des verwendeten Testgerätes kann allerdings keine IPv6-Adresse zugewiesen werden. Es können also auch keine IPv6-Pakete direkt an den Loopback-Port adressiert werden.

Dieses Problem kann umgangen werden, zum Beispiel durch die Nutzung von Network address translation (kurz: NAT), wodurch IPv6-Adressen auf entsprechende IPv4-Adressen gemappt werden. Dadurch würden die IPv6-Pakete, wenn sie bei einem der beiden Computer ankommen, in entsprechende IPv4-Pakete umgewandelt werden. Diese IPv4-Pakete würden anschließend normal an den Loopback-Port gesendet werden. Wenn die IPv4-Pakete wieder beim Computer ankommen, würden sie zurück in IPv6-Pakete umgewandelt werden.

Der Nachteil bei solchen Lösungen ist aber, dass die sich daraus ergebenden Messergebnisse nicht mehr mit den anderen Ergebnissen dieser Arbeit vergleichbar wären, da jedes Paket noch einmal extra umgewandelt werden müsste. Aus diesem Grund wurde in dieser Arbeit auf Messungen mit IPv6 verzichtet.

6 Fazit und Ausblick

In diesem Kapitel werden die Ergebnisse der Messungen zusammengefasst und ein abschließendes Fazit zur anfänglichen Problemstellung gegeben. Es werden außerdem mögliche Verbesserungsmöglichkeiten, welche die Performance der Messungen verbessern könnten, erläutert und ein Ausblick auf Fragestellungen möglicher nachfolgender Arbeiten gegeben.

Es zeigt sich, dass bei keinem der Tests die maximalen Übertragungsraten von 100 Gbit/s erreicht werden konnten. Selbst ohne die Nutzung von IPsec waren nur Übertragungsraten von 12,7 Gbit/s möglich. Die Messergebnisse legen nahe, dass die Übertragungsraten beim verwendeten Testsystem hauptsächlich durch die Verarbeitungsgeschwindigkeit der CPU und die fehlende Parallelisierung dieser limitiert wird. Der Nutzen einer Netzwerkkarte mit maximalen Durchsätzen von 100 Gbit/s scheint beim verwendeten Testsystem nicht sinnvoll. Generell ist eine 100 Gbit/s Netzwerkkarte nur zu empfehlen, wenn Netzwerkkarten mit einem geringeren maximalen Durchsatz sich als unzureichend und bereits maximal ausgelastet gezeigt haben. Andernfalls sind Flaschenhälse eher bei anderen Komponenten des Systems zu suchen.

Durch die Nutzung von IPsec sinkt der Durchsatz, unabhängig von der Framegröße, bereits um 55%, selbst wenn keine Verschlüsselung oder Authentifikation der Pakete vorgenommen wird. Wird zusätzlich eine Verschlüsselung mit AES-GCM vorgenommen, sinkt der Durchsatz erneut um rund ein Drittel. Die größten Verluste bei der Performance der Übertragung sind also nicht durch die Verschlüsselung, sondern durch die sonstige Verarbeitung der Pakete durch IPsec festzustellen. Dazu gehören unter anderem das Matching der Pakete gegen die Security Policies und die Kapselung in einem ESP-Paket.

Für Systeme die bereits IPsec nutzen, lässt sich Hardware-Offloading sehr einfach und ohne weitere Komplikationen aktivieren. Durch Hardware-Offloading steigt der erreichte Durchsatz um 23% bis 50%, je nach Paketgröße. Größere Pakete erzielen dabei eine stärkere Verbesserung des Durchsatzes. Damit sind die Ergebnisse bei der Verschlüsselung mit Hardware-Offloading sogar besser als die Ergebnisse ohne eine Verschlüsselung der Pakete.

Bei den Messungen wurden allerdings diverse Situationen identifiziert, die den mit Hardware-Offloading erreichbaren Durchsatz reduzieren:

- Kleine Pakete erzielen generell einen geringen Durchsatz. Da Hardware-Offloading bei kleinen Paketen allerdings deutlich geringere Gewinne erzielt als bei größeren Paketen, ist auch mit Hardware-Offloading bei kleinen Paketen nur ein geringerer Durchsatz erreichbar.
- Bei alten Systemen (vor Linux Kernel v5.8) kommt es, aufgrund eines Fehlers im Netzwerkstack, bei einer hohen Auslastung durch zu viele Frames zu Paketverlusten, welche den Durchsatz reduzieren. Dies führt insbesondere bei einer Nutzung von kleinen Frames zu einem deutlichen Einbruch beim Durchsatz.
- Messungen mit einem IMix suggerieren, dass bei Übertragungen kleinere Pakete überproportional bevorzugt werden. Sollte die Anzahl der übertragenen Pakete zu groß sein, würde

durch die überproportionale Behandlung von kleinen Paketen der Durchsatz zusätzlich reduziert werden.

- Bei zu großen Paketen ist nach der Kapselung in einem ESP-Paket eine IP-Fragmentierung nötig. Diese kann allerdings nicht durch Hardware-Offloading übernommen werden, weshalb die gesamte Verarbeitung des Paketes, inklusive der Verschlüsselung und Kapselung, von der CPU übernommen werden muss. Hardware-Offloading kann in solchen Fällen entsprechend nicht genutzt werden, wodurch auch keine Verbesserung des Durchsatzes erzielt wird.

Insgesamt zeigen sich somit Probleme, wenn viele kleine Pakete oder viele große Pakete versendet werden müssen. Ob und wie sehr die Nutzung von Hardware-Offloading den Durchsatz verbessert hängt somit stark von der Größe der Pakete ab, die auf dem System übertragen werden. Insbesondere durch die Behebung der letzten beiden Punkte könnte der durchschnittliche Durchsatz deutlich erhöht werden. Ob dies möglich ist und welche Performancegewinne dadurch zu erwarten sind, muss allerdings noch geklärt werden.

Tests bei denen mehrere Endgeräte simuliert wurden, zeigen einen deutlich höheren Gesamtdurchsatz, als bei nur einem Endgerät. Bei der Simulation von 8 Endgeräten wurde ein maximaler Durchsatz von bis zu 11 Gbit/s festgestellt. Wie hoch der Durchsatz ist hängt stark davon ab, wie viele Kerne zur Verarbeitung der Pakete zur Verfügung stehen. Die Effizienz bei der Nutzung mehrerer Kerne sinkt allerdings mit steigender Kernanzahl. Noch ungeklärt ist, ob sich die Effizienz durch stärkere CPUs mit einer höheren Taktfrequenz oder den Einsatz mehrerer CPUs parallel verbessern ließe. Auch wurden bessere Ergebnisse erzielt, wenn die Pakete auf mehrere IPsec-Tunnel aufgeteilt wurden. In den Messungen dieser Arbeit gab es für jeden Tunnel genau einen Datenstrom. Da dies in der Praxis aber nur selten der Fall ist, sollte zudem erforscht werden, wie sich der Durchsatz verhält, wenn es mehrere Datenströme pro Tunnel gibt und diese nicht gleichmäßig aufgeteilt sind. Auch führt eine hohe Tunnelanzahl zu einer erhöhten Komplexität des Gesamtsystems. Es wäre somit sinnvoll, nach Möglichkeiten zu suchen, die Tunnelanzahl zu reduzieren, ohne den Durchsatz allzu stark negativ zu beeinflussen.

Positiv anzumerken ist, dass durch die Aktivierung von Hardware-Offloading der Durchsatz in keinem Fall schlechter wurde. Durch die Nutzung von Hardware-Offloading sind also keine Performanceverluste zu erwarten. (Mit Ausnahme der Nutzung von nicht standardmäßigen MTUs, wie in Abschnitt 5.4 erläutert.)

Ein Thema, welches in dieser Arbeit nicht behandelt wurde, ist die Nutzung von Hardware-Offloading mit alternativen Netzwerk-Frameworks wie dem Data Plane Development Kit (DPDK). Die Entwicklung der dafür benötigten Treiber und deren Evaluierung ist ebenfalls ein Thema für spätere Arbeiten.

A StrongSwan Konfiguration

In diesem Kapitel ist die verwendete `swanctl.conf` Datei auf dem OptiPlex 7070 zu sehen. Auf dem OptiPlex 7090 sind die `local` und `remote` Adressen getauscht. Je nach durchgeführtem Test werden andere `esp_proposals` genutzt und `hw_offload` aktiviert oder deaktiviert. Bei Tests mit mehreren Streams werden weitere Child-SA konfiguriert, ähnlich zu `child1`.

```
1 connections {
2     conn1 {
3         version = 2
4         local_addrs = 10.0.0.21
5         remote_addrs = 10.0.0.22
6
7         local {
8             auth = psk
9             id = psk1
10        }
11
12        remote {
13            auth = psk
14            id = psk1
15        }
16
17        children {
18            child1 {
19                local_ts = 10.0.1.0/24
20                remote_ts = 10.0.2.0/24
21                start_action = none
22                close_action = none
23                rekey_time = 0s
24
25                esp_proposals = null-sha512-modp4096
26                # esp_proposals = aes128gcm128-sha512-modp4096
27                # esp_proposals = aes256gcm128-sha512-modp4096
28
29                # hw_offload = yes
30            }
31        }
32
33        proposals=aes256gcm128-sha512-modp4096
34    }
```

```
35 }
36
37 secrets {
38     psk1 {
39         secret=0
39             x338c21f24b7623ec8f109272c08d8d665cadffef73cba496d5d10542f51f7829
40     }
41 }
```

B Weitere Messergebnisse

In diesem Kapitel sind tabellarische Auflistungen weiterer Messergebnisse zu finden.

Sende- rate	68 Byte	68 Byte Offloading	128 Byte	128 Byte Offloading	68 Byte ohne IPsec
1	0.088	0.123	0.272	0.373	0.258
5	0.088	0.123	0.272	0.372	0.256
10	0.088	0.123	0.272	0.371	0.257
20	0.088	0.123	0.273	0.371	0.256
30	0.088	0.122	0.272	0.370	0.257
40	0.070	0.097	0.272	0.371	0.257
50	0.057	0.080	0.272	0.371	0.257
60	0.045	0.062	0.241	0.329	0.256
70	0.037	0.052	0.207	0.282	0.255
80	0.032	0.045	0.177	0.241	0.256
90	0.035	0.046	0.157	0.211	0.249
100	0.026	0.036	0.139	0.193	0.251
Sende- rate	256 Byte	256 Byte Offloading	1400 Byte	1400 Byte Offloading	
1	0.628	0.788	0.959	0.959	
5	0.628	0.904	3.799	4.794	
10	0.629	0.903	3.796	5.669	
20	0.629	0.900	3.802	5.660	
30	0.629	0.903	3.796	5.669	
40	0.630	0.903	3.801	5.675	
50	0.630	0.903	3.801	5.668	
60	0.629	0.900	3.799	5.660	
70	0.629	0.902	3.791	5.671	
80	0.629	0.900	3.795	5.667	
90	0.629	0.898	3.794	5.667	
100	0.629	0.901	3.798	5.658	

Tabelle B.1: Die Ergebnisse von Experiment 1. Zu sehen ist der erreichte Durchsatz bei verschiedenen Senderaten. Alle Werte sind in Gbit/s angegeben und auf drei Nachkommastellen gerundet.

Anzahl Tunnel	Multiple Tunnel	Multiple Tunnel Offloading	Single Tunnel	Single Tunnel Offloading	Multiple Tunnel IRQ	Multiple Tunnel IRQ, Offloading
1	0.073	0.085	0.082	0.085	0.076	0.085
2	0.104	0.142	0.083	0.114	0.108	0.138
4	0.164	0.240	0.112	0.149	0.130	0.151
8	0.199	0.294	0.087	0.146	0.147	0.171
1	2.877	2.876	2.877	2.876	2.877	2.877
2	4.071	5.606	3.344	5.082	3.810	5.749
4	6.437	7.616	4.373	6.538	5.377	8.659
8	7.999	11.300	3.447	6.445	5.128	9.101

Tabelle B.2: Die Ergebnisse von Experiment 4. Zu sehen ist der erreichte Durchsatz bei verschiedenen Tunnelanzahlen. Oben sind die Werte für Frames mit einer Größe von 68 Byte und unten für Frames mit einer Größe von 1400 Byte angegeben. Alle Werte sind in Gbit/s angegeben und auf drei Nachkommastellen gerundet.

Abbildungsverzeichnis

2.1	einfaches Ethernet-Netzwerk	4
2.2	Darstellung IPv4 und IPv6-Paketheader	5
2.3	einfaches IP-Netzwerk	7
2.4	Darstellung VPNs	8
2.5	IKEv2 Verbindungsaufbau	11
2.6	Encapsulating Security Payload	12
2.7	Kapselung mit ESP	13
2.8	StrongSwan und IPsec Komponenten	14
2.9	XFRM Paketfluss	16
2.10	Darstellung GCM	19
3.1	Aufbau Testnetzwerk	25
4.1	Durchsatz bei verschiedenen Senderaten	35
4.2	Durchsatz bei verschiedenen Senderaten	36
4.3	relativer Durchsatz bei verschiedenen Senderaten	36
4.4	Vergleich Durchsatz bei verschiedenen Framegrößen	37
4.5	Vergleich Frames pro Sekunde bei verschiedenen Framegrößen	38
4.6	Round-Trip-Delay bei verschiedenen Framegrößen	39
4.7	verwendete CPU-Zeit bei verschiedenen Framegrößen	40
4.8	Vergleich Durchsatz IMix	44
4.9	Durchsatz bei parallelen Datenströmen	47
4.10	CPU-Auslastung bei parallelen Datenströmen	48

Tabellenverzeichnis

2.1	Aufbau eines Ethernet-Frames	4
3.1	verwendeter IMix	30
4.1	Vergleich Durchsatz bei verschiedenen Framegrößen	38
4.2	Vergleich Frames pro Sekunde bei verschiedenen Framegrößen	39
4.3	Vergleich Durchsatz IMix	44
4.4	Vergleich übertragenen Frames im IMix	45
B.1	Messergebnisse Experiment 1	58
B.2	Messergebnisse Experiment 3	59

Glossar

AES, Advanced Encryption Standard ein symmetrisches Verschlüsselungsverfahren, siehe auch Teilabschnitt [2.6.1](#)

AES-GCM, GCM, Galois/Counter Mode ein Betriebsmodus für AES, zur Verschlüsselung und Authentifizierung langer Datensätze, siehe auch Teilabschnitt [2.6.2](#)

Authentizität eine Eigenschaft von Daten, die deren Ursprung (Sender) nachweist, häufig durch Prüfsummen realisiert

Durchsatz angegeben in Bit/s oder Frames/s, eine Messgröße die angibt, wie viele Daten pro Sekunde übertragen werden können, siehe auch Abschnitt [3.4](#), nicht zu verwechseln mit Senderate

ESP, Encapsulating Security Payload ein Protokoll welches die Authentizität, Integrität und Vertraulichkeit von Daten sicherstellt, siehe auch Teilabschnitt [2.4.3](#)

Ethernet eine Technologie zur Übertragung von Daten zwischen Computern in einem Netzwerk, siehe auch Abschnitt [2.1](#)

Hardware-Offloading ein Prozess bei dem bestimmte Aufgaben von speziell dafür vorgesehener Hardware übernommen werden, anstatt von der CPU, siehe auch Abschnitt [2.7](#)

ICV, Integrity Check Value, Prüfsumme eine Zahl, anhand derer die Integrität und Authentizität von Daten überprüft werden kann

IKE, IKEv2, Internet Key Exchange ein Protokoll zum Austausch von Schlüsseln und der Erstellung von Security Associations, siehe auch Teilabschnitt [2.4.2](#) und Teilabschnitt [2.5.1](#)

IP, Internet Protocol ein Protokoll zur Übertragung von Daten zwischen Computern und Netzwerken, siehe auch Abschnitt [2.2](#)

IP-Adresse eine Adresse zur Adressierung von Computern im Internet-Protocol

IPsec eine Architektur für das Internet Protocol, zur Erstellung von VPNs und Sicherung von übertragenen Daten, siehe auch Abschnitt [2.4](#)

Integrität eine Eigenschaft von Daten, die deren Echtheit und Korrektheit nachweist und Modifikationen verhindert, häufig durch Prüfsummen realisiert

Kapselung das Transportieren von Paketen als Payload anderer Pakete oder Frames

Latenz, RTD, Round-Trip Delay die Zeit die eine Nachrichteneinheit braucht, um vom Sender zum Empfänger (Latenz) oder zurück zum Sender (RTD) zu kommen, siehe auch Abschnitt [3.4](#)

- MAC-Adresse** eine Adresse zur Adressierung von Computern mit Ethernet
- Netzwerk, Subnetz** eine Zusammenfassung von mehreren, miteinander verbundenen Computern
- Netzwerkinterface** eine Repräsentation eines Ports einer Netzwerkkarte im Betriebssystem
- Netzwerkkarte, NIC** Hardware zum Anschluss eines Computers an ein Netzwerk und zur Übertragung von Daten
- Paket, Frame** eine Dateneinheit zur Übertragung, im Kontext von Ethernet wird von Frame gesprochen, sonst meist von Paket
- Payload** Nutzdaten, die in einem Frame oder Paket transportiert werden, können auch andere Pakete sein
- Routing** Weiterleitung von IP-Paketen an Netzwerkinterfaces, anhand von vorher definierten Regeln
- SA, Security Association** eine Sammlung von Informationen zur Verschlüsselung von Paketen, siehe auch Teilabschnitt [2.4.1](#)
- Senderate** angegeben in Bit/s oder Frames/s, eine Einheit die angibt, wie viele Daten pro Sekunde versendet werden, nicht zu verwechseln mit Durchsatz
- smartNIC** eine Netzwerkkarte mit extra Hardware, die Hardware-Offloading ermöglicht
- SP, Security Policy** eine Regel die beschreibt, wie IP-Pakete verschlüsselt werden sollen, siehe auch Teilabschnitt [2.4.1](#)
- SPI, Security Parameter Index** eine Zahl zur Identifizierung von Security Associations
- VPN, Virtual Private Network** ein nicht physisch existierendes Netzwerk zur sicheren Übertragung von Daten über unsichere Netzwerke, siehe auch Abschnitt [2.3](#)
- Vertraulichkeit** eine Eigenschaft von Daten, die das Mitlesen durch Dritte verhindert, häufig durch eine Verschlüsselung realisiert
- XFRM** ein Framework im Linux Network Stack, zur Verschlüsselung von IP-Paketen mit ESP, siehe auch Teilabschnitt [2.5.2](#)

Literaturverzeichnis

- [ADF⁺10] Kahraman D. Akdemir, Martin G. Dixon, Wajdi K. Feghali, Patrick G Fay, Vinodh Gopal, Jim Guilford, Erdinc Ozturk, Gil Wolrich, and Ronen Zohar. Breakthrough aes performance with intel® aes new instructions. 2010. Available from: <https://www.intel.com/content/www/us/en/developer/articles/technical/advanced-encryption-standard-instructions-aes-ni.html>.
- [Atk95a] R. Atkinson. IP Authentication Header. RFC 1826, Internet Engineering Task Force, August 1995. Available from: <http://www.rfc-editor.org/rfc/rfc1826.txt>.
- [Atk95b] R. Atkinson. IP Encapsulating Security Payload (ESP). RFC 1827, Internet Engineering Task Force, August 1995. Available from: <http://www.rfc-editor.org/rfc/rfc1827.txt>.
- [BM99] S. Bradner and J. McQuaid. Benchmarking Methodology for Network Interconnect Devices. RFC 2544, Internet Engineering Task Force, March 1999. Available from: <http://www.rfc-editor.org/rfc/rfc2544.txt>.
- [Bra91] S. Bradner. Benchmarking Terminology for Network Interconnection Devices. RFC 1242, Internet Engineering Task Force, July 1991. Available from: <http://www.rfc-editor.org/rfc/rfc1242.txt>.
- [CK74] V. Cerf and R. Kahn. A protocol for packet network intercommunication. *IEEE Transactions on Communications*, 22(5):637–648, 1974.
- [DH76] Whitfield Diffie and Martin E. Hellman. New directions in cryptography. *IEEE Trans. Inf. Theory*, 22:644–654, 1976.
- [DH17] Dr. Steve E. Deering and Bob Hinden. Internet Protocol, Version 6 (IPv6) Specification. RFC 8200, Internet Engineering Task Force, July 2017. Available from: <https://www.rfc-editor.org/info/rfc8200>.
- [DR99] Joan Daemen and Vincent Rijmen. Aes proposal: Rijndael. 09 1999.
- [Dro97] R. Droms. Dynamic Host Configuration Protocol. RFC 2131, Internet Engineering Task Force, March 1997. Available from: <http://www.rfc-editor.org/rfc/rfc2131.txt>.
- [Dwo07] Morris Dworkin. Recommendation for block cipher modes of operation: Galois/counter mode (gcm) and gmac. *NIST Special Publication 800-38D*, 11 2007.
- [FL06] V. Fuller and T. Li. Classless Inter-domain Routing (CIDR): The Internet Address Assignment and Aggregation Plan. RFC 4632, Internet Engineering Task Force, August 2006. Available from: <http://www.rfc-editor.org/rfc/rfc4632.txt>.

- [Gue10] Shay Gueron. Intel advanced encryption standard (aes) new instructions set, 05 2010. Available from: <https://www.intel.com/content/www/us/en/developer/articles/tool/intel-advanced-encryption-standard-aes-instructions-set.html>.
- [HC98] D. Harkins and D. Carrel. The Internet Key Exchange (IKE). RFC 2409, Internet Engineering Task Force, November 1998. Available from: <http://www.rfc-editor.org/rfc/rfc2409.txt>.
- [iee84] Ieee standards for local area networks: Carrier sense multiple access with collision detection (csma/cd) access method and physical layer specifications. *ANSI/IEEE Std 802.3-1985*, pages 1–146, 1984.
- [iee18] Ieee standard for ethernet. *IEEE Std 802.3-2018 (Revision of IEEE Std 802.3-2015)*, pages 1–5600, 2018.
- [Kau05] C. Kaufman. Internet Key Exchange (IKEv2) Protocol. RFC 4306, Internet Engineering Task Force, December 2005. Available from: <http://www.rfc-editor.org/rfc/rfc4306.txt>.
- [Ken05a] S. Kent. IP Authentication Header. RFC 4302, Internet Engineering Task Force, December 2005. Available from: <http://www.rfc-editor.org/rfc/rfc4302.txt>.
- [Ken05b] S. Kent. IP Encapsulating Security Payload (ESP). RFC 4303, Internet Engineering Task Force, December 2005. Available from: <http://www.rfc-editor.org/rfc/rfc4303.txt>.
- [KHN⁺14] C. Kaufman, P. Hoffman, Y. Nir, P. Eronen, and T. Kivinen. Internet Key Exchange Protocol Version 2 (IKEv2). RFC 7296, Internet Engineering Task Force, October 2014. Available from: <http://www.rfc-editor.org/rfc/rfc7296.txt>.
- [KS05] S. Kent and K. Seo. Security Architecture for the Internet Protocol. RFC 4301, Internet Engineering Task Force, December 2005. Available from: <http://www.rfc-editor.org/rfc/rfc4301.txt>.
- [Moy98] J. Moy. OSPF Version 2. RFC 2328, Internet Engineering Task Force, April 1998. Available from: <http://www.rfc-editor.org/rfc/rfc2328.txt>.
- [MvOV18] A.J. Menezes, P.C. van Oorschot, and S.A. Vanstone. *Handbook of Applied Cryptography*. Discrete Mathematics and Its Applications. CRC Press, 2018. Available from: <https://books.google.de/books?id=YyCyDwAAQBAJ>.
- [oSN01] U.S. National Institute of Standards and Technology (NIST). Announcing the advanced encryption standard (aes). *Federal Information Processing Standards Publication (FIPS PUBS) 197*, 11 2001.
- [Plu82] D. Plummer. Ethernet Address Resolution Protocol: Or Converting Network Protocol Addresses to 48.bit Ethernet Address for Transmission on Ethernet Hardware. RFC 0826, Internet Engineering Task Force, November 1982. Available from: <http://www.rfc-editor.org/rfc/rfc826.txt>.

- [Pos81] J. Postel. Internet Protocol. RFC 0791, Internet Engineering Task Force, September 1981. Available from: <http://www.rfc-editor.org/rfc/rfc791.txt>.
- [Res18] E. Rescorla. The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446, Internet Engineering Task Force, August 2018. Available from: <https://www.rfc-editor.org/info/rfc8446>.
- [RTM22] E. Rescorla, H. Tschofenig, and N. Modadugu. The Datagram Transport Layer Security (DTLS) Protocol Version 1.3. RFC 9147, Internet Engineering Task Force, April 2022. Available from: <https://www.rfc-editor.org/info/rfc9147>.
- [Tou13] J. Touch. Updated Specification of the IPv4 ID Field. RFC 6864, Internet Engineering Task Force, February 2013. Available from: <http://www.rfc-editor.org/rfc/rfc6864.txt>.
- [VM05] J. Viega and D. McGrew. The Use of Galois/Counter Mode (GCM) in IPsec Encapsulating Security Payload (ESP). RFC 4106, Internet Engineering Task Force, June 2005. Available from: <http://www.rfc-editor.org/rfc/rfc4106.txt>.